

RIEPILOGO OGNI ESERCIZIO

Nel presente documento affronterò alcuni argomenti che potrebbero comparire nei compiti scritti. La sezione sarà dedicata esclusivamente ai concetti trattati dal Prof. Giuseppe Bianchi, ovvero la parte relativa alle reti.

Per la parte di segnali ho già predisposto un documento specifico e dettagliato. Per ogni esercizio saranno presenti richiami teorici, in modo che tutti possano comprendere la teoria sottostante, poiché l'obiettivo non è imparare a memoria gli esercizi, ma conoscere i principi teorici che ne sono alla base.

Come per tutte le materie di ogni corso, i miei documenti non intendono in alcun modo sostituirsi al materiale del professore; al contrario, non mi permetterei mai di farlo: devono essere considerati esclusivamente come un supporto allo studio.

I concetti non sono particolarmente difficili; l'unica vera difficoltà di questa materia è la grande quantità e varietà degli esercizi, che sono davvero molto svariati. Il **Prof. Bianchi**, inoltre, ne propone spesso di nuovi, **dimostrando grande creatività**.

Per questo motivo, come piace a me, in questo documento non impareremo nulla a memoria, ma cercheremo di comprendere a fondo i concetti, così da essere pronti ad affrontare qualsiasi variazione nella tipologia degli esercizi.

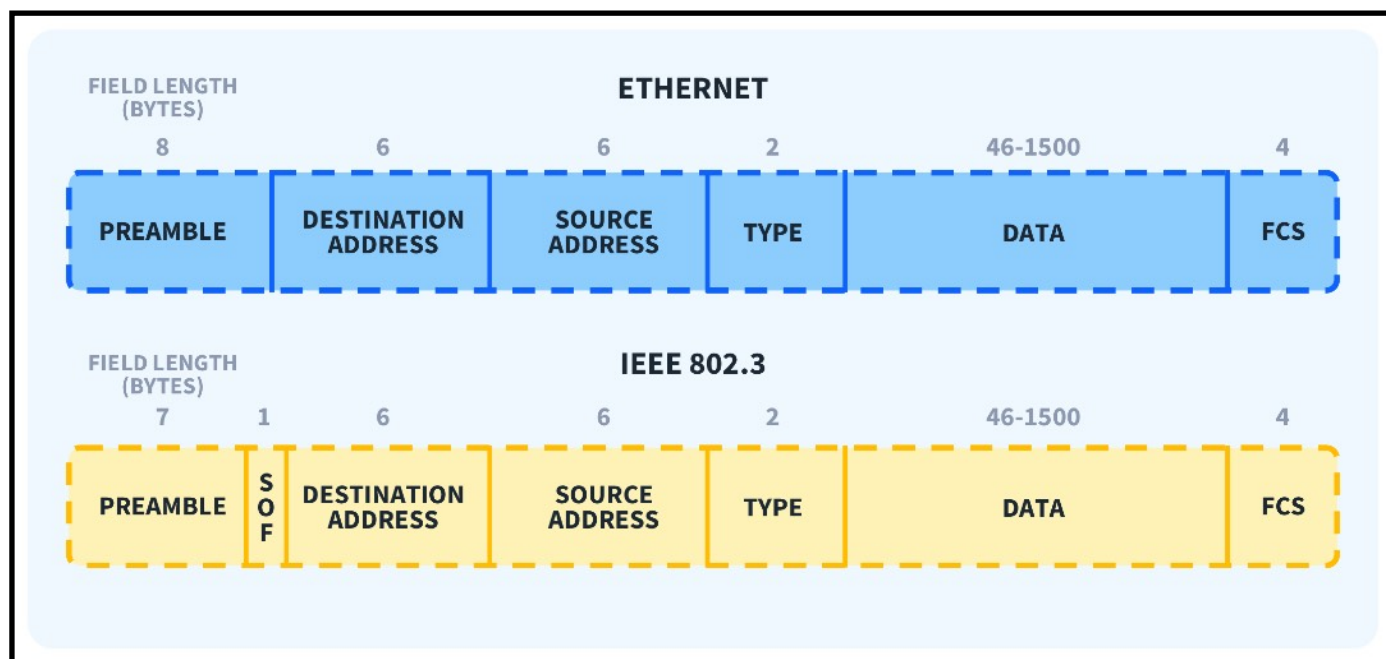
Tendo anche a esprimere un giudizio, sebbene possa essere soggettivo: la parte relativa alle reti è molto interessante e il professore è uno dei migliori che abbiamo qui a Tor Vergata.

Ora procediamo partendo dalle basi, facendo finta di non conoscere nulla.

L'unica rete che conosciamo è quella da pesca: da qui inizieremo il nostro percorso per capire, passo dopo passo, i concetti fondamentali.

Simone Remoli.

ETHERNET



Ethernet è una **tecnologia di rete** che permette a più dispositivi di comunicare tra loro all'interno di una **rete locale (LAN)**.

L'immagine sopra rappresenta il pacchetto Ethernet (più correttamente **frame Ethernet**).

Esso è composto da campi ben definiti, ognuno con una funzione precisa.

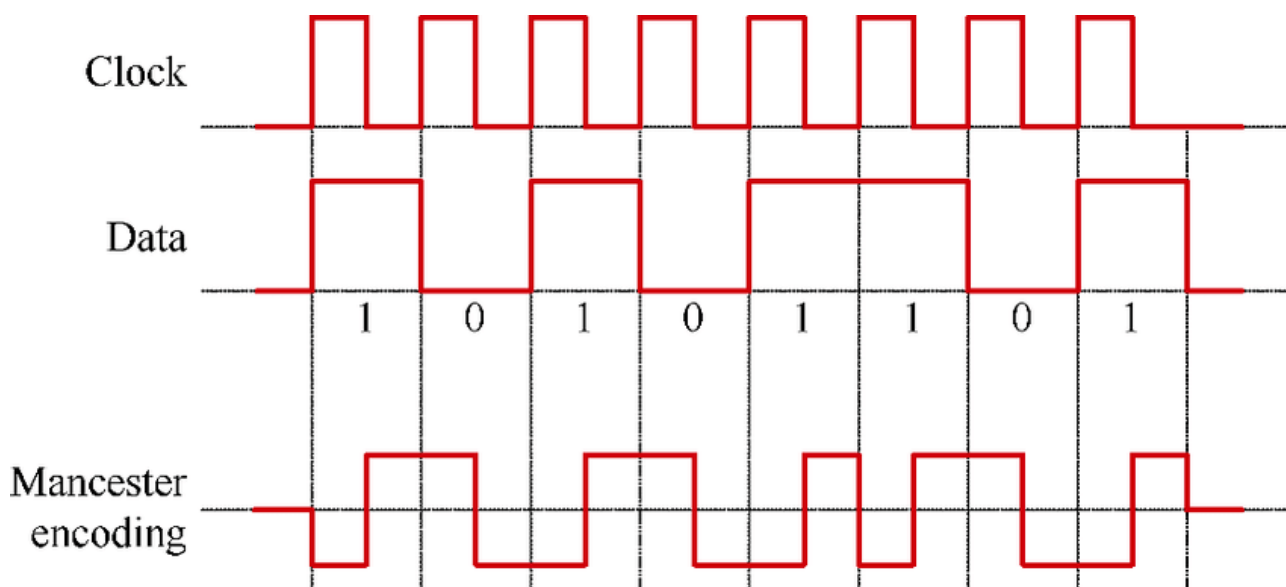
Il preambolo non fa parte della trama Ethernet vera e propria: è posto prima di essa e serve a realizzare la sincronizzazione a livello di **tempo di simbolo** tra trasmettitore e ricevitore.

Il suo scopo è quello di far capire al ricevitore quando "comincia" un pacchetto.

L'hardware della **scheda di rete (NIC)** utilizza il preambolo esclusivamente per riconoscere che da quel punto avrà inizio la trama Ethernet.

Il preambolo ha senso nel momento in cui il ricevitore si sincronizza attraverso la **codifica Manchester**.

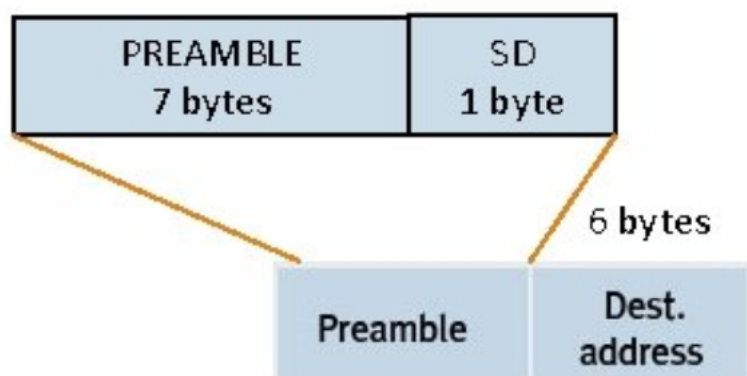
La codifica Manchester è una tecnica di codifica di linea in cui ogni bit è rappresentato da una transizione del segnale a metà del tempo di simbolo.



Il tempo di simbolo è la durata associata a un singolo bit trasmesso sul mezzo fisico. A **metà del tempo di simbolo** avviene **sempre** una transizione di livello (salita o discesa). Questo comporta una cosa **fondamentale** per il funzionamento della comunicazione: **il clock è incorporato nel segnale stesso**. Nel preambolo Ethernet, questa caratteristica viene sfruttata al massimo: la sequenza regolare di transizioni permette alla scheda di rete di **agganciarsi al tempo di simbolo** prima che inizi la trama vera e propria.

Come si riconosce l'inizio della trama Ethernet (SFD)?

Dopo 7 byte di preambolo (10101010 ripetuto), arriva l'ottavo byte, chiamato SFD – **Start Frame Delimiter**, che segnala l'inizio della parte utile della trama (MAC destinazione ecc.)



I **primi 7 bit** continuano il ritmo del preambolo (1010101) mentre l'ultimo bit è diverso (...11). Questa rottura dello schema regolare è il segnale chiave.

Cosa fa la scheda di rete (NIC)?

Durante il preambolo la NIC si sincronizza sul **tempo di simbolo** e non interpreta ancora i dati. Quando arriva l'SFD la NIC rileva la sequenza anomala, capisce che la sincronizzazione è completata e sa che **il prossimo bit sarà il primo campo della trama Ethernet** (MAC di destinazione).

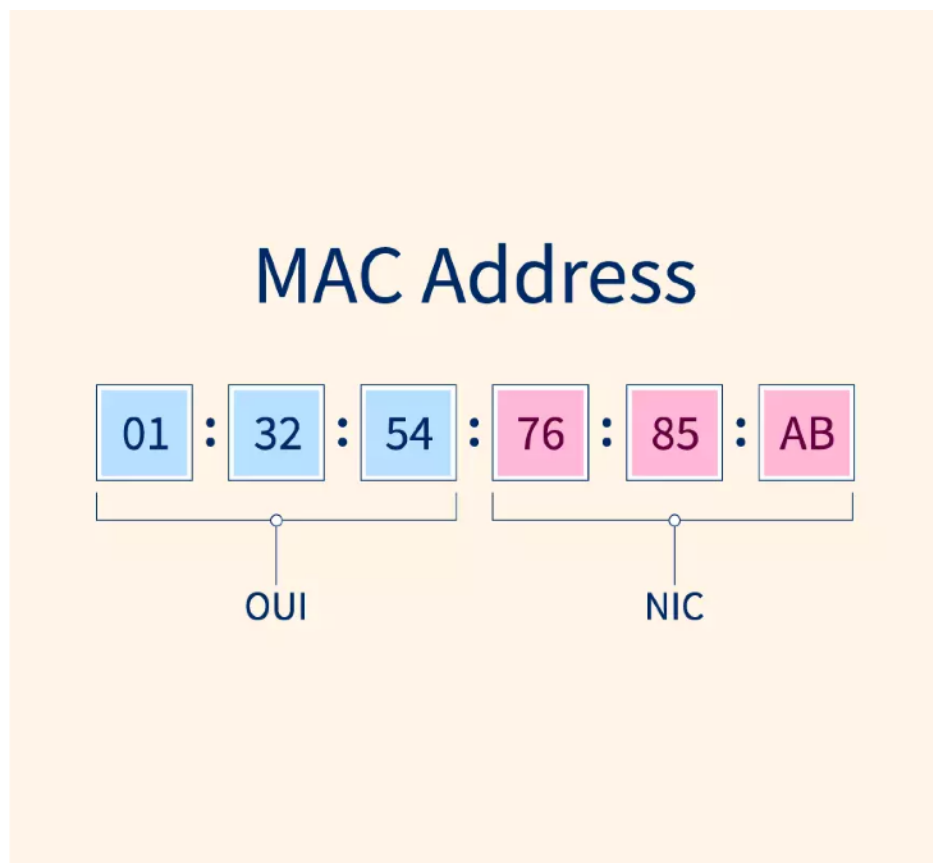
Quindi, per rispondere alla domanda: la trama Ethernet vera e propria viene riconosciuta grazie allo Start Frame Delimiter, un byte che interrompe la sequenza regolare del preambolo e segnala alla NIC l'inizio dei campi della trama.

Dopo lo Start Frame Delimiter, la NIC legge il campo di indirizzo MAC di destinazione e decide se la trama è rivolta a sé; in caso contrario, la scarta immediatamente. La NIC confronta l'indirizzo MAC di destinazione con il proprio MAC. Se **non corrisponde**, la trama viene **scartata subito** (**early discard**). Se **corrisponde**, la NIC continua a ricevere l'intera trama.

Questo meccanismo evita di far elaborare al sistema operativo dati **non destinati a lui**, migliorando efficienza e prestazioni.

Che cos'è un MAC Address?

Un **MAC address** (Media Access Control) è un **indirizzo fisico univoco** assegnato a ogni **scheda di rete (NIC)**, usato da Ethernet per identificare in modo univoco i dispositivi **all'interno di una rete locale (LAN)**. È lungo **48 bit (6 byte)**, scritto in esadecimale.



Un MAC address è composto da 48 bit (6 byte) ed è strutturato in due parti ben distinte:

1. **OUI – Organizationally Unique Identifier (24 bit)**

- Sono i **primi 3 byte**
- Identificano il **produttore** della scheda di rete (es. Intel, Realtek, Apple)

- Vengono assegnati dall'IEEE

2. NIC Specific (24 bit)

- Sono gli **ultimi 3 byte**
- Identificano in modo univoco **quella specifica interfaccia** prodotta dal costruttore

A livello Ethernet, il MAC serve esclusivamente per l'**indirizzamento locale** all'interno della LAN.

Cosa succede dopo?

Se il MAC di destinazione è valido la NIC accetta la trama. Non viene scartata e **la scheda continua a riceverla fino alla fine**. Quindi legge l'indirizzo della scheda di rete che sta trasmettendo il pacchetto.

Dopodiché la NIC legge il campo **Type** (2 byte).

Campo Type: che cos'è?

Indica **quale protocollo di livello superiore** è incapsulato. **Ethernet non interpreta il contenuto, si limita a leggere il Type e a consegnare i dati al modulo giusto.**

Ogni livello fa **solo il suo lavoro**.

Come stanno davvero le cose?

In realtà ora allarghiamo i nostri orizzonti. Le reti non sono fatte da un unico blocco che fa tutto. Sono organizzate in **livelli**, ognuno con un **compito preciso**.



Senza nomi complicati, possiamo dire che esistono:

1. Un livello che si occupa del mezzo fisico: segnali elettrici, cavi, bit.
2. Un livello data link che collega i dispositivi nella stessa rete locale: indirizzi fisici, invio delle trame (**qui vive Ethernet**).
3. Un livello network che permette di comunicare tra reti diverse indirizzi logici, instradamento (qui vive IP).
4. Un livello trasporto che gestisce la comunicazione tra programmi. Affidabilità, ordine, controllo (qui vive TCP).
5. Un livello applicativo ossia ciò che l'utente usa davvero (siti web, messaggi, login...).

Quando un'informazione deve viaggiare, il livello più alto crea i dati, il livello sotto li prende e li inserisce in una struttura più grande. Questo processo si ripete scendendo di livello.

Questo si chiama **incapsulamento**.

Ethernet è il livello che prende i dati prodotti dai livelli superiori, li inserisce in una **trama** e permette la trasmissione sulla **rete locale**.

Le reti sono organizzate a livelli: **ogni livello incapsula i dati del livello superiore e utilizza i servizi del livello inferiore**.

Quindi, tornando al campo type.

Il campo Type della trama Ethernet indica quale protocollo di livello superiore è incapsulato nel payload, permettendo la corretta decapsulazione dei dati.

Il **payload** è **l'insieme dei dati incapsulati all'interno della trama Ethernet**, che vengono trasportati ma non interpretati dal livello Ethernet. Ethernet aggiunge il proprio header (MAC, Type, ecc.) tutto ciò che sta dopo è il payload, quel payload appartiene ai **livelli superiori**.

Cosa succede in ricezione (concettualmente)

1. Arriva una **trama Ethernet**
2. La NIC la accetta (MAC valido, CRC ok)
3. Ethernet rimuove il proprio header
4. Rimane un blocco di byte **indifferenziato**
5. Il campo **Type** indica **come interpretare quei byte**

Infine esiste il campo FCS.

Il FCS (Frame Check Sequence) è un campo di 4 byte posto alla fine della trama Ethernet.

Serve a **verificare l'integrità dei dati trasmessi**, cioè a capire se la trama è arrivata **senza errori**.

Il CRC (Cyclic Redundancy Check), nel campo FCS, è un codice di controllo usato per rilevare errori nella trasmissione di una trama Ethernet.

Come funziona (a grandi linee)

1. **Il mittente** calcola un valore (CRC) su:
 - header Ethernet (destination, source e type)
 - payload
2. Inserisce questo valore nel campo **FCS**.
3. **Il ricevitore** ricalcola lo stesso valore sui dati ricevuti.
4. Confronta:
 - se i valori coincidono: trama **corretta**
 - se differiscono: trama **scartata**

Questo è importante perché il mezzo fisico può introdurre errori (rumore, interferenze). Il FCS permette di scartare trame corrotte ed evitare che dati errati salgano ai livelli superiori. È un controllo rapido ed efficiente, fatto a livello hardware.

Ethernet è un protocollo UNRELIABLE, la trasmissione del pacchetto si dà per scontato che vada a buon fine.

Ethernet **non corregge gli errori, li rileva soltanto.**

Abbiamo concluso una prima parte teorica, volutamente più generale e introduttiva, che aveva lo scopo di presentare la materia e fornire una visione d'insieme degli argomenti trattati. Era l'unico modo per inquadrare, a grandi linee, il contesto di cui stiamo parlando.

Ora possiamo procedere in modo più diretto, entrando nel merito del funzionamento del protocollo.

FUNZIONAMENTO TRASMISSIONE ETHERNET

1. La prima regola è “ascoltare prima di trasmettere”.

La stazione effettua quindi un'operazione di **carrier sense**: prima di inviare dati, ascolta il mezzo di trasmissione e verifica se qualcun altro sta già trasmettendo. Se il mezzo risulta occupato, la stazione **posticipa la trasmissione**, evitando così interferenze e collisioni.

Quando un'applicazione genera dati, questi vengono incapsulati nei vari livelli fino a formare una trama Ethernet. Prima della trasmissione, la scheda di rete effettua un'operazione di carrier sense per verificare che il mezzo sia libero. Una volta rilevato il canale libero, deve trascorrere un intervallo di tempo minimo, detto **Inter Frame Spacing (IFS)**, dopo il quale la stazione può iniziare la trasmissione.

Le operazioni di carrier sense e l'intervallo di Inter Frame Spacing sono meccanismi utilizzati **esclusivamente in fase di trasmissione** e non durante la ricezione delle trame.

Quando un host accede a un sito web, i dati vengono incapsulati in una trama Ethernet e trasmessi solo dopo aver verificato che il mezzo sia libero e dopo l'intervallo di Inter Frame Spacing; il destinatario riceve la trama senza effettuare carrier sense e procede al decapsulamento.

Quando dal tuo PC vai su un sito web: **IP di destinazione è quello del server, MAC di destinazione è quello del gateway/router della tua LAN.**

Quando è il MAC del server?

Solo se il server è nella tua stessa LAN. In quel caso: MAC di destinazione = MAC del server.

2. Si esegue un'operazione detta “**collision detection**”. Quando una stazione termina la trasmissione e due stazioni, B e C, intendono trasmettere, non esiste una regola che stabilisca a priori quale delle due intervenga per prima: entrambe possono iniziare quasi simultaneamente. Il connettore Ethernet è in grado di trasmettere e ascoltare contemporaneamente, poiché dispone di una linea dedicata alla trasmissione e di una separata per la ricezione. Se la stazione B inizia a trasmettere e rileva, durante la trasmissione, la presenza di un altro segnale sul mezzo, viene individuata una collisione e **entrambe** le stazioni interrompono la trasmissione.

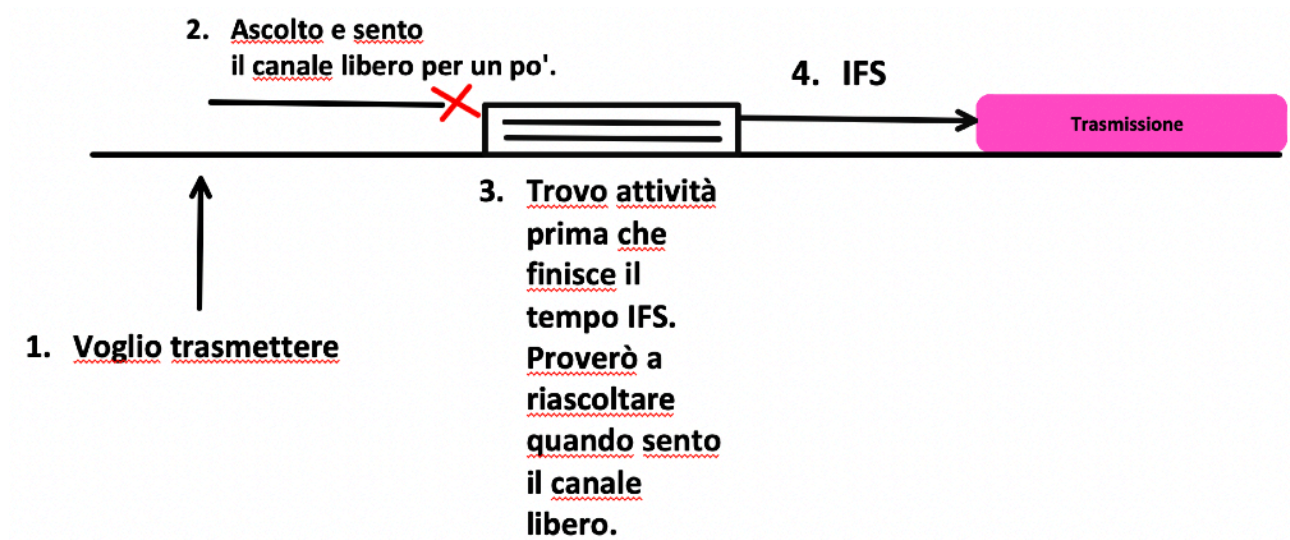
3. Il conflitto viene risolto mediante un **backoff casuale**. Quando entrambe le stazioni rilevano una collisione, ciascuna attende un intervallo di tempo casuale prima di ritentare la trasmissione. Poiché questi intervalli di attesa sono, con elevata probabilità, diversi tra loro, la stazione che termina per prima il proprio backoff riesce a trasmettere e vince la contesa per l'accesso al mezzo.

Cominciamo a fare qualche calcolo.

Era stato standardizzato che 96 bit fosse il minimo IFS da usare a 10 Mbps.

$$\frac{96 \text{ bit}}{10 \text{ Mbps}} = 9,6 \mu s$$

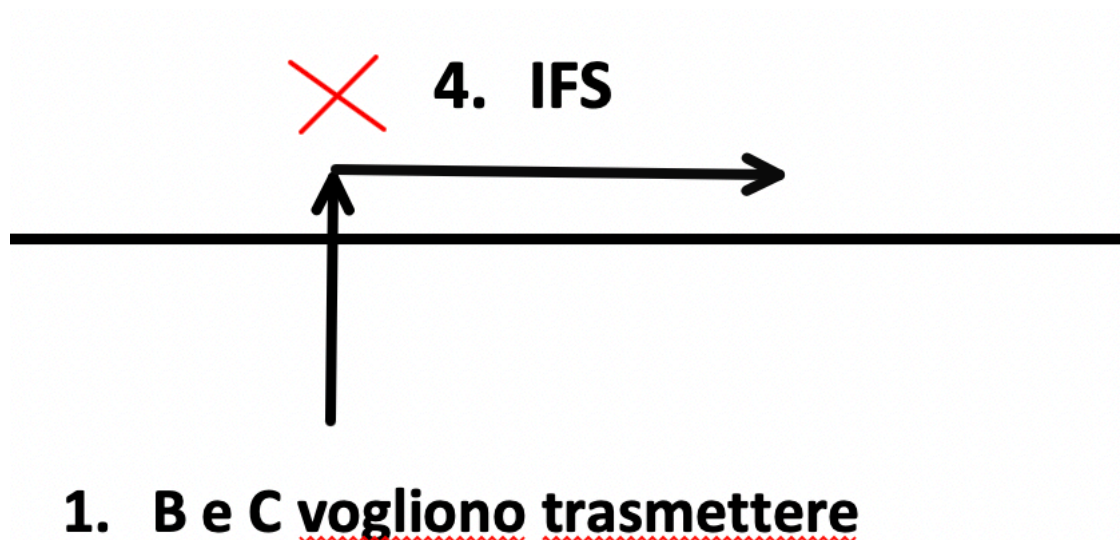
Quindi un tempo molto piccolo.
Riassumendo:



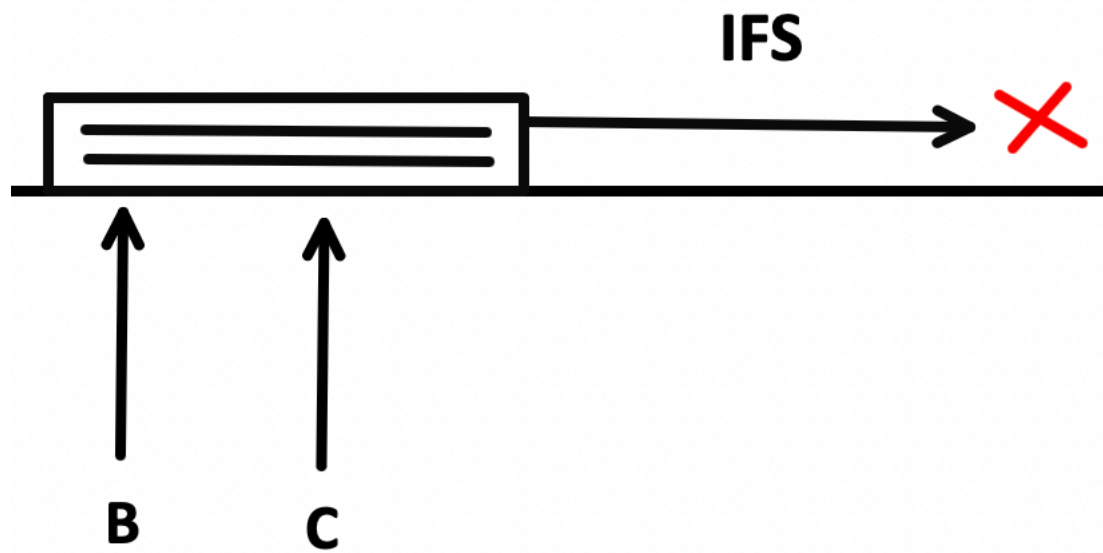
Quindi, se sento il canale occupato aspetto comunque un IFS.

Ma come si può collidere?

Due stazioni B e C possono “attivarsi” nello stesso istante di tempo e, di conseguenza, attendere lo stesso intervallo di IFS e iniziare a trasmettere contemporaneamente.

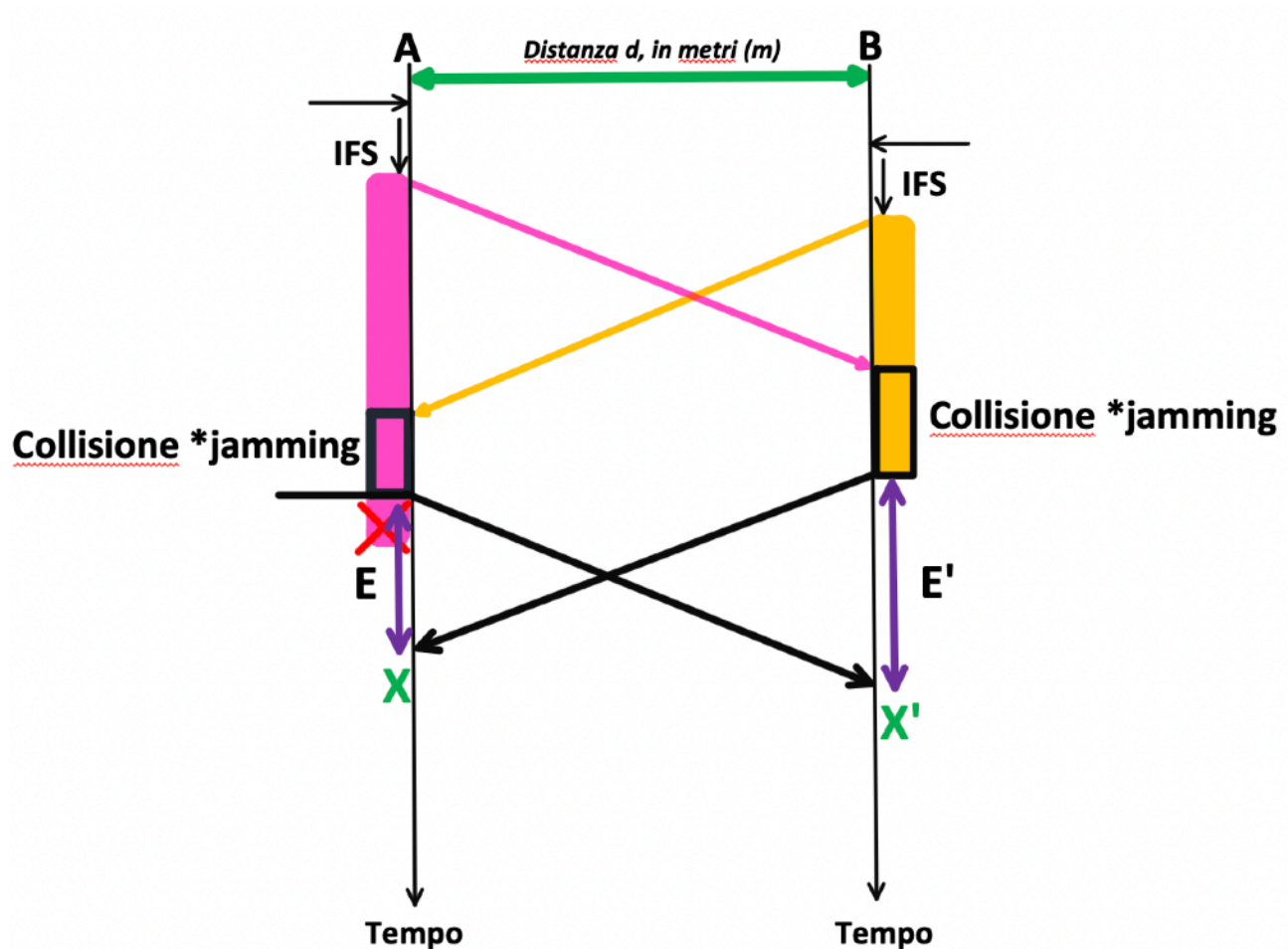


In alternativa, il mezzo trasmissivo può essere già occupato da una trasmissione, mentre le stazioni B e C si attivano in istanti di tempo differenti.



È vero che le stazioni si sono attivate in istanti diversi; tuttavia entrambe ascoltano il mezzo e iniziano ad attendere. La stazione B ascolta per un tempo T_1 , mentre la stazione C ascolta per un tempo T_2 . La presenza del pacchetto sul mezzo le sincronizza, **poiché entrambe attendono lo stesso intervallo di IFS** e avviano quindi la trasmissione simultaneamente, causando una collisione.

Nel mondo reale, tuttavia, il funzionamento non è esattamente questo: stiamo infatti trascurando un concetto fondamentale, legato ai **tempi di propagazione** del segnale sul mezzo trasmissivo.



Ora tutto assume un senso. Questo disegno è fondamentale: comprenderlo è necessario per comprendere il corso.

Nella stazione A, dopo aver atteso il tempo di IFS, si inizia a trasmettere. Quando A invia il primo bit del proprio pacchetto (rappresentato in viola), questo non si propaga istantaneamente verso il ricevitore, ma impiega un certo tempo di propagazione. Nel frattempo, mentre il pacchetto di A si sta propagando sul mezzo, la stazione B invia a sua volta il proprio pacchetto, poiché in quell'istante percepisce il canale come libero e ritiene che nessun'altra stazione stia trasmettendo. Il primo bit trasmesso da A raggiunge B mentre B sta già trasmettendo: in quel momento, sulla stazione B viene rilevata una collisione.

Una volta rilevata la collisione, la stazione la rafforza inviando la cosiddetta **sequenza di jamming**: al termine di questa sequenza, la trasmissione viene interrotta.

Che cos'è la sequenza di jamming?



La sequenza di jamming è una **sequenza di bit** trasmessa dopo il rilevamento di una collisione, con lo scopo di **assicurare che tutte le stazioni sul mezzo ne vengano a conoscenza**.

Quando una stazione rileva una collisione non interrompe immediatamente la trasmissione, anzi, invia per un breve intervallo una **sequenza di jamming** e solo dopo termina la trasmissione.

La stazione A sentirà il canale libero quando B smetterà di trasmettere, quindi al tempo **X**.

La stazione B sentirà libero al tempo **X'**.

E ed E' corrispondono ai tempi, rispettivamente per le due stazioni, nei quali ciascuna rileva che il canale risulta occupato.

Ora cominciamo a fare dei conti rapidi. Il tempo di propagazione lo possiamo calcolare.

La velocità della luce è:

$$(\approx 300.000 \text{ km/s})$$

Ovvero:

$$3 \times 10^8 \text{ m/s}$$

Sapendo che:

$$1 \text{ ms} = 10^{-3} \text{ s}$$

Ottengo:

$$3 \times 10^5 \text{ m/ms}$$

Ossia:

$$300 \text{ km/ms}$$

Ossia:

$$300 \text{ m}/\mu\text{s}$$

Nel cavo, invece, l'attenuazione della velocità di propagazione è:

$$200.000 \text{ km/s} = 2 \times 10^8 \text{ m/s}$$

Pertanto:

$$\frac{2 \times 10^8 \text{ m}}{10^6 \mu\text{s}} = 200 \text{ m}/\mu\text{s}$$

A questo punto, se la distanza $d=5 \text{ km}$, ne passano di microsecondi. E calcoliamoli allora. Se la velocità di propagazione nel cavo è $200 \text{ m}/\mu\text{s}$ e la distanza è $5\text{km}=5000\text{m}$, il tempo di propagazione è:

$$t = \frac{d}{v} = \frac{5000 \text{ m}}{200 \text{ m}/\mu\text{s}} = 25 \mu\text{s}$$

Quindi il segnale impiega **25 microsecondi** per percorrere 5 km di cavo.

Ma che succede se la trama è troppo corta?

È possibile che due stazioni vadano in collisione senza che **entrambe** se ne accorgano.

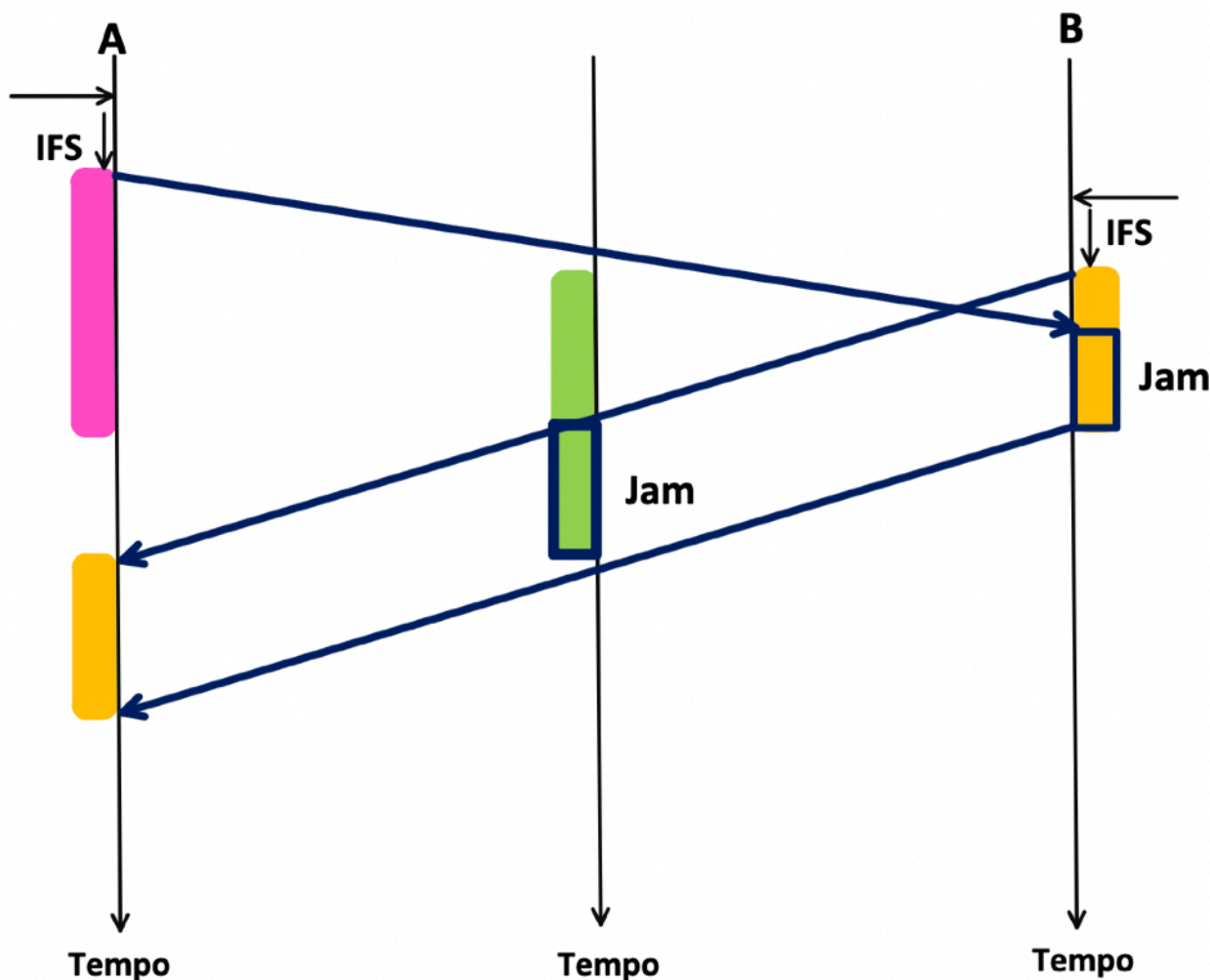
Il motivo è legato ai **tempi di propagazione** e alla **durata delle trame**.

Nel caso appena descritto sono state considerate trame sufficientemente lunghe sia per la stazione A sia per la stazione B. Supponiamo che il punto in cui la collisione avviene sia una stazione intermedia. La stazione A inizia a trasmettere e il suo segnale si propaga sul mezzo; successivamente anche la stazione B inizia a trasmettere e il suo segnale si propaga.

La collisione avviene nel punto centrale nel momento in cui B sta trasmettendo.

La stazione B rileva la collisione perché il segnale proveniente da A la raggiunge mentre è ancora in trasmissione; di conseguenza, B rafforza la collisione inviando la **sequenza di jamming**. Tuttavia, la stazione A potrebbe **non rilevare la collisione**, poiché la sua trama è già terminata prima che la sequenza di jamming generata da B riesca a propagarsi fino ad essa.

Questo scenario mostra che, se la trama è troppo corta rispetto al **tempo di propagazione**, **una collisione può verificarsi senza essere rilevata da tutte le stazioni coinvolte.**



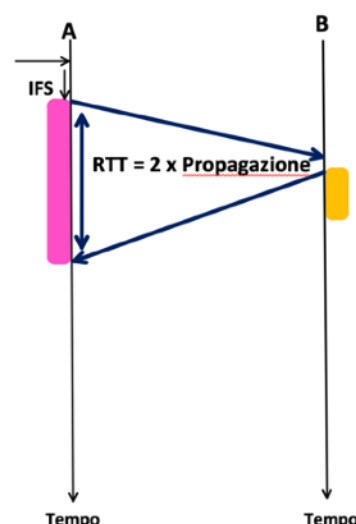
Questo va a violare la regola del collision detection: se sento la collisione io devo bloccarmi e ritrasmettere.

Se le trame avessero una lunghezza minima potrei risolvere questo problema.

Sto per arrivare ad un concetto di fondamentale importanza nella trasmissione.

Supponiamo di imporre che la trama A (viola immagine a destra) abbia una **dimensione minima di 64 byte**. Consideriamo due stazioni. La stazione A inizia a trasmettere e la stazione B non percepisce immediatamente il primo bit trasmesso da A; di conseguenza, dopo aver atteso l'IFS, B avvia la propria trasmissione, non avendo ancora avuto il tempo di rilevare l'eventuale collisione. Successivamente, però, il segnale di A raggiunge B mentre quest'ultima è in trasmissione: a quel punto B **rileva la collisione** e interrompe la trasmissione.

In A, non devo finire prima che il primo bit sia tornato. Il tempo di trasmissione di una trama di lunghezza minima deve essere uguale al **tempo di andata e ritorno del segnale**, questa propagazione di andata e ritorno si chiama **RTT**.



Nel contesto di Ethernet e delle collisioni, il RTT è fondamentale perché una stazione deve rimanere in trasmissione **almeno per un tempo pari al RTT** per essere certa di poter rilevare una collisione che avvenga nel punto più distante del mezzo.

Quindi il problema lo abbiamo risolto, ora riusciamo a rilevare la collisione in A. La stazione A riesce a rilevare la collisione perché, grazie alla lunghezza minima della trama Ethernet, rimane in trasmissione per un tempo almeno pari al RTT, consentendo al segnale di collisione di propagarsi e raggiungerla prima della fine della trasmissione.

Quindi, possiamo dire che la condizione per salvarci da questa situazione (**ossia per rilevare la collisione da parte di A**) è che il tempo di trasmissione di una trama di lunghezza minima sia maggiore uguale al tempo di propagazione andata e ritorno.

$$T_{tx} \geq RTT$$

Dove:

$$RTT = 2 \cdot T_p = 2 \cdot \frac{d}{v}$$

d è la distanza tra le due stazioni in metri (diametro della rete), v è la velocità di propagazione del segnale nel mezzo, e T_p è il tempo di propagazione di sola andata.

Occhio ai conti.

Se standardizzo una dimensione minima della trama (cosa che è stata fatta da ethernet) a 64 bytes, ossia 512 bit, quanto tempo impiego a trasmettere 512 bit in 10 Mbps?

$$51,2 \mu s$$

Questo è esattamente T_{tx} (ossia il tempo di trasmissione di una trama se vado a 10 Mbps).

Adesso, seguiamo la relazione a 10 Mbps:

$$\underbrace{51,2 \mu s}_{T_{tx}} \geq \underbrace{\left(2 \cdot \frac{d \text{ (m)}}{200 \text{ m}/\mu s} \right)}_{RTT}$$

Da qui si ricava che:

$$d \leq 5120 \text{ m}$$

La distanza viene ad essere 5 km circa.

Quindi d rappresenta la massima estensione fisica della rete (o del cavo Ethernet) tale per cui il tempo di trasmissione minimo della trama è sufficiente a coprire il Round Trip Time del segnale.

Quindi stiamo ricavando la distanza massima consentita del mezzo trasmissivo affinché una collisione possa essere rilevata.

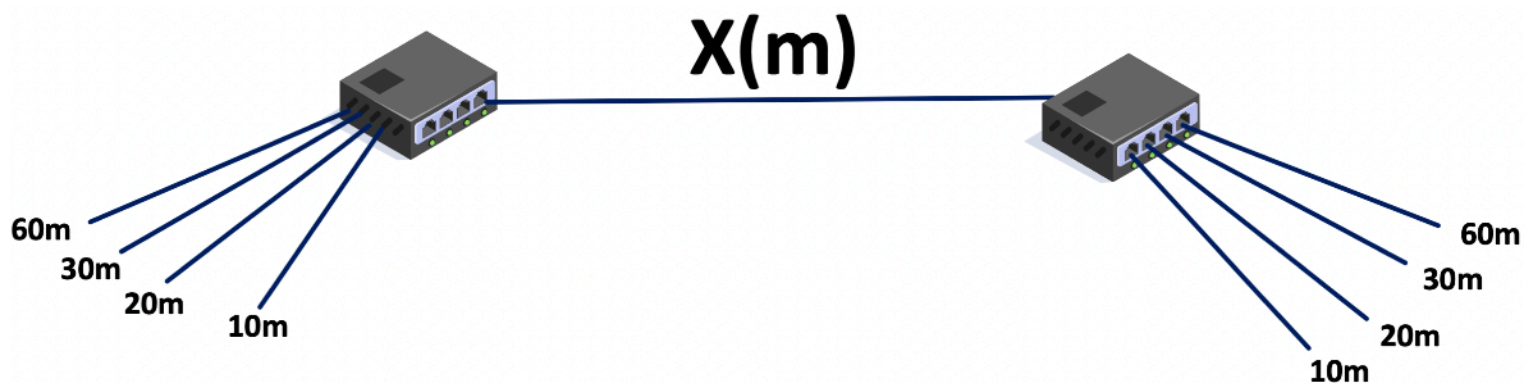
Ma se vado a 100 Mbps quanto è la distanza in rete?

$$d \leq 512 \text{ m}$$

Se vado più veloce il tempo di trasmissione si accorcia.

ESERCIZIO D'ESAME SUL DIAMETRO DELLA RETE

Per la rete Ethernet a **100 Mbps** in figura, si dica qual è il valore massimo (in metri) della lunghezza X, assumendo che i due hub introducano rispettivamente un ritardo pari ad **1 microsecondo e 0.5 microsecondi**. La dimensione della trama è di 64 bytes.



Lo scopo è capire la distanza. Perché la distanza che deve essere coperta non è solo X, ma anche 60m per lo switch a sinistra e 60m per lo switch a destra.

Quindi, utilizziamo la formula del **Round Tripe Time**.

$$T_{tx} \geq RTT$$

Quindi:

$$\underbrace{\frac{512 \text{ bit}}{100}}_{\mu s} \geq \left(2 \cdot \frac{x + 60 + 60}{200 \text{ m}/\mu s} + 1 \mu s + 0,5 \mu s \right)$$

Da qui si ottiene:

$$x \leq 242 \text{ m}$$

Questo valore rappresenta la **massima distanza x** consentita affinché la collisione venga rilevata entro il tempo minimo di trasmissione della trama.

Backoff esponenziale

Il **backoff esponenziale** è il meccanismo con cui Ethernet risolve le collisioni **dopo che sono state rilevate**, evitando che le stazioni coinvolte collidano di nuovo immediatamente.

Dopo una collisione le stazioni non ritentano subito a trasmettere, ma attendono un tempo di attesa casuale, che **aumenta esponenzialmente** se le collisioni continuano.

Il termine esponenziale deriva dal concetto che dopo la **prima collisione** l'attesa è breve, se le collisioni si ripetono, l'intervallo possibile **raddoppia**, riducendo drasticamente la probabilità di nuove collisioni.

Come funziona?

1. Avviene una collisione.
2. Ogni stazione incrementa il **contatore di collisioni k**.
3. La stazione sceglie un numero casuale r tale che:

$$r \in \{0, 1, 2, \dots, 2^k - 1\}$$

4. Il tempo di attesa è:

$$\text{Backoff} = r \cdot \text{slot time}$$

Lo **slot time** in Ethernet è **512 bit time**. Lo **slot time** è un **intervallo di tempo fisso** definito dallo standard Ethernet.

Regola pratica: all'inizio trasmetto veloce, se c'è una collisione vuol dire che la rete è piena e quindi comincio a rallentare di più.

ELENCO COMPLETO DI ESERCIZI D'ESAME E SOLUZIONE

Prova d'esame del 4 luglio 2018

Un **bridge** è configurato con il parametro *aging time* = 180s.

Il forwarding database del bridge è **inizialmente vuoto**; al bridge arrivano le seguenti trame:

T = 000 porta = 1 src = 11 dest = broadcast

T = 060 porta = 2 src = 33 dest = 11

T = 120 porta = 3 src = 44 dest = 22

T = 200 porta = 4 src = 33 dest = 66

T = 210 porta = 3 src = 22 dest = 11

T = 220 porta = 2 src = 22 dest = 22

1. Qual è il contenuto del forwarding database del bridge al tempo T = 230?
2. Una stazione in ascolto sulla quinta porta eventuale, quali trame sentirebbe?

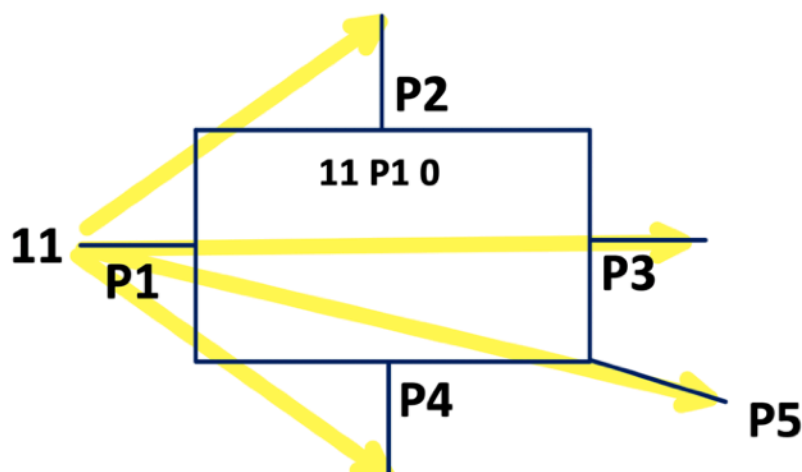
Svolgimento.

Il **forwarding database** è una tabella interna allo switch che contiene le informazioni sulle porte di uscita verso cui inoltrare una trama. Inizialmente il forwarding database è vuoto e il dispositivo apprende autonomamente la posizione delle stazioni nel tempo, osservando il traffico che attraversa le sue porte.

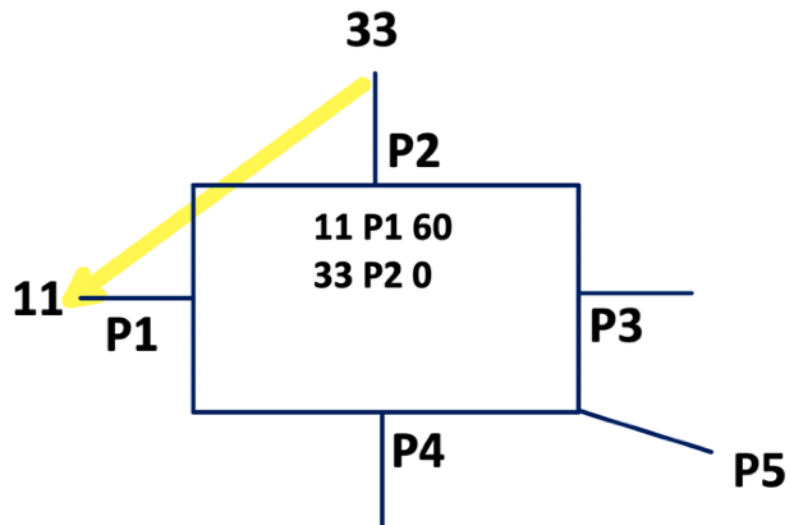
L'aging time rappresenta il tempo trascorso dall'ultima volta in cui una stazione è stata "rilevata" dallo switch. Poiché più un'informazione è datata, meno è probabile che la stazione si trovi ancora nella stessa posizione, viene fissato un limite massimo per questo intervallo. Se entro tale tempo lo switch non riceve più alcun pacchetto associabile a una determinata porta, il relativo record viene rimosso dal forwarding database.

Ora risolviamo l'esercizio.

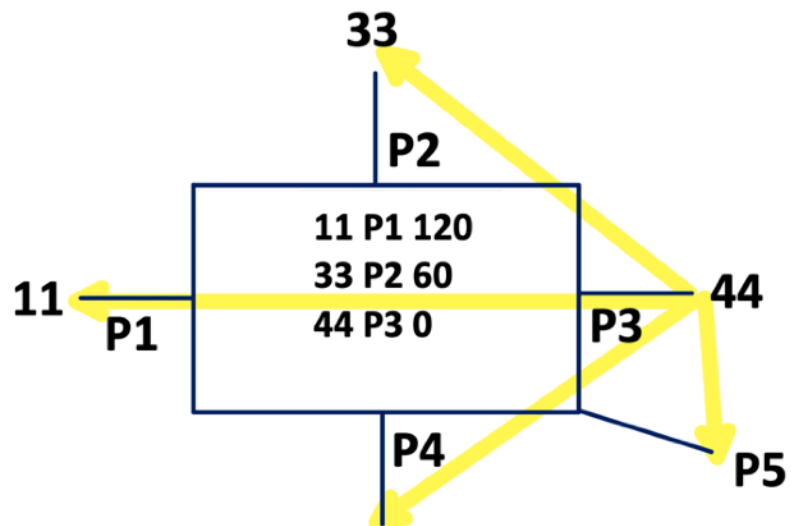
T = 0 mi registro che dietro la porta 1 c'è la stazione 11.



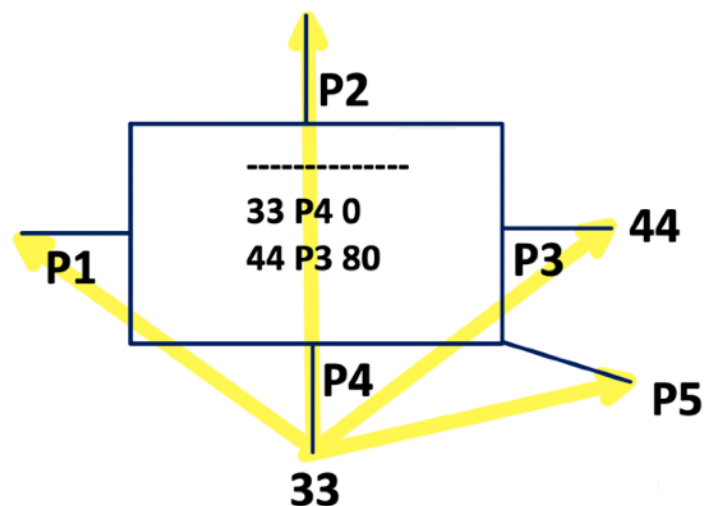
T = 60 mi registro che dietro la porta 2 c'è la stazione 33.



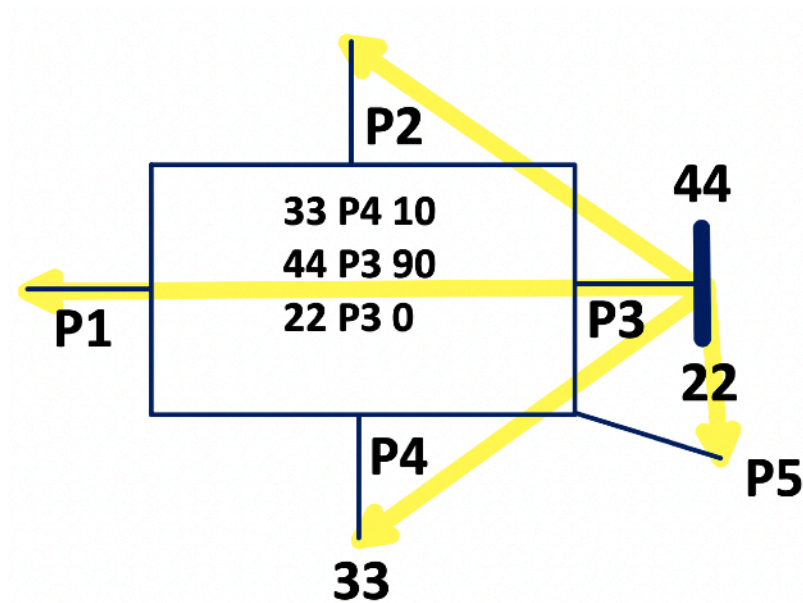
T = 120 mi registro che dietro la porta 3 c'è la stazione 44.



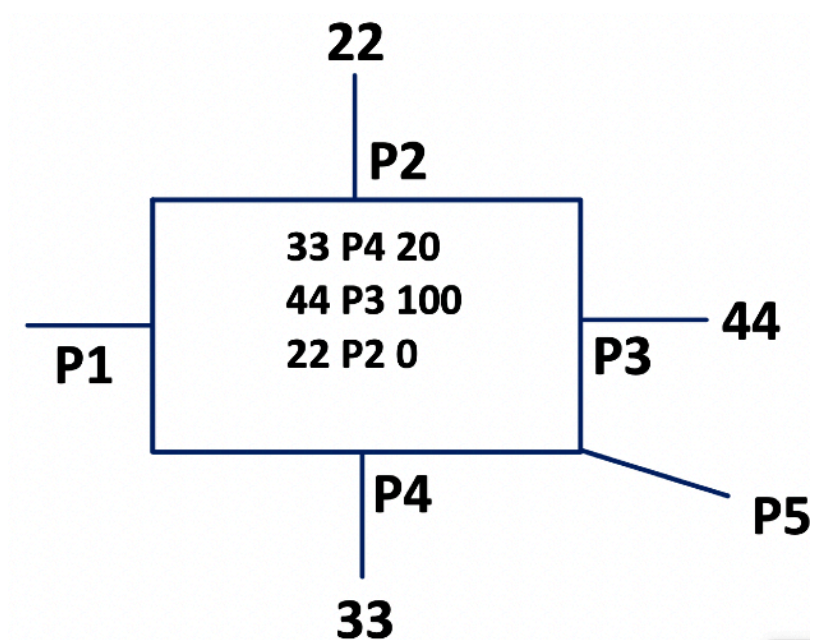
T = 200 la stazione 33 ha cambiato porta! Non è più dietro alla porta P2 ma dietro la porta P4. Per alcune entry l'aging time è scaduto quindi invalidiamo il record.



T = 210 mi registro che dietro la porta 3 c'è la stazione 22.



T = 220 mi registro che la sorgente 22 si è spostata dalla porta 3 alla porta 2.



Risposta 1:

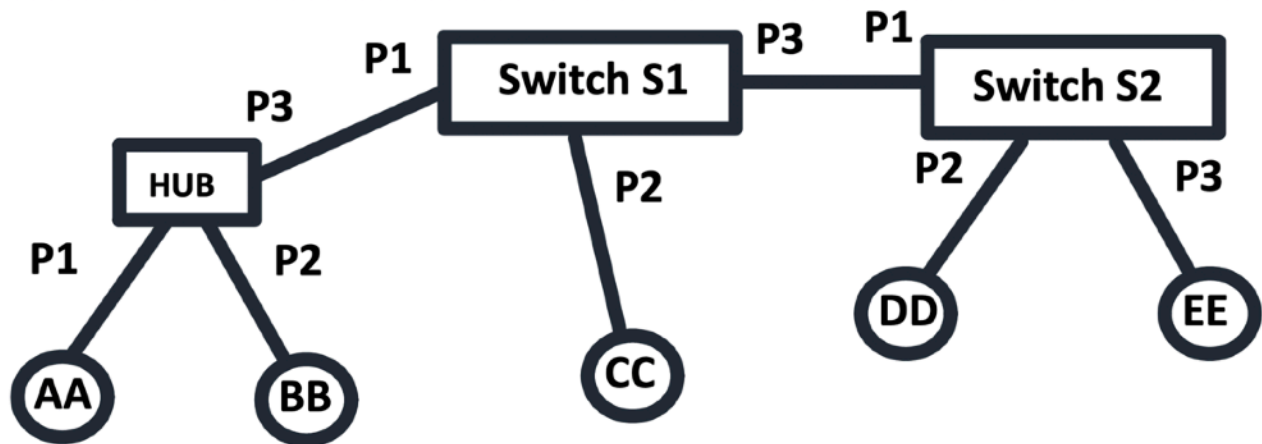
33 P4 20
44 P3 100
22 P2 0

Risposta 2: trama 1,3,4,5.

Prova d'esame del 10 giugno 2014

Con riferimento alla rete indicata si assuma che:

1. Al tempo $T = 1$ la stazione AA $\rightarrow$ CC.
2. Al tempo $T = 2$ la stazione BB $\rightarrow$ AA.
3. Al tempo $T = 3$ la stazione DD $\rightarrow$ CC.
4. Al tempo $T = 4$ la stazione DD $\rightarrow$ AA.
5. Al tempo $T = 5$ la stazione DD $\rightarrow$ BB.



Tralasciando l'aging time, assumendo che al tempo 0 tutti i forwarding database siano vuoti, e indicando le trame con il numero sopra riportato, si chiede di:

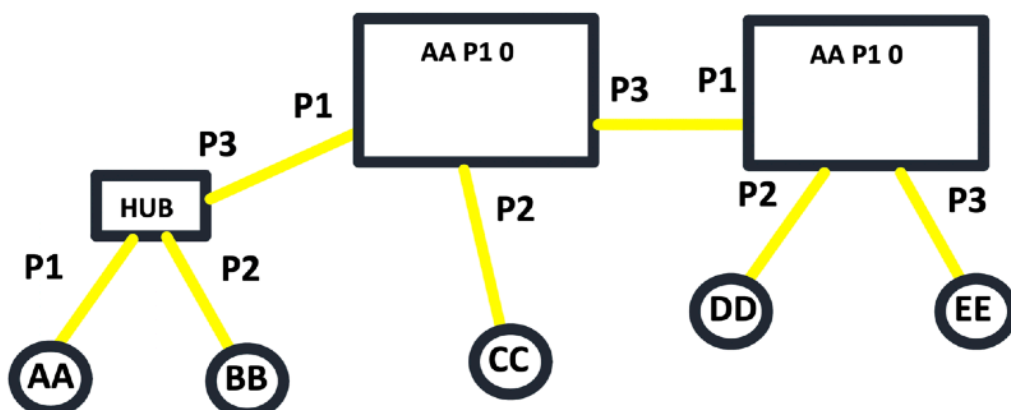
1. Dire quali trame vengono ricevute dalla stazione EE.
2. Dire quali trame vengono ricevute dalla stazione BB.
3. Dire qual è lo stato del forwarding database dello switch S2 al tempo 5.

Svolgimento.

Non si scrive un forwarding database per un hub: AA e BB è come se fossero connessi allo switch S1.

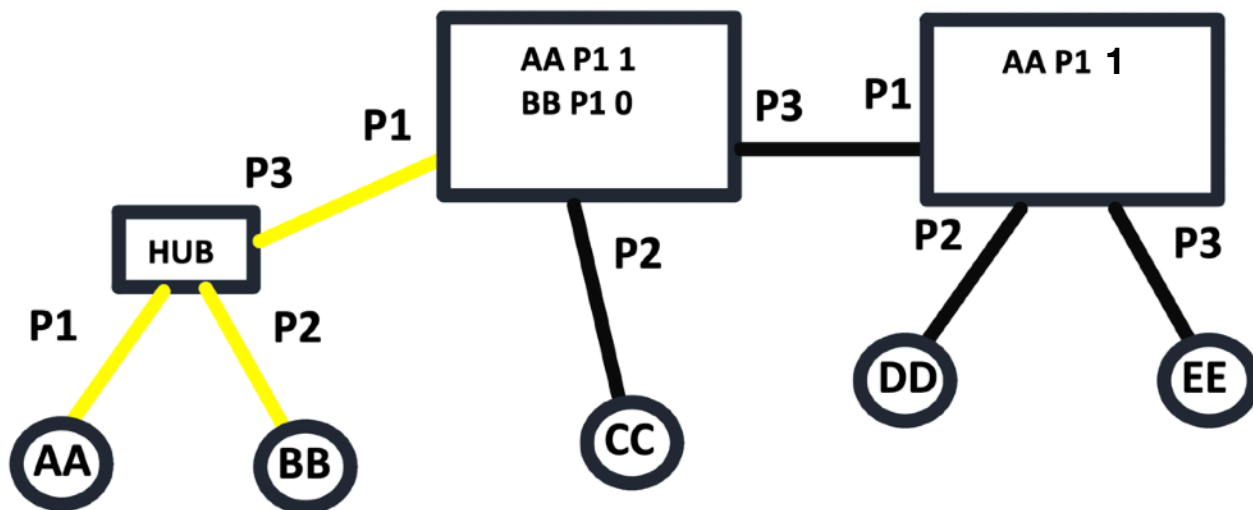
Iniziamo.

$T = 1$, AA $\rightarrow$ CC, l'hub per definizione copia la trama su tutti e due i fronti, lo switch lo copia su tutte le sue porte perchè inizialmente ha un forwarding database vuoto.

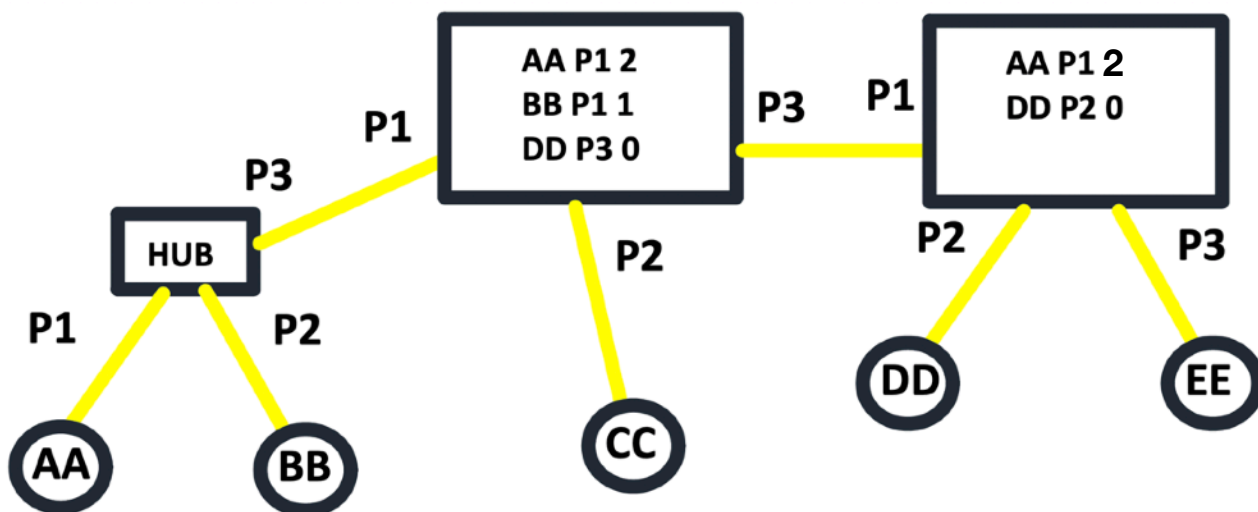


La trama viene ricevuta dalla stazione BB e anche dalla stazione EE.

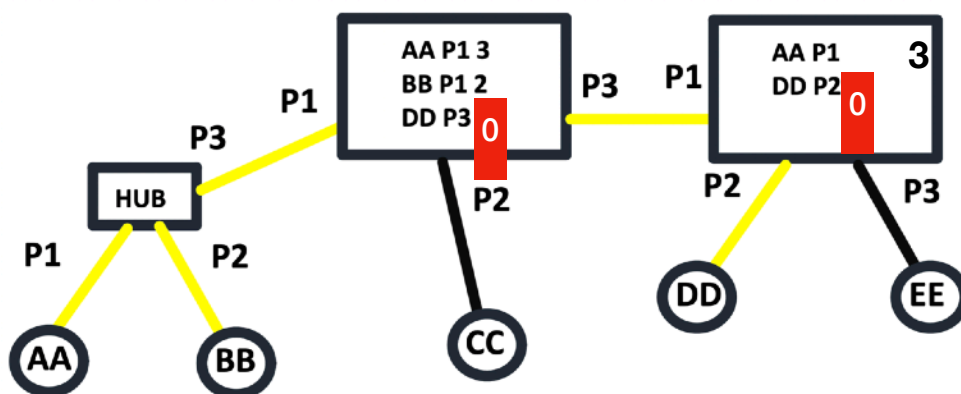
T = 2, BB->AA, lo switch S1 non forwarda perchè sa che AA, che è la destinazione, sta dietro la sua porta P1.



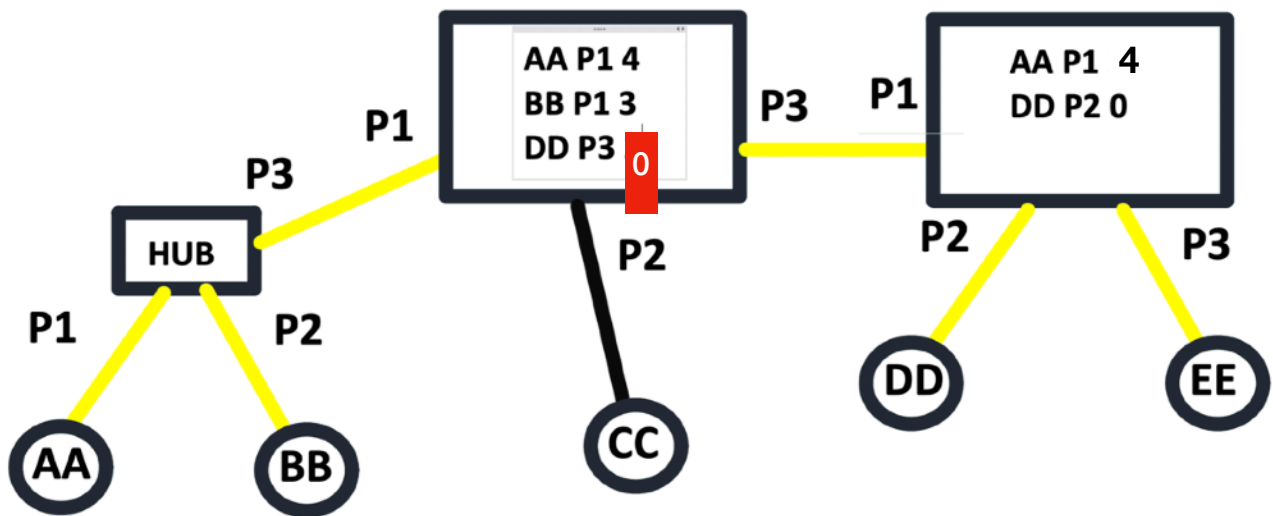
T = 3, DD->CC



T = 4, DD->AA



T = 5, DD->BB.

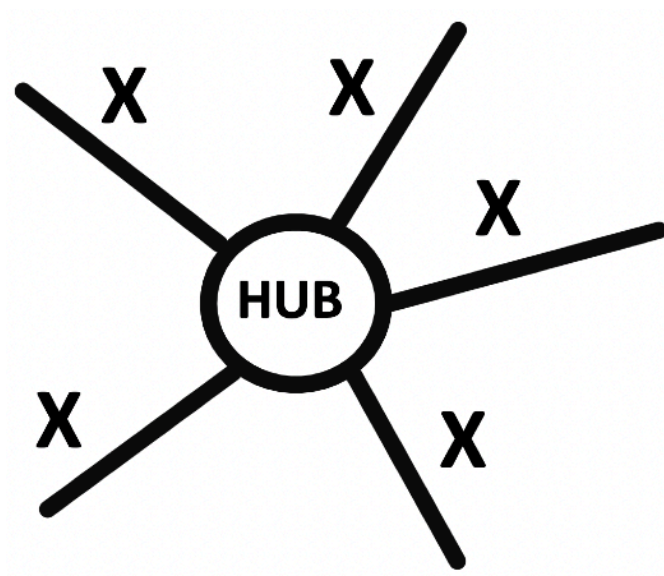


Risposta alla domanda 1: T1, T3, T5.
 Risposta alla domanda 2: T1, T3, T4, T5.
 Risposta alla domanda 3:

AA P1 4
 DD P2 0

Prova d'esame del 10 ottobre 2018

In una rete ethernet **100 Mbps a stella** avente un **Hub al centro**, quale risulta essere la lunghezza massima di ogni singolo collegamento nell'assunzione che il ritardo introdotto dall'hub sia pari a 0.9 microsecondi?



Svolgimento.

Chiamiamo X la lunghezza del collegamento.
Il diametro $d = 2X$.

$$5,12 \mu s \geq 2 \times \left(\frac{2x}{200 \text{ m}/\mu s} + 0,9 \mu s \right)$$

$$x \leq 166$$

Risposta: 166m.

Prova d'esame del 28 maggio 2012 - Vero o Falso

Un dominio di collisione è sempre maggiore o uguale ad un dominio di broadcast: **falso**.

Un router IP con interfaccia ethernet termina un dominio di collisione: **vero**.

Un router IP con interfaccia ethernet termina un dominio di broadcast: **vero**.

Su un medesimo dominio di collisione è possibile avere terminali con link rate differente: **falso**.

Su un medesimo dominio di broadcast è possibile avere terminali con link rate differente: **vero**.

Ricorda che:

Un dominio di collisione è l'insieme delle stazioni che condividono lo stesso mezzo di trasmissione e possono generare collisioni.

Un dominio di broadcast è l'insieme delle stazioni che ricevono i frame di broadcast inviati su una rete.

Prova d'esame del 10 marzo 2021

Si spieghi il motivo per cui, in un sistema ethernet, si utilizzano dei valori di backoff discreti e multipli di un determinato tempo di slot, e si determini il valore di tale slot nel caso fittizio di un'eventuale tecnologia ethernet a 400 Mbps.

Svolgimento.

Prima parte:

Dopo una collisione le stazioni non ritentano subito a trasmettere, ma attendono un tempo di attesa casuale, che **aumenta esponenzialmente** se le collisioni continuano.

Il termine esponenziale deriva dal concetto che dopo la **prima collisione** l'attesa è breve, se le collisioni si ripetono, l'intervallo possibile **raddoppia**, riducendo drasticamente la probabilità di nuove collisioni.

Seconda parte:

$$\frac{512 \text{ bit}}{400 \times 10^6 \text{ bit/s}} = 1,28 \times 10^{-6} \text{ s} = 1,28 \mu s$$

Nel caso di una tecnologia Ethernet a 400 Mbps, lo slot time risulta pari a $512/400 \cdot 10^6 = 1,28 \mu s$.

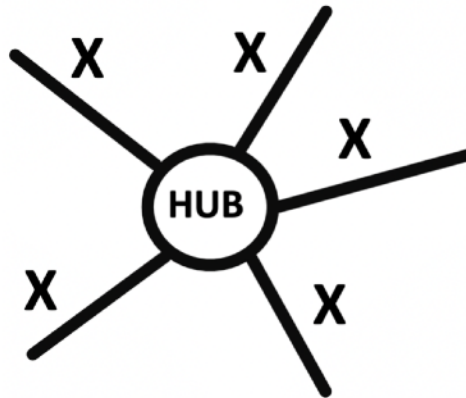
Prova d'esame del 22 maggio 2019

Una rete a stella ha al centro un hub che opera con un ritardo $T \mu s$.

I terminali attestati all'hub sono tutti equidistanti, e posti alla distanza massima d .

Il rate della rete è di 100 Mbps e la velocità del segnale nel mezzo è di 200 km/ms.

Se si acquista un nuovo hub in grado di ridurre il ritardo di una quantità $\Delta T = 0,36 \mu s$, di quanto è possibile allungare ogni singolo collegamento dal terminale al centro stella?



Svolgimento.

Intanto ormai sappiamo come si fanno questi esercizi.

Ricaviamoci subito il tempo di trasmissione di un pacchetto.

$$\frac{512 \text{ bit}}{100 \times 10^6 \text{ bit/s}} = 5,12 \times 10^{-6} \text{ s} = 5,12 \mu s$$

$$5,12 \mu s = 2 \times \left(\frac{2d}{200 \text{ m}/\mu s} + T \right) + 2 \times \left(\frac{2d'}{200 \text{ m}/\mu s} + T' \right)$$

$$5,12 \mu s = \frac{4d}{200 \text{ m}/\mu s} + 2T = \frac{4d'}{200 \text{ m}/\mu s} + 2T'$$

$$\frac{4d - 4d'}{200 \text{ m}/\mu s} + 2T - 2T' = 0$$

$$\frac{4(d - d')}{200} + 2(T - T') = 0$$

$$\frac{d - d'}{50} + 2(T - T') = 0$$

$$\frac{d - d'}{100} + (T - T') = 0$$

$$\frac{\Delta d}{100} + \Delta t = 0$$

$$\frac{\Delta d}{100} = -\Delta t$$

$$\frac{\Delta d}{100} = -0,36$$

La distanza è di 36 metri.

Wi-Fi

Con l'introduzione della trasmissione radio il contesto cambia significativamente.

In una rete Ethernet cablata, se un pacchetto viene trasmesso da un nodo e ricevuto da un altro, la probabilità che esso contenga errori è estremamente bassa, grazie alla natura controllata del mezzo fisico.

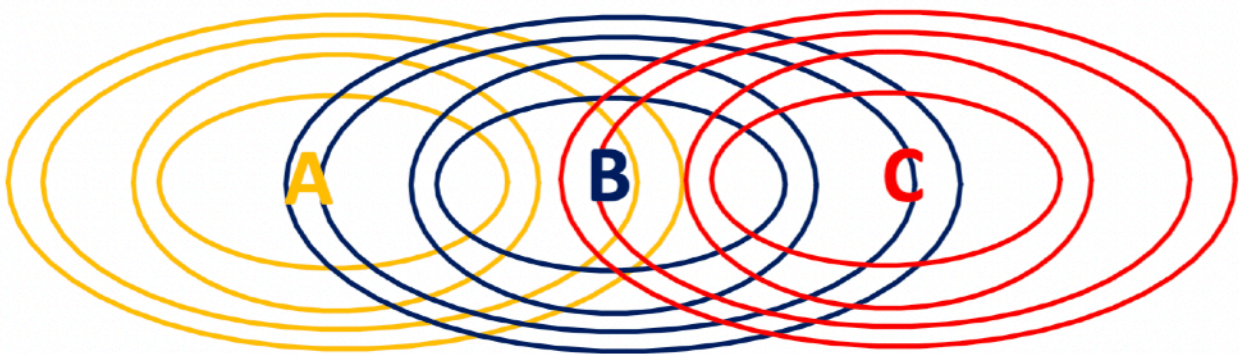
In una rete wireless, invece, la probabilità che un pacchetto non venga ricevuto correttamente è sensibilmente più elevata, a causa di interferenze, attenuazioni e rumore del canale radio.

Per questo motivo, nel Wi-Fi non è possibile fare a meno di un meccanismo di riscontro a livello di collegamento radio: è necessario introdurre un **acknowledgment (ACK) esplicito per ogni pacchetto trasmesso**, così da confermare l'avvenuta ricezione corretta.

Tra due trame si verifica una **collisione** quando i segnali trasmessi da due stazioni si sovrappongono nello stesso intervallo di tempo sul mezzo radio. Se una stazione sta ricevendo e i segnali provenienti da due trasmettitori arrivano con potenza comparabile, il ricevitore non riesce a distinguere correttamente nessuno dei due: il risultato è un segnale disturbato, percepito come rumore.

Si consideri, ad esempio, uno scenario in cui la stazione A trasmette, la stazione C trasmette contemporaneamente e la stazione B si trova tra le due. In presenza di interferenze, B può ricevere una sovrapposizione dei segnali di A e C, rendendo impossibile la decodifica corretta della trama.

Per questo motivo, nelle reti wireless l'obiettivo non è tanto rilevare la collisione dopo che è avvenuta, quanto prevenirla o ridurne significativamente la probabilità. Il protocollo adottato nel Wi-Fi prende infatti il nome di **CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)**, ossia accesso multiplo con ascolto del canale e prevenzione delle collisioni.



In Ethernet, quando il mezzo diventa libero, la stazione attende un intervallo minimo chiamato **IFS (Inter Frame Spacing)** prima di poter trasmettere.

Nel Wi-Fi esiste un intervallo analogo, denominato **DIFS (Distributed Inter Frame Space)**. Una stazione che desidera trasmettere deve prima verificare che il canale sia inattivo e mantenerlo libero per un intervallo pari al DIFS; solo al termine di questo periodo può iniziare la trasmissione del proprio dato.

A differenza di Ethernet cablata, nel Wi-Fi ogni pacchetto deve essere **esplicitamente riscontrato**. Ciò significa che, se una stazione riceve correttamente una trama, deve inviare un **acknowledgment (ACK)** per confermarne la ricezione.

Nel momento in cui invia l'ACK, il ricevitore assume temporaneamente il ruolo di trasmettitore e deve quindi, a sua volta, accedere correttamente al canale secondo le regole previste dal protocollo.

Ora tutto assume coerenza.

Il ricevitore deve inviare un **riscontro (ACK)** dopo aver ricevuto correttamente una trama. Se il ricevitore, prima di trasmettere l'ACK, attendesse un intervallo **DIFS**, si creerebbe un problema: una stazione che sta aspettando di trasmettere percepirebbe il canale occupato durante la trasmissione della trama, poi lo vedrebbe libero e, dopo il proprio DIFS, potrebbe iniziare a trasmettere proprio nello stesso momento in cui il ricevitore sta inviando l'ACK.

Si verificherebbe quindi una collisione tra il nuovo pacchetto e l'ACK.

Una collisione che coinvolge un ACK è particolarmente critica, perché l'ACK serve a confermare la corretta ricezione della trama precedente.

L'idea è la seguente: dopo la trasmissione di una trama dati, il ricevitore non attende un DIFS, ma un intervallo più breve chiamato **SIFS (Short Inter Frame Space)**.

Il **SIFS** è il tempo di attesa più corto previsto dal protocollo Wi-Fi e viene riservato alle risposte immediate, come gli ACK o i frame di controllo CTS.

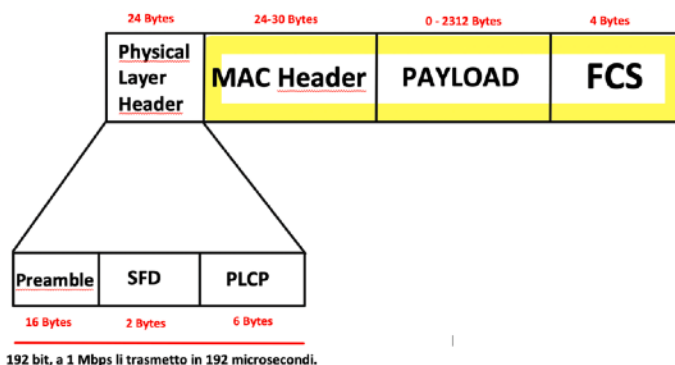
In questo modo, il ricevitore ottiene una priorità implicita nell'accesso al mezzo: **poiché il SIFS è più breve del DIFS, nessun'altra stazione in attesa può iniziare a trasmettere prima che l'ACK venga inviato**.

Questo meccanismo evita collisioni con l'ACK e garantisce che il riscontro venga trasmesso tempestivamente, aumentando l'affidabilità della comunicazione wireless.



Al momento non abbiamo evitato le collisioni fra pacchetti diversi, però ho protetto l'ack dalla collisione con altre stazioni.

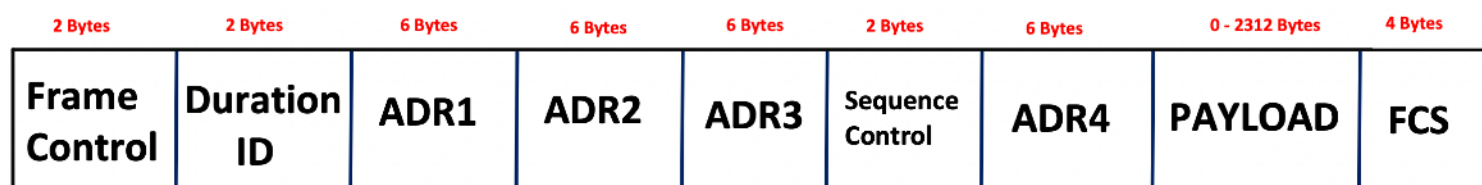
In **802.11 (Wi-Fi)** la struttura è leggermente più complessa rispetto a Ethernet, perché il mezzo è radio e servono più informazioni di controllo.



Il **preambolo** nel Wi-Fi ha lo stesso scopo generale di Ethernet: sincronizzare il ricevitore prima dell'inizio del frame vero e proprio.

All'interno di **PLCP** ci sarà scritta la tipologia di modulazione adottata per lo specifico pacchetto.

Ora andiamo ad analizzare la struttura della trama.



Possono esserci fino a **4 indirizzi MAC**, perché nel Wi-Fi esistono: stazione, access point, destinatario finale e mittente originale.

Il Payload è il dato vero e proprio.

Come in Ethernet, **FCS** contiene un CRC e serve a verificare l'integrità.

Il campo **Duration** (2 byte) nel frame 802.11 indica **per quanto tempo il canale radio sarà occupato** dalla trasmissione in corso e dalle eventuali risposte (ACK, CTS, ecc.).

Non indica la lunghezza del frame in byte. Indica un tempo espresso in microsecondi.

Nel Wi-Fi non possiamo permetterci collisioni frequenti.

Il campo Duration serve a permettere alle altre stazioni di sapere che il canale sarà occupato.

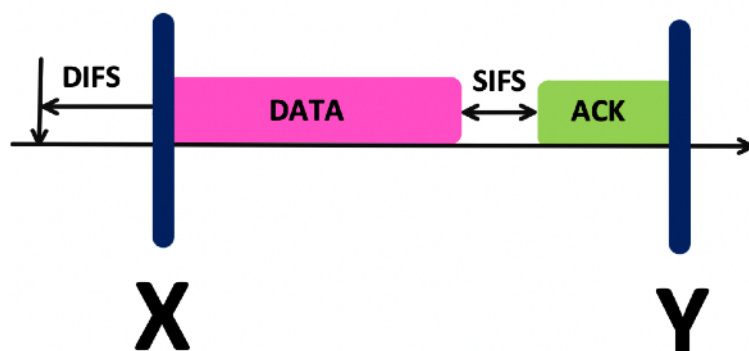
Sequence Control serve a numerare i frame, gestire ritrasmissioni e ricomporre eventuali frammentazioni.

Il **Frame Control** è un campo fondamentale che indica il tipo di frame (Data, Control, Management), il sottotipo (ACK, RTS, CTS, Beacon...) e un flag (retry, power management, ecc.).

Dice "che tipo di messaggio è".

IL CAMPO DURATION [ANALISI IN DETTAGLIO]

Il campo Duration riveste un significato particolarmente rilevante, poiché non indica la durata della singola trama, bensì dell'intero procedimento di handshake. Esso specifica fino a quale istante il canale rimarrà occupato nell'ambito della sequenza di trasmissioni e riscontri prevista dal protocollo.



Si può affermare che l'handshake si estende da un determinato istante iniziale (X) fino a un istante finale (Y); l'intervallo temporale compreso tra questi due istanti costituisce l'intera sequenza di scambio prevista dal protocollo.

Quindi, le stazioni possono effettuare quel procedimento che prende il nome di:

Carrier Sense Virtuale.

Non solo ascolto il canale, ma leggo anche *la prima parte della trama di un pacchetto* e mi leggo quanto durerà lo scambio.

Quindi abbiamo risolto un problema immenso.

Si consideri un esempio con tre stazioni: una stazione centrale trasmette un pacchetto che viene ricevuto dalla stazione posta alla sua destra. Quest'ultima, dopo aver ricevuto correttamente la trama, invia un **ACK**. Tuttavia, la stazione situata alla sinistra del trasmettitore potrebbe non percepire l'ACK di ritorno, a causa della diversa posizione o delle condizioni del canale radio. Se tale stazione tentasse di trasmettere in quel momento, si verificherebbe una collisione con l'ACK. Per evitare questa situazione, la stazione che non riesce a sentire l'ACK può fare affidamento sul **campo Duration** contenuto nella trama iniziale. Leggendo tale campo, essa è in grado di determinare per quanto tempo l'intero handshake manterrà occupato il canale.



Ma la fregatura è dietro l'angolo. Se cambio l'ordine e il ricevitore lo metto al centro e il sender a destra, il sender manderà pacchetti al ricevitore e il ricevitore manderà l'ack, e fin qui tutto ok. Ma la stazione STA questa volta non sente la trasmissione, e la soluzione del field duration non funziona.

Siamo in un caso di "**Hidden Station**".

Quindi STA trasmette, sente che il canale è libero (quindi il carrier sense non mi blocca) e trasmette, e sul receiver arriva la collisione.

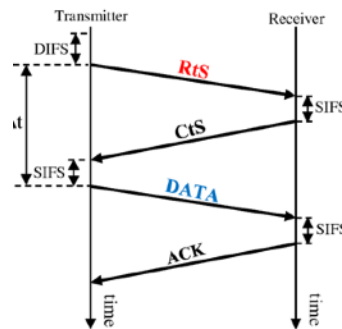
Modalità RTS/CTS in IEEE 802.11

La modalità **RTS/CTS (Request To Send / Clear To Send)** è un meccanismo opzionale del protocollo Wi-Fi (802.11) progettato per **ridurre le collisioni**, in particolare nel caso dei **nodi nascosti**.

Nel Wi-Fi può accadere che due stazioni non si sentano tra loro, ma entrambe riescano a raggiungere lo stesso destinatario. Questo è il classico problema dei nodi nascosti.

Quindi succede che la stazione che vuole trasmettere invia un piccolo frame di controllo **RTS**.

Contiene il campo **Duration**. Il destinatario risponde con un **CTS**. Anche questo contiene un campo **Duration** aggiornato. Il mittente trasmette il frame dati. Il destinatario invia il **riscontro finale**.



Se una stazione è **nascosta** rispetto al mittente potrebbe non sentire l'**RTS**, ma sentirà il **CTS** del destinatario, e quindi saprà di non dover trasmettere.

RTS/CTS funziona perché anche se non senti il mittente, sentirai quasi sempre il destinatario.

Ed è il **CTS** che protegge la trasmissione.

La figura seguente è il riassunto di quanto ci siamo detti:



Ovviamente questa soluzione peggiora le prestazioni perché impiega più tempo per mandare la stessa quantità d'informazione,

Quando parte il backoff?

Nel Wi-Fi non abbiamo collision detection (non possiamo ascoltare mentre trasmettiamo).

Quindi si usa **collision avoidance**.

Il backoff serve a ridurre la probabilità che due stazioni trasmettano insieme, distribuire nel tempo i tentativi di accesso.

Una stazione vuole trasmettere: fa **carrier sensing**, se il canale è libero per un tempo **DIFS** NON trasmette subito, ma entra in **backoff casuale**.

In ethernet, quando c'è la collisione, il backoff scatta dopo. La prima trasmissione è sempre diretta. In WiFi è diverso: nella maggior parte dei casi scatta un backoff prima della trasmissione.

Come funziona il backoff

La stazione sceglie un numero casuale:

$$k \in [0, CW]$$

dove CW = Contention Window è una finestra di valori interi.

Il tempo di attesa è:

$$\text{Backoff} = k \times \text{SlotTime}$$

Se il canale resta libero → il contatore **decrementa**.

Se il canale diventa occupato → il contatore **si congela**.

Quando il canale torna libero (e passa un DIFS) → il conteggio **riprende**.

Esempio.

Immagina 3 stazioni Wi-Fi: A, B e C. Tutte vogliono parlare. Il canale è libero. Tutte e tre vogliono trasmettere.

Tutte: ascoltano, vedono il canale libero, aspettano il **DIFS**. Finito il DIFS, nessuna trasmette subito.

Supponiamo che la CW permetta numeri tra 0 e 7.

- A sceglie **5**
- B sceglie **2**
- C sceglie **6**

Questo numero è il **numero di slot da aspettare**.

Questi numeri NON sono secondi. Sono **numero di slot**.

Uno slot è un piccolo intervallo fisso di tempo.

Immagina il tempo a blocchetti:

| slot1 | slot2 | slot3 | slot4 | slot5 | slot6 |

Slot 1

- A passa da **5** → **4**
- B passa da **2** → **1**

- C passa da 6 $\rightarrow$ 5

Nessuno ha ancora 0.

Slot 2

- A passa da 4 $\rightarrow$ 3
- B passa da 1 $\rightarrow$ 0
- C passa da 5 $\rightarrow$ 4

B arriva a 0.

Cosa significa arrivare a 0?

Significa:

“È il mio turno. Trasmetto adesso.”

B inizia a trasmettere.

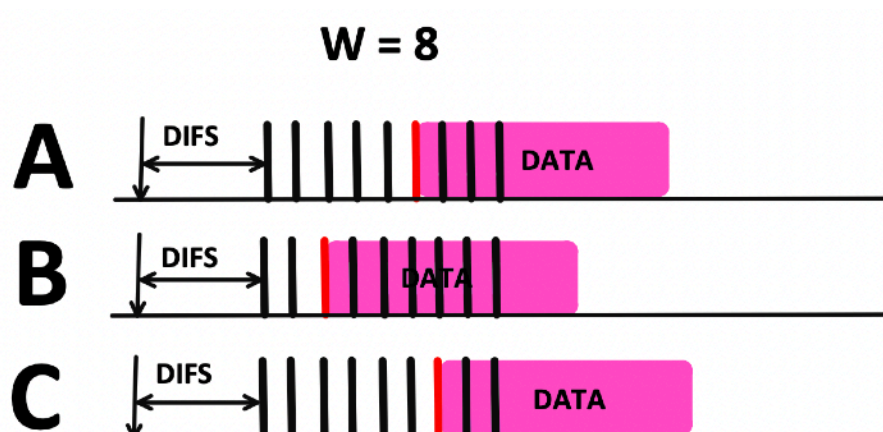
Appena B inizia a trasmettere: il canale diventa occupato, A e C smettono di contare, NON azzerano, NON ripartono da capo, si **congelano** a:

- A = 3
- C = 4

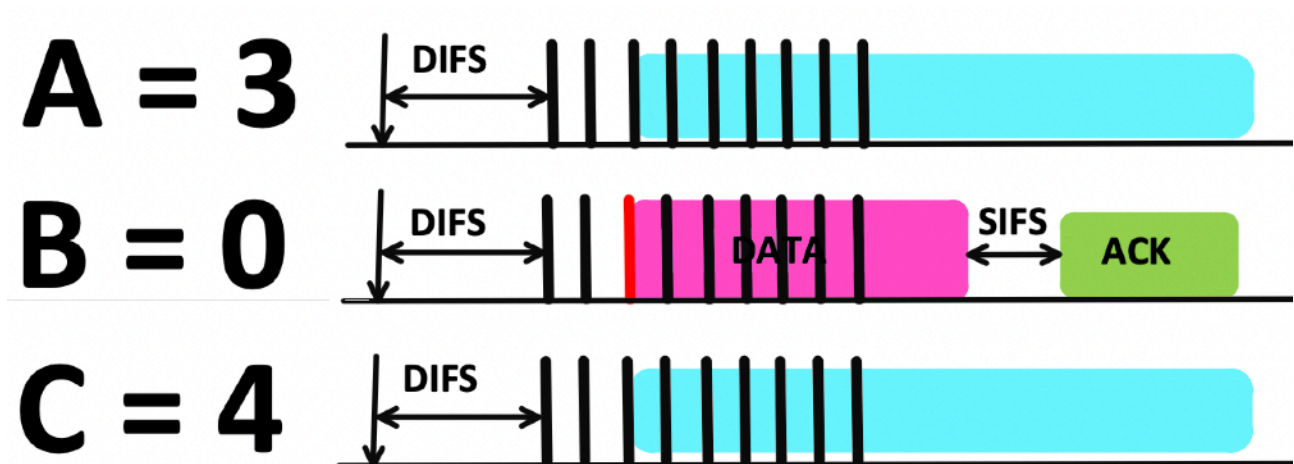
Appena qualcuno parla $\rightarrow$ stop conteggio.

Quando B finisce:

1. arriva l'ACK
2. passa un DIFS
3. A e C riprendono a contare da dove erano rimasti

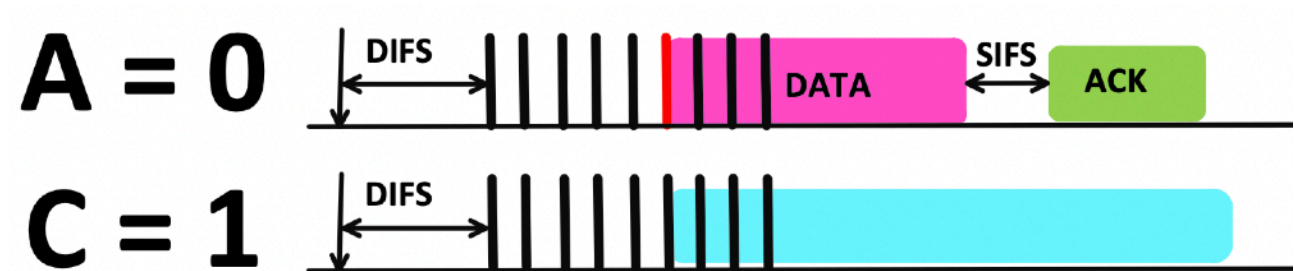


B = 0, sono passati due slot.



A SCENDE A 3, C SCENDE A 4.

Finita la trasmissione di B, A e C riprendono a contare da dove erano rimasti.



A trasmette.

Poi trasmette C.



Abbiamo evitato la collisione.

Dopo il freezing aspetto sempre un DIFS e poi decremento.

Quando esco dal freezing, aspetto DIFS + 1 SLOT come minimo.

Ora, se ho un **pacchetto** successivo da trasmettere genero un post-backoff.

Il **post-backoff** è un intervallo di backoff che una stazione esegue **dopo aver completato con successo una trasmissione**, anche se non ha immediatamente un nuovo pacchetto pronto.

Immagina questa situazione: Una stazione A ha appena trasmesso con successo. Il canale torna libero. A ha subito un altro pacchetto pronto.

Se A potesse trasmettere immediatamente dopo il DIFS, avrebbe un vantaggio enorme rispetto alle altre stazioni. Dominerebbe il canale.

Quindi, completando tutto:

Canale libero
→ DIFS
→ Backoff (se necessario)
→ DATA
→ SIFS
→ ACK
→ DIFS
→ Post-Backoff



Il Post-Backoff non parte immediatamente dopo l'ACK. Prima deve passare un **DIFS**.
Perché l'ACK termina l'handshake. Dopo quello si torna al meccanismo normale di accesso al mezzo.

THROUGHPUT

La quantità effettiva di dati utili trasmessi con successo nell'unità di tempo.

I bit utili sono tutti quelli che si trovano nel payload.

Prestare molta attenzione alla formula sotto.

Il throughput è dato dal valore medio del pacchetto diviso il tempo di ciclo:

$$\text{Throughput} = \frac{E[\text{Payload}]}{E[\text{Cycle-Time}]} \neq E\left[\frac{\text{Payload}}{\text{Cycle-Time}}\right]$$

Ricorda che il rapporto fra le medie non è uguale alla media del rapporto.

Il **tempo di ciclo** è un intervallo temporale che si ripete nel funzionamento del sistema di accesso al mezzo. Tuttavia, esso **non è deterministico**, poiché la sua durata dipende da grandezze aleatorie, in particolare dal **backoff**, che è selezionato casualmente. Per tale motivo si considera il suo valor medio.

ESERCIZIO D'ESAME

La velocità/data rate è 11 Mbps, l'ack viene mandato a 1 Mbps e il payload del pacchetto è di 1500 bytes.

Calcolare il throughput.

Svolgimento.

Ora qui mettiamo insieme tutto ciò che abbiamo visto.

Ack, Rts e Cts vengono solitamente trasmessi a 1 Mbps. **Il preamble (192 bit) non dipende dal bit rate, è fisso, e viene trasmesso a 1 Mbps.** Ricaviamo i tempi in microsecondi.

L'ack sono 14 bytes, perchè è rappresentato da Frame Control, Duration, Address1 e FCS.

Ora troviamo il tempo di trasmissione del dato completo:

$$T_{\text{MPDU}} = 192 \mu s + \frac{(28 + 1500) \cdot 8}{11 \text{ Mbit/s}} = 192 \mu s + \frac{1528 \cdot 8}{11 \cdot 10^6} s = 192 \mu s + 1111.27 \mu s = 1303.27 \mu s \approx 1303 \mu s$$

Ora calcoliamo il tempo di ack.

$$T_{\text{ACK}} = 192 \mu s + \frac{14 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + \frac{112}{10^6} s = 192 \mu s + 112 \mu s = 304 \mu s$$

Ora l'unica cosa variabile è il tempo di backoff.

Poiché la durata di ogni slot è pari a 20 microsecondi e il valore del backoff viene scelto uniformemente nell'intervallo [0,31], si considera il valore medio dell'intervallo, pari a 31/2, e lo si moltiplica per la durata di uno slot per ottenere il tempo medio di backoff.

$$E[\text{Backoff}] = \frac{31}{2} \cdot 20 \mu s = 310 \mu s$$

$$\text{SIFS} = 10 \mu s$$

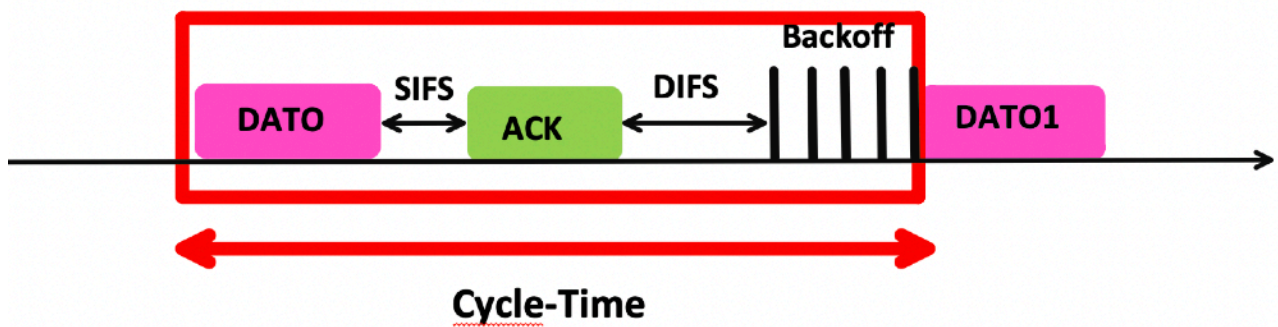
$$\text{SlotTime} = 20 \mu s$$

$$\text{DIFS} = 10 + 2 \cdot 20 = 50 \mu s$$

Questi sono i valori tipici dei tempi di SIFS e DIFS.

A questo punto abbiamo il throughput:

$$\text{Throughput} = \frac{1500 \cdot 8 \text{ bit}}{(1303 + 10 + 304 + 50 + 310) \mu s} = \frac{12000 \text{ bit}}{1977 \mu s} = 6.07 \text{ bit}/\mu s = 6.07 \text{ Mbit/s}$$



Quindi, anche se ho acquistato una rete con velocità nominale pari a 11 Mbit/s in realtà non raggiungerò mai tale valore.

In condizioni ideali, e persino in assenza di altre stazioni (quindi trasmettendo da solo), il throughput effettivo risulta pari a circa 6,07 Mbit/s, a causa degli overhead introdotti dal preambolo, dagli header, dagli intervalli interframe (SIFS, DIFS), dall'ACK e dal tempo medio di backoff.

Se aumento il rate a 54 Mbps ho un throughput di circa 10 Mbps.

Capite il problema? Io pago 54 Mbps per andare a 10 Mbps.

Se aumento il rate a 300 Mbps ho un throughput di circa 13 Mbps.

Se oggi uso RTS/CTS è un problema, perchè devo contare anche i preamboli delle trame RTS e CTS. Lo posso usare solo se ho trasmissioni nascoste.

ESERCIZIO D'ESAME DEL 22 MAGGIO 2019

Un terminale Wifi usa il protocollo 802.11b e trasmette al **rate di 11 Mbps** usando accesso **basic** per pacchetti di payload fino a 999 bytes;

Usa la modalità **RTS/CTS** per i pacchetti di dimensione maggiore;

I pacchetti trasmessi hanno dimensione multipla di 100 bytes (ovvero 100,200,300, etc...) e sono distribuiti uniformemente tra 100 e 1500 bytes.

Si calcoli il throughput della stazione, assumendo che le trame di controllo siano trasmesse al basic rate di 1 Mbps ed i rimanenti parametri sono quelli standard di 802.11b:

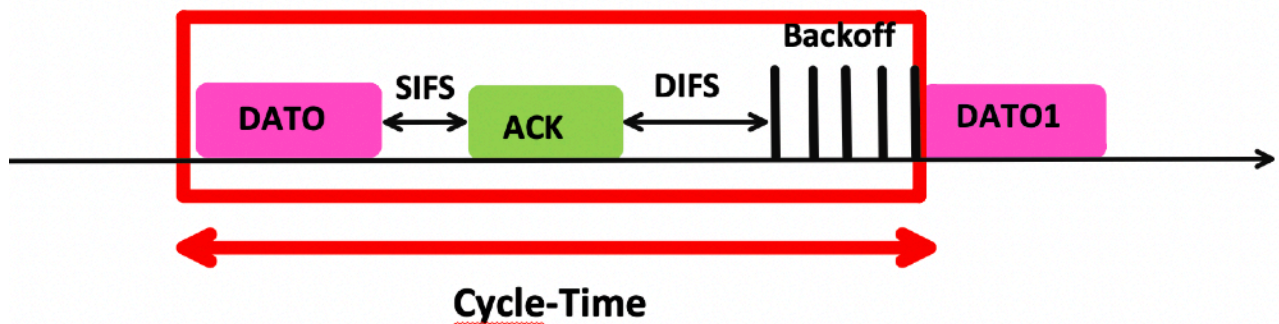
(Preambolo = 192 microsecondi, Difs = 50 microsecondi, Sifs = 10 microsecondi, Ack = Cts = 14 bytes, Rts = 20 bytes, Cwmin = 31, slot-time = 20 microsecondi).

Svolgimento.

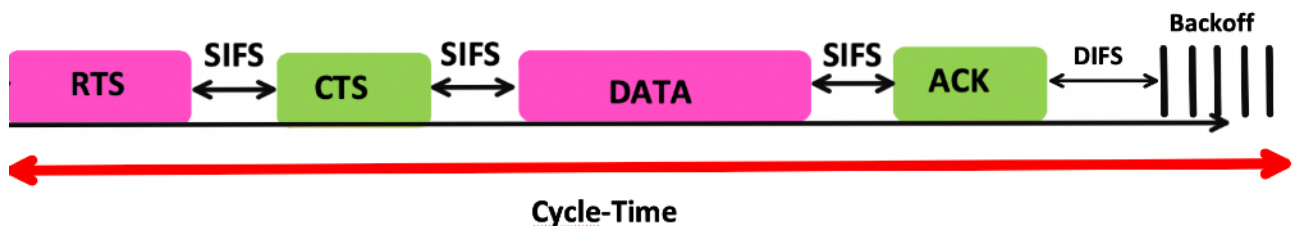
Come prima operazione è necessario determinare il **tempo di ciclo** relativo sia alla modalità di trasmissione **Basic** sia alla modalità **RTS/CTS**.

In entrambi i casi occorre calcolare la durata complessiva di un ciclo di trasmissione, includendo tutti gli intervalli temporali coinvolti (frame dati, eventuali frame di controllo, SIFS, DIFS, ACK e tempo medio di backoff).

Basic:



Rts/Cts:



L'esercizio si basa sulla **dimensione dei pacchetti**, in particolare sulla dimensione del loro **payload**.

Per payload pari a **100, 200, 300, 400, ..., 900 byte** si utilizza la modalità di trasmissione **Basic**.
Per payload pari a **1000, 1100, 1200, 1300, 1400, 1500 byte** si utilizza invece la modalità **RTS/CTS**.

Innanzitutto la dimensione del payload deve essere stimata tramite il suo **valor medio**.

Se sono in Rts/Cts il valor medio è:

$$\frac{1000 + 1500}{2} = 1250$$

Se sono in basic il valor medio è:

$$\frac{100 + 900}{2} = 500$$

Di seguito il tempo di ciclo per ogni modalità di trasmissione.

$$T_{\text{cycle, RTS}} = T_{\text{RTS}} + T_{\text{SIFS}} + T_{\text{CTS}} + T_{\text{SIFS}} + T_{\text{trasmissione Data}} + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + T_{\text{backoff}}$$

$$T_{\text{Cycle, Basic}} = T_{\text{trasmissione Data}} + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + T_{\text{backoff}}$$

Quindi ho che:

$$T_{\text{RTS}} = 192 \mu s + \frac{20 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + \frac{160}{10^6} s = 192 \mu s + 160 \mu s = 352 \mu s$$

E quindi:

$$T_{\text{CTS}} = 192 \mu s + \frac{14 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + \frac{112}{10^6} s = 192 \mu s + 112 \mu s = 304 \mu s$$

Analogamente:

$$T_{\text{ACK}} = 192 \mu s + \frac{14 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + 112 \mu s = 304 \mu s$$

È altrettanto vero che:

$$T_{\text{TrasmissioneData, Basic}} = 192 \mu s + \frac{(28 + 500) \cdot 8}{11 \text{ Mbit/s}} = 192 \mu s + \frac{528 \cdot 8}{11 \cdot 10^6} s = 192 \mu s + 384 \mu s = 576 \mu s$$

$$T_{\text{TrasmissioneData, RTS/CTS}} = 192 \mu s + \frac{(28 + 1250) \cdot 8}{11 \text{ Mbit/s}} = 192 \mu s + \frac{1278 \cdot 8}{11 \cdot 10^6} s = 192 \mu s + 929.45 \mu s = 1121.45 \mu s \approx 1121 \mu s$$

Ora ricordiamoci la formula del throughput:

$$\text{Throughput} = \frac{E[\text{Payload}]}{E[\text{Cycle-Time}]} \neq E\left[\frac{\text{Payload}}{\text{Cycle-Time}}\right]$$

Al numeratore bisogna inserire la dimensione media del pacchetto. In modo particolare il payload medio.

$$E[\text{Payload}] = \frac{100 + 1500}{2} \text{ byte} = 800 \text{ byte} = 800 \cdot 8 \text{ bit} = 6400 \text{ bit}$$

Ora troviamo i rispettivi tempi di ciclo sostituendo i valori calcolati.

$$T_{\text{cycle, RTS}} = 352 + 10 + 304 + 10 + 1121 + 10 + 304 + 50 + 310$$

$$T_{\text{cycle, RTS}} = 2471 \mu s$$

Va da sé:

$$T_{\text{Cycle, Basic}} = 576 + 10 + 304 + 50 + 310$$

$$T_{\text{Cycle, Basic}} = 1250 \mu s$$

Ora, per effettuare il calcolo del throughput serve il valor medio dei tempi di ciclo.

Ricordiamo un po' di statistica.

Il basic lo utilizzo per la trasmissione di 9 pacchetti.

Rts/Cts lo utilizzo per la trasmissione di 6 pacchetti.

$9 + 6 = 15$ casi totali.

$$P(\text{Basic}) = \frac{9}{15} = 0.6$$

$$P(\text{RTS/CTS}) = 1 - P(\text{Basic}) = 1 - 0.6 = 0.4 = \frac{6}{15}$$

Quindi:

$$E[T_{\text{cycle}}] = T_{\text{cycle, Basic}} \cdot \frac{9}{15} + T_{\text{cycle, RTS/CTS}} \cdot \frac{6}{15}$$

$$E[T_{\text{cycle}}] = 1250 \mu s \cdot 0.6 + 2471 \mu s \cdot 0.4$$

$$E[T_{\text{cycle}}] = 750 \mu s + 988.4 \mu s = 1738.4 \mu s$$

Quindi il throughput finale sarà:

$$\text{Throughput} = \frac{6400 \text{ bit}}{1738.4 \mu s} = 3.68 \text{ bit}/\mu s$$

Velocissimi con le unità di misura, che corrisponde a:

$$1 \text{ bit}/\mu s = 1 \text{ Mbit/s}$$

E allora il throughput finale è:

$$\text{Throughput} \approx 3.68 \text{ Mbit/s}$$

Fine.

ESERCIZIO D'ESAME DEL 10 SETTEMBRE 2018

Una stazione 802.11b usa il seguente meccanismo di **rate adaption**.

Ogni pacchetto è inizialmente trasmesso a **11 Mbps**.

Se la trasmissione fallisce, la stazione ritrasmette il pacchetto usando il rate a **5.5 Mbps**.

Se anche questa fallisce, la stazione trasmette a **2 Mbps**.

Se anche quest'ultima trasmissione fallisce, la stazione passa al pacchetto successivo e ricomincia.

La stazione trasmette pacchetti di dimensione **750 bytes**, ed usa le solite regole del protocollo, con parametri;

(Preambolo = 192 microsecondi, Difs = 50 microsecondi, Sifs = 10 microsecondi, Ack = 14 bytes, Cwmin = 31, slot-time = 20 microsecondi e trasmissione ack a basic rate di 1 Mbps).

Si calcoli il throughput della stazione, assumendo che la trasmissione a 11 Mbps fallisca nel **70%** dei casi, la trasmissione a 5.5 Mbps fallisca nel **50%** dei casi, e la trasmissione a 2 Mbps fallisca nel **20%** dei casi.

Svolgimento.

Questo è l'esercizio corretto per stabilire se il backoff è stato pienamente compreso. Nel protocollo, se la trasmissione **fallisce** (per esempio per collisione o mancata ricezione dell'ACK), il **contention window (CW)** viene *raddoppiato*.

$CW_{min} = 31 \rightarrow \text{backoff} \in [0, 31]$.

Dopo 1 collisione $\rightarrow CW = 63 \rightarrow \text{backoff} \in [0, 63]$.

Dopo 2 collisioni $\rightarrow CW = 127$.

Etc...

Ricordiamo la formula del valor medio del backoff:

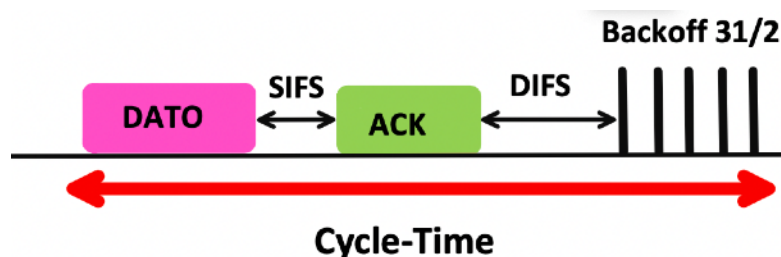
$$E[\text{Backoff}] = \frac{CW}{2} \cdot \text{SlotTime}$$

Se la trasmissione va a buon fine, si riparte dal valore minimo:

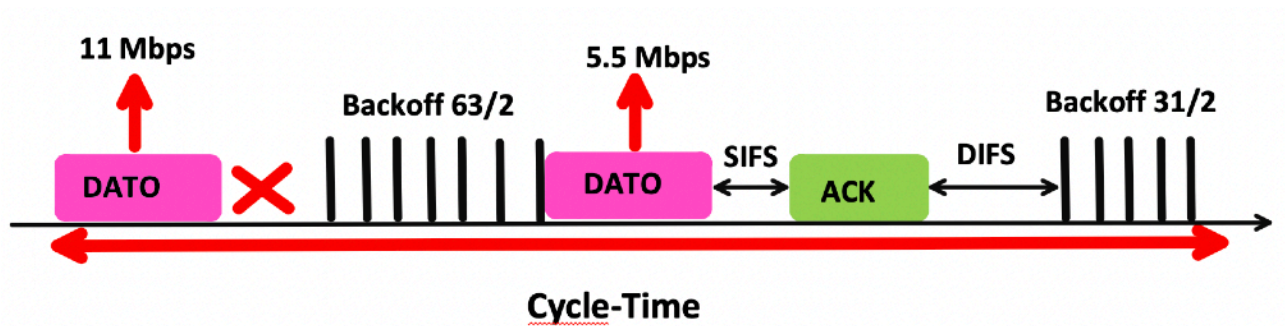
$$CW \rightarrow CW_{min}$$

Adesso posso considerare 4 casi:

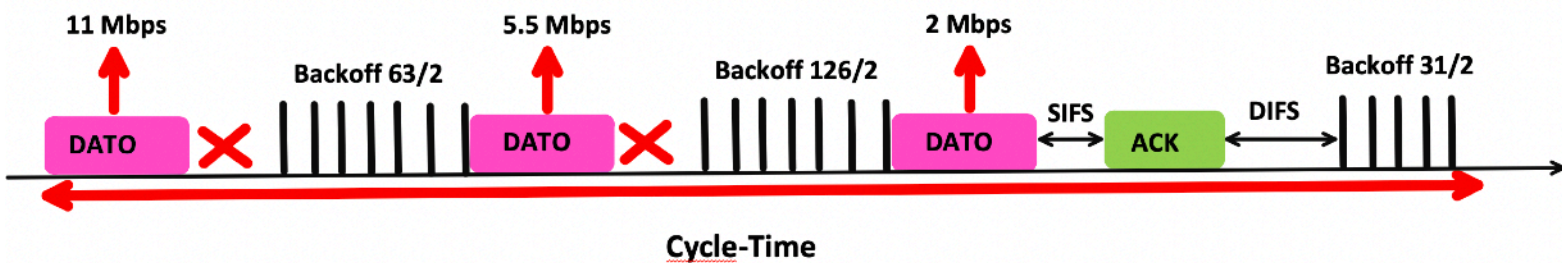
1. Mando la prima trama a 11 Mbps e non sbaglio. Quindi continuo a 11 Mbps.



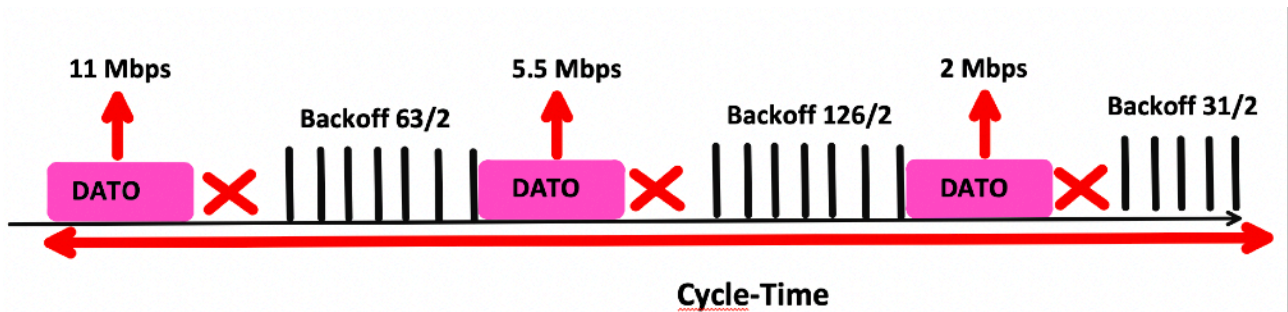
2. Trasmetto a 11 Mbps e sbaglio: estraggo un backoff di 63/2 e trasmetto a 5.5 Mbps.



3. Sbaglio a 11, sbaglio a 5.5, trasmetto perfettamente a 2 Mbps.



4. Va tutto male. In questo caso il pacchetto viene scartato.



E ricomincio con tempo di backoff pari a 31/2.

Analizziamo le probabilità.

Il primo caso può fallire nel 70% dei casi, quindi la probabilità che possa accadere è $1 - 0.7 = 0.3$.

Il secondo caso rappresenta la probabilità che mi va male al primo colpo moltiplicato per la probabilità che vada bene al secondo colpo: $0.7 \times 0.5 = 0.35$.

Il terzo caso corrisponde ad una probabilità di $0.7 \times 0.5 \times 0.8 = 0.280$.

Il quarto caso ha una probabilità di accadere di $0.7 \times 0.5 \times 0.2 = 0.07$.

Casistica	Probabilità che si verifichi
Primo Caso	30%
Secondo Caso	35%
Terzo Caso	28%
Quarto Caso	7%

Totale = 100%.

Adesso l'esercizio è completamente uguale al precedente.

Per ognuna delle casistiche calcoliamo il tempo di ciclo per il primo caso, il secondo, il terzo e il quarto.

Dopodiché faremo la media.

$$T_{\text{cycle, primo caso}} = T_{\text{dato}} + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, secondo caso}} = T_{\text{dato}}(11 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, terzo caso}} = T_{\text{dato}}(11 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{126}{2}} + T_{\text{dato}}(2 \text{ Mbit/s}) + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, quarto caso}} = T_{\text{dato}}(11 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{126}{2}} + T_{\text{dato}}(2 \text{ Mbit/s}) + t_{\text{backoff}}^{\frac{31}{2}}$$

Attenzione:

Tempo di ACK → durata effettiva di trasmissione del frame ACK (preambolo + header), ad esempio 304 μs.

ACK timeout → intervallo massimo che il mittente attende, dopo aver inviato il frame dati, prima di dichiarare fallita la trasmissione.

In questi tipi di esercizi vige una regola fondamentale:

$$T_{\text{ACK timeout}} \approx T_{\text{ACK}}$$

Quindi, sulla base di questa enorme semplificazione adottata dal Prof. Bianchi, se consideriamo esplicitamente l'**ACK timeout**, allora nei casi in cui la trasmissione fallisce **non compaiono SIFS e ACK**, ma compare il tempo di attesa dell'ACK timeout.

$$T_{\text{cycle, primo caso}} = T_{\text{dato}} + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, secondo}} = T_{\text{dato}}(11 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, terzo}} = T_{\text{dato}}(11 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{126}{2}} + T_{\text{dato}}(2 \text{ Mbit/s}) + T_{\text{SIFS}} + T_{\text{ACK}} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle, quarto}} = T_{\text{dato}}(11 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{63}{2}} + T_{\text{dato}}(5.5 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{126}{2}} + T_{\text{dato}}(2 \text{ Mbit/s}) + T_{\text{ACK timeout}} + t_{\text{backoff}}^{\frac{31}{2}}$$

Okay, abbiamo tutto, ora ricaviamo i tempi.

$$T_{\text{dato}}(11 \text{ Mbit/s}) = 192 \mu s + \frac{(28 + 750) \cdot 8}{11 \text{ Mbit/s}} = 192 \mu s + \frac{778 \cdot 8}{11 \cdot 10^6} s = 192 \mu s + 565.82 \mu s = 757.82 \mu s \approx 758 \mu s$$

$$T_{\text{dato}}(5.5 \text{ Mbit/s}) = 192 \mu s + \frac{6224}{5.5 \cdot 10^6} s = 192 \mu s + 1131.64 \mu s = 1323.64 \mu s \approx 1324 \mu s$$

$$T_{\text{dato}}(2 \text{ Mbit/s}) = 192 \mu s + \frac{6224}{2 \cdot 10^6} s = 192 \mu s + 3112 \mu s = 3304 \mu s$$

$$T_{\text{ACK}} = 192 \mu s + \frac{14 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + 112 \mu s = 304 \mu s$$

$$T_{\text{ACK}} = T_{\text{ACK timeout}}$$

$$E[\text{Backoff}] = \frac{31}{2} \cdot 20 \mu s = 310 \mu s$$

$$E[\text{Backoff}] = \frac{63}{2} \cdot 20 \mu s = 630 \mu s$$

$$E[\text{Backoff}] = \frac{126}{2} \cdot 20 \mu s = 1260 \mu s$$

$$T_{\text{cycle},1} = 758 + 304 + 50 + 310$$

$$T_{\text{cycle},1} = 1422 \mu s$$

$$T_{\text{cycle},2} = 758 + 304 + 630 + 1324 + 304 + 50 + 310$$

$$T_{\text{cycle},2} = 3680 \mu s$$

$$T_{\text{cycle},3} = 758 + 304 + 630 + 1324 + 304 + 1260 + 3304 + 304 + 50 + 310$$

$$T_{\text{cycle},3} = 8548 \mu s$$

$$T_{\text{cycle},4} = 758 + 304 + 630 + 1324 + 304 + 1260 + 3304 + 304 + 310$$

$$T_{\text{cycle},4} = 8498 \mu s$$

Ora

$$\text{Throughput} = \frac{E[\text{Payload}]}{E[\text{Cycle-Time}]} \neq E\left[\frac{\text{Payload}}{\text{Cycle-Time}}\right]$$

Quindi:

$$E[\text{Payload}] = 750 \cdot 0.3 + 750 \cdot 0.35 + 750 \cdot 0.28 + 0 \cdot 0.07$$

$$E[\text{Payload}] = 225 + 262.5 + 210 + 0$$

$$E[\text{Payload}] = 697.5 \text{ byte}$$

In bit:

$$E[\text{Payload}] = 697.5 \text{ byte} = 5580 \text{ bit}$$

$$E[T_{\text{cycle}}] = T_{\text{cycle},1} \cdot 0.3 + T_{\text{cycle},2} \cdot 0.35 + T_{\text{cycle},3} \cdot 0.28 + T_{\text{cycle},4} \cdot 0.07$$

$$E[T_{\text{cycle}}] = 1422 \cdot 0.3 + 3680 \cdot 0.35 + 8548 \cdot 0.28 + 8498 \cdot 0.07$$

$$E[T_{\text{cycle}}] = 426.6 + 1288 + 2393.44 + 594.86$$

$$E[T_{\text{cycle}}] = 4702.9 \mu s$$

Quindi il throughput:

$$\text{Throughput} = \frac{E[\text{Payload}]}{E[T_{\text{cycle}}]} = \frac{5580 \text{ bit}}{4702.9 \mu s}$$

$$\text{Throughput} = 1.186 \text{ bit}/\mu s$$

$$\text{Throughput finale} \approx 1.19 \text{ Mbit/s}$$

Fine.

ESERCIZIO D'ESAME DEL 15 MARZO 2022

Una stazione 802.11b cerca di usare più efficientemente il canale di trasmissione trasmettendo, ove possibile, **due trame consecutive**.

Nello specifico, **se la trasmissione della prima trama ha successo** (ossia l'ack viene ricevuto correttamente), **la stazione trasmette la seconda trama dopo un SIFS**.

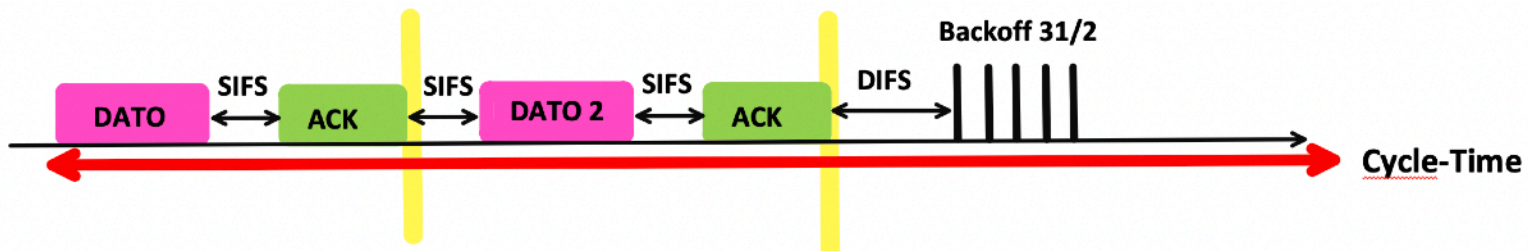
Assumendo che ogni singola trasmissione abbia successo con probabilità **75%**, si calcoli il **throughput** della stazione:

Payload = 1500 bytes,
Rate = 11 Mbps,
Preambolo = 192 microsecondi,
Difs = 50 microsecondi,
Sifs = 10 microsecondi,
Slot-Time = 20 microsecondi,
Ack = 14 bytes,
Ack Timeout = Tempo di Ack,
Cwmin = 31,
Trame di controllo trasmesse a basic rate 1 Mbps.

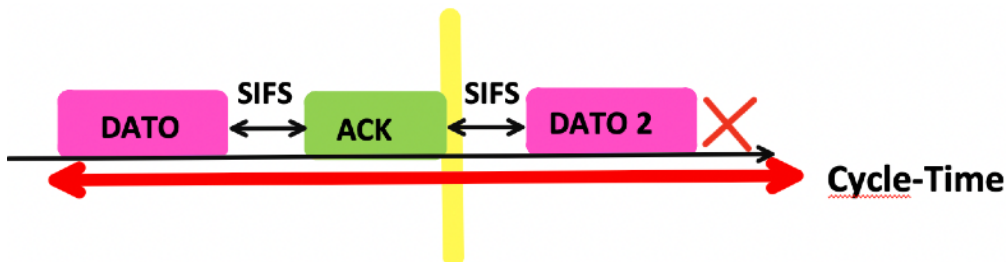
Svolgimento.

Al solito, distinguiamo le casistiche.

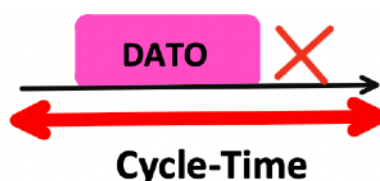
1. Trasmetto bene entrambe le trame.



2. Sbaglio la seconda trasmissione dopo la prima corretta.



3. Sbaglio la prima e non avrò la seconda.



Ogni singola trasmissione ha successo con il 75% di probabilità.

Quindi, il primo tempo di ciclo può avere successo con una probabilità di $0.75 \times 0.75 = 0.5625$.

Il secondo tempo di ciclo può avere successo con una probabilità di $0.75 \times 0.25 = 0.1875$.

Il terzo ed ultimo tempo di ciclo può avere successo con una probabilità di 0.25.

Trasmissione	Probabilità che accade
1	56%
2	19%
3	25%

Totale = 100%.

$$T_{\text{cycle},1} = T_{\text{dato}_1} + T_{\text{SIFS}} + T_{\text{ACK}_1} + T_{\text{SIFS}} + T_{\text{dato}_2} + T_{\text{SIFS}} + T_{\text{ACK}_2} + T_{\text{DIFS}} + t_{\text{backoff}}^{\frac{31}{2}}$$

$$T_{\text{cycle},2} = T_{\text{dato}_1} + T_{\text{SIFS}} + T_{\text{ACK}_1} + T_{\text{SIFS}} + T_{\text{dato}_2} + T_{\text{ACK timeout}}$$

$$T_{\text{cycle},3} = T_{\text{dato}} + T_{\text{ACK timeout}}$$

$$T_{\text{dato}} = 192 \mu s + \frac{(28 + 1500) \cdot 8}{11 \text{ Mbit/s}} = 192 \mu s + 1111.27 \mu s = 1303.27 \mu s \approx 1303 \mu s$$

$$T_{\text{ACK}} = T_{\text{ACK timeout}} = 192 \mu s + \frac{14 \cdot 8}{1 \text{ Mbit/s}} = 192 \mu s + 112 \mu s = 304 \mu s$$

$$E[\text{Backoff}] = \frac{31}{2} \cdot 20 \mu s = 310 \mu s$$

$$T_{\text{cycle},1} = 1303 + 10 + 304 + 10 + 1303 + 10 + 304 + 50 + 310 = 3604 \mu s$$

$$T_{\text{cycle},2} = 1303 + 10 + 304 + 10 + 1303 + 304 = 3234 \mu s$$

$$T_{\text{cycle},3} = 1303 + 304 = 1607 \mu s$$

$$E[\text{Payload}] = (2 \cdot 1500) \cdot 0.56 + 1500 \cdot 0.19 + 0 \cdot 0.25$$

$$E[\text{Payload}] = 3000 \cdot 0.56 + 1500 \cdot 0.19 + 0$$

$$E[\text{Payload}] = 1680 + 285 + 0 = 1965 \text{ byte}$$

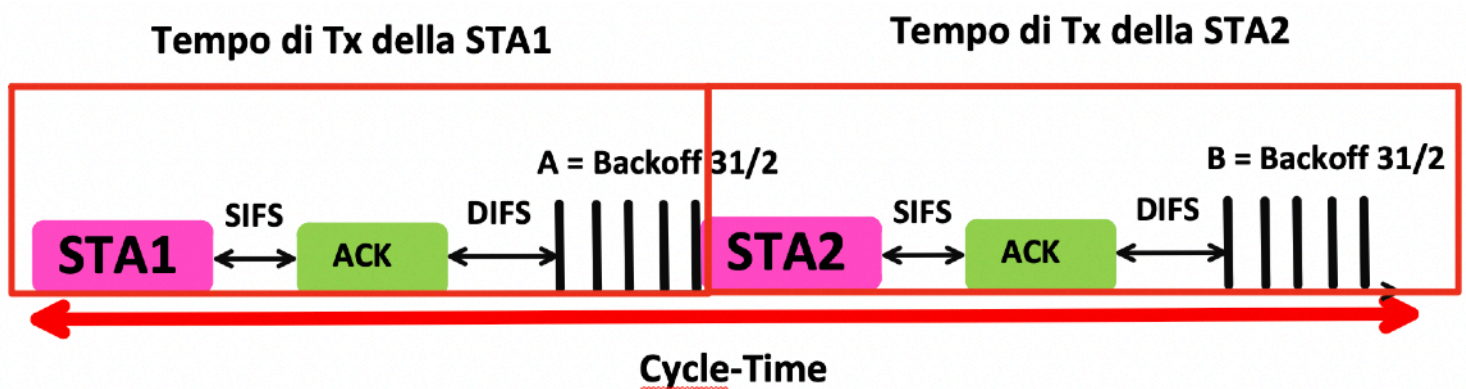
$$E[T_{\text{cycle}}] = 3604 \cdot 0.56 + 3234 \cdot 0.19 + 1607 \cdot 0.25 = 3034.45 \mu s$$

$$\text{Throughput} \approx 5.18 \text{ Mbit/s}$$

Se una stazione 1 trasmette a **2 Mbps** e l'altra stazione 2 a **11 Mbps**, e entrambe inviano pacchetti con payload di **1500 byte**, accade che la stazione a 2 Mbps occupa il mezzo trasmissivo per un tempo significativamente maggiore rispetto a quella a 11 Mbps.

Poiché il canale radio è condiviso e l'accesso avviene tramite meccanismo di contesa, la stazione più lenta consuma una quantità di tempo di trasmissione (airtime) molto superiore per ciascun frame. Di conseguenza, riduce il tempo disponibile anche per la stazione più veloce.

Pertanto, il throughput complessivo della rete non si avvicina a 11 Mbps, ma risulta fortemente condizionato dalla presenza della stazione a 2 Mbps, che determina una significativa riduzione dell'efficienza globale del sistema.



$$Tx_{sta1} = 192 + 6112 = 6304 \mu s$$

$$Tx_{sta2} = 192 + 1111 = 1303 \mu s$$

Si vede chiaramente che la stazione lenta occupa il canale per circa **5 volte più tempo** rispetto alla stazione veloce.

$$\text{Throughput} = \frac{1500 \cdot 8}{1303 \mu s + 310 \mu s + 2 (10 \mu s + 304 \mu s + 50 \mu s) + 6304 \mu s}$$

$$\text{Throughput} \approx 1,39 \text{ Mbps}$$

Nel modello medio si dimostra che, per due stazioni saturate e senza collisioni, la somma dei due backoff medi equivale a **un solo backoff medio**. Statisticamente il tempo medio di contesa per due stazioni è pari a una sola finestra media.

CRC - Cyclic Redundancy Check

Il **CRC** (Cyclic Redundancy Check, in italiano *controllo di ridondanza ciclica*) è un **codice di rilevazione degli errori** usato per verificare se i dati trasmessi sono stati alterati durante il trasferimento.

Nel frame MAC di 802.11 il CRC prende il nome di **FCS (Frame Check Sequence)** ed è lungo **32 bit**.

In pratica: Il trasmettitore calcola un valore numerico (il CRC) a partire dai bit del messaggio. Questo valore viene aggiunto alla fine del frame. Il ricevitore ricalcola il CRC sui dati ricevuti. Se il risultato coincide, il frame è considerato **integro**. Se non coincide, il frame viene **scartato**.

Perché si chiama “ciclico”?

Perché il calcolo si basa su una **divisione polinomiale binaria**.

I bit del messaggio vengono trattati come coefficienti di un polinomio e divisi per un polinomio generatore prefissato. Il resto della divisione è proprio il CRC.

ANALISI IN DETTAGLIO - FUNZIONAMENTO CRC

Si prende il dato da trasmettere, viene allungato di un certo numero di bit (8 zeri nel caso di CRC8) e tutto viene diviso per il **polinomio generatore** del CRC in modo tale da ottenere un risultato della divisione con resto.

Dato da trasmettere: 1001100101.

Supponiamo di utilizzare CR8 da 8 bit.

Scriviamo il dato come polinomio:

$$X^9 + X^6 + X^5 + X^2 + 1$$

Polinomio generatore più comune per CRC-8:

$$G(x) = x^8 + x^2 + x + 1$$

In forma binaria diventa: 100000111.

Ora il dato da trasmettere deve essere esteso: Lo estendi aggiungendo **r zeri in coda**, dove r è il grado del polinomio generatore. Se usi **CRC-8**, allora r = 8.
Quindi aggiungi **8 zeri**.

100110010100000000

Con CRC-8 aggiungiamo **8 zeri**, cioè moltiplichiamo il polinomio per x elevato alla 8.

$$(x^9 + x^6 + x^5 + x^2 + 1) \cdot x^8$$

Quindi ho:

$$x^{17} + x^{14} + x^{13} + x^{10} + x^8$$

A questo punto bisogna effettuare la divisione polinomiale.

La formula generale del CRC si scrive così:

$$\frac{\text{Dato} \cdot x^r}{G(x)} = \text{quoziante} + \frac{\text{resto}}{G(x)}$$

Quindi, sostituendo:

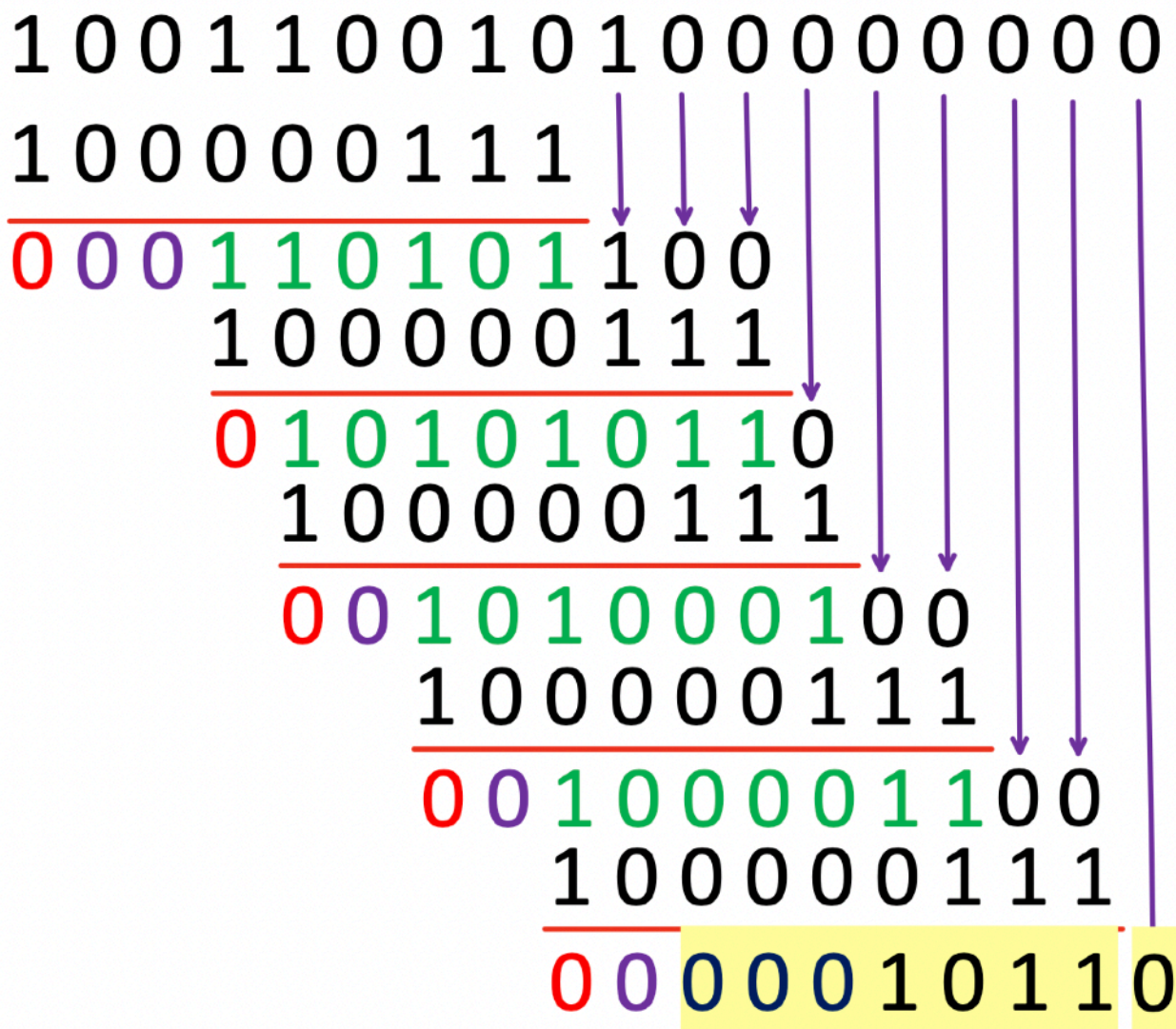
$$\frac{x^{17} + x^{14} + x^{13} + x^{10} + x^8}{x^8 + x^2 + x + 1}$$

Come si fa la divisione in binario:

La “divisione in binario” del CRC si fa come una **divisione polinomiale**, ma usando **XOR** al posto della sottrazione.



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0



Ogni volta che trovi un 1 “utile” (il primo 1 a sinistra del blocco), **allinei** il polinomio generatore e fai XOR poi **abbassi** (porti giù) i bit successivi del messaggio.

Il resto che ho ottenuto è: 00010110.

Ora lo scriviamo in forma polinomiale:

$$x^4 + x^2 + x$$

Ora il quoziente? il quoziente non è scritto esplicitamente, **ma si può ricostruire** guardando dove ho fatto gli XOR.

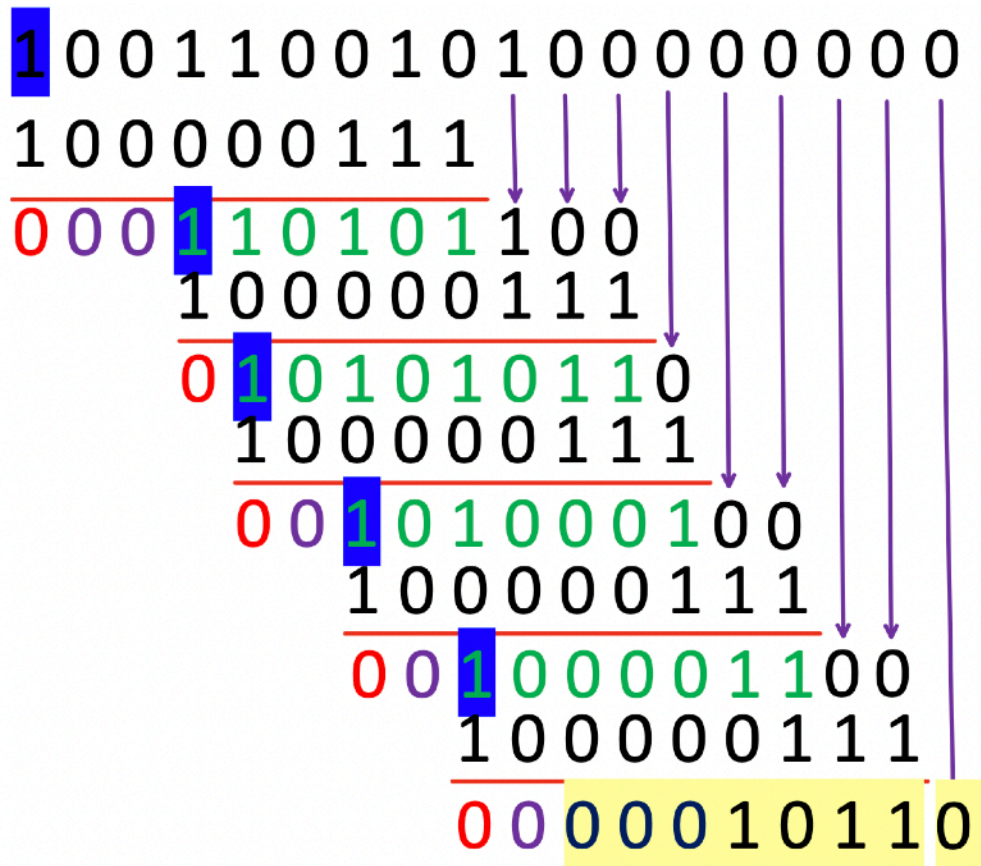
Ma ricorda: nel CRC il quoziente **non viene mai usato**. Serve solo il resto.

Se il dividendo ha N bit, il divisore ha M bit, allora il **quoziente** ha:

$$\text{bit quoziente} = \text{bit dividendo} - \text{bit divisore} + 1$$

divisore è lungo 9 bit → quindi il quoziente avrà 10 bit (perché $18 - 9 + 1 = 10$), da 0 a 9. II

Partiamo da 9 fino a scendere a 0.



9, 6, 5, 3, 1.

$$q_i = \begin{cases} 1 & \text{se il bit più significativo del blocco corrente è 1} \\ 0 & \text{se è 0} \end{cases}$$

Quindi il quoziente è:

1001101010

Che in forma polinomiale diventa:

$$x^9 + x^6 + x^5 + x^3 + x$$

Quindi, il risultato di tutto è:

$$\frac{x^{17} + x^{14} + x^{13} + x^{10} + x^8}{x^8 + x^2 + x + 1} = x^9 + x^6 + x^5 + x^3 + x^2 + \frac{x^4 + x^2 + x}{x^8 + x^2 + x + 1}$$

Costruzione del codice CRC

Il messaggio originale era: 1001100101.

Il resto della divisione è: 00010110. (Deve essere da 8 Bit, perchè CRC8).

Quindi deve essere lungo **8 bit** perché il polinomio generatore del **CRC-8** ha **grado 8**.

A questo punto trasmetto il messaggio originale + resto.

Tx [100110010100010110].

CRC calcolato. Fine.

Il ricevitore ora che cosa farà?

Il ricevitore prende questo dato trasmesso Tx e lo *divide* per il polinomio generatore.

Se ora dividi questo numero completo per il generatore:

$$\frac{T(x)}{G(x)}$$

il resto deve essere 0.

Questo è il controllo di correttezza.

Se il resto è diverso da zero vuol dire che c'è stato un errore.

CRC corretto / Trasmissione valida.

PATTERN D'ERRORE

Quando si verifica un errore sul canale, alcuni bit del frame trasmesso possono essere alterati. Supponiamo che durante la trasmissione vengano invertiti (flip) tre bit. In questo caso si introduce un errore che può essere rappresentato formalmente mediante una **maschera di errore**, cioè un **pattern di errore**.

La **maschera di errore** è un **polinomio** i cui coefficienti valgono 1 nelle posizioni in cui il bit è stato invertito e 0 nelle posizioni corrette.

Sequenza trasmessa T: 1 0 0 0 0 1 1 0 1 0.

Sequenza ricevuta R: 1 **1** 0 0 **1** 0 1 0 1 0.

Facciamo lo XOR tra le due sequenze e troviamo il pattern di errore:

Posizione	(T)	(R)	(Pattern)
1	1	1	0
2	0	1	1
3	0	0	0
4	0	0	0
5	0	1	1
6	1	0	1
7	1	1	0
8	0	0	0
9	1	1	0
10	0	0	0



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

Quindi la maschera di errore E è: **0 1 0 0 1 1 0 0 0 0**.

I bit flippati sono esattamente **3** (posizioni **2, 5, 6**), perché lì E vale 1.

Se prendo il dato T, e lo metto in XOR con il pattern d'errore E, ottengo R, ossia il risultato finale errato.

$$T \oplus E = R$$

```
T: 1 0 0 0 0 1 1 0 1 0
E: 0 1 0 0 1 1 0 0 0 0
-----
R: 1 1 0 0 1 0 1 0 1 0
```

Abbiamo imparato come funziona il rilevamento dell'errore, non la ritrasmissione. Ora bisogna gestire il processo di ritrasmissione.

Nel caso di **Ethernet**, se una trama risulta errata (ossia il controllo CRC produce un resto diverso da zero), essa viene semplicemente scartata dal ricevitore. Il protocollo non prevede alcun meccanismo di ritrasmissione a livello MAC: la gestione dell'eventuale recupero dell'informazione è demandata ai livelli superiori, ad esempio al protocollo di trasporto (come TCP), se previsto.

Nel caso del **Wi-Fi (IEEE 802.11)**, invece, la situazione è differente. Poiché il mezzo radio è più soggetto a interferenze e perdite, il protocollo MAC integra un meccanismo di affidabilità. Se una trama non viene ricevuta correttamente, il destinatario non invia l'ACK (Acknowledgment). L'assenza dell'ACK viene interpretata dal trasmettitore come un fallimento della trasmissione, che comporta la ritrasmissione della trama secondo le regole previste dal meccanismo di backoff e controllo della contesa.

Protocolli ARQ (Automatic Repeat reQuest)

I **protocolli ARQ** sono meccanismi di controllo dell'errore che garantiscono la consegna affidabile dei dati.

Primo Protocollo: Stop-and-Wait ARQ

Il protocollo **Stop-and-Wait** prevede la trasmissione di un pacchetto e la trasmissione del successivo avviene sempre a fronte dell'ack ricevuto precedentemente. Se trasmettiamo un pacchetto alla volta noi trasmetteremo un pacchetto successivo solo a “completa ricezione” del pacchetto precedente.

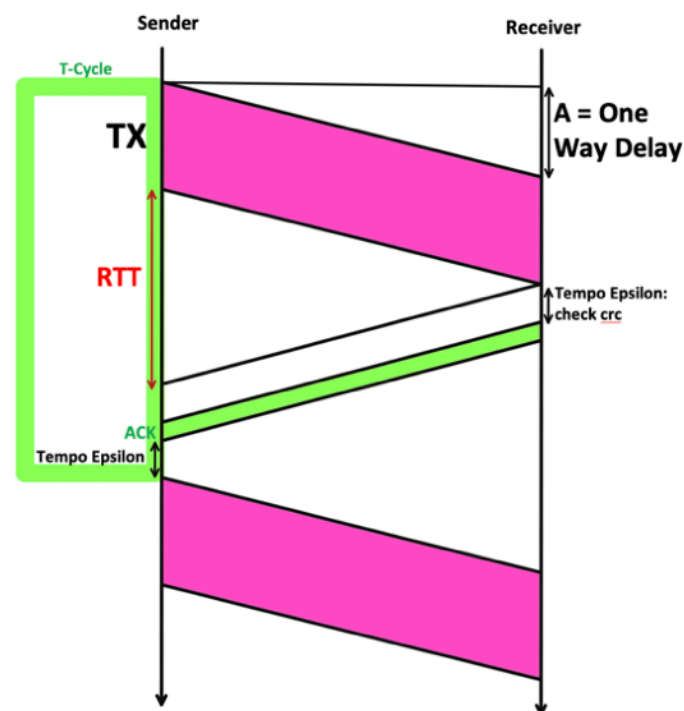
Wi-Fi (802.11) implementa un meccanismo ARQ di tipo Stop-and-Wait.

Prima di addentrarci fornisco delle definizioni preliminari da conoscere:

RTO: L'**RTO (Retransmission Timeout)** è il tempo massimo che il mittente attende prima di ritrasmettere un pacchetto se non riceve l'ACK.

Sequence Number: Il **sequence number** è un numero associato a ogni segmento o trama trasmessa.

One Way Delay: Il **One Way Delay** è il tempo che un pacchetto impiega per andare **dal mittente al destinatario**, in una sola direzione. Non dipende dalla velocità di trasmissione! Che io vada a 1 Mbps, 1 Gbps o 1 bit/s la propagazione è legata alla velocità della luce, quindi dipende dalla lunghezza del collegamento.



Quello che dipende dalla velocità della linea, per i motivi ovvi discussi precedentemente, è proprio la TX del dato, la linea viola.

Rapidissimi conti, non si accetta ignoranza.

Assumendo che tutti i bit del MSG siano bit utili e la velocità della linea è 100 bps e il payload da 100 bit, impiego 1 secondo per inviarli. Se il payload di 200 bit, 2 secondi, e così via.

Quindi, il messaggio (quello viola) arriva al ricevitore al tempo **A + Prima linea viola**. Poi il ricevitore dovrà **riscontrare** questo pacchetto: questo ACK verrà mandato dopo un tempo epsilon (perchè giustamente deve verificare il CRC se coincide in ricezione).

Attenzione: negli esercizi d'esame, nella maggior parte dei casi, questo tempo epsilon sarà trascurabile.

Se assumiamo che il canale di ritorno vada alla stessa velocità del canale di andata, anche questo tempo sarà pari al rapporto tra la dimensione dell'ack in bit e la velocità della linea.

Una volta ricevuto l'ACK di ritorno, il trasmittente dopo un tempo epsilon rimanderà il prossimo messaggio.

Tatuaggio su pelle: L'intervallo di tempo compreso tra l'istante in cui viene trasmesso il primo messaggio e l'istante in cui viene trasmesso il secondo messaggio costituisce il **tempo di ciclo**.

$$T_{\text{cycle}} = T_x + RTT + \varepsilon + T_{\text{ack}} + \varepsilon$$

Allora possiamo calcolare il throughput.

Ricorda: l'header del MSG non fa parte dei bit utili (payload).

Assumiamo che il MSG sia composto da tutti bit utili e sia una costante, altrimenti dovevamo mettere il valor medio (come abbiamo visto).

$$Thr = \frac{MSG}{T_x + RTT + T_{\text{ack}} + 2\varepsilon}$$

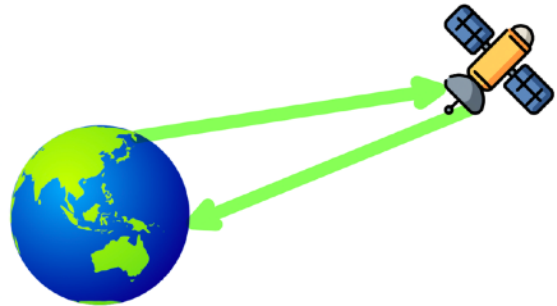
Attenzione: al denominatore, se avessi avuto un header nel MSG, in Tx avrei dovuto considerarlo.

Al numeratore è presente il contenuto del messaggio, al denominatore il tempo di trasmissione del messaggio.

Satellite Geostazionario.

Poiché è necessario comprendere i concetti in modo approfondito e non limitarsi a impararli a memoria, è opportuno analizzare un esempio concreto di comunicazione che avvenga tramite un satellite geostazionario.

Un satellite geostazionario è un satellite che rimane apparentemente immobile nel cielo perché orbita alla stessa velocità di rotazione della Terra, consentendo comunicazioni stabili ma con un ritardo non trascurabile.



Se un messaggio viene trasmesso dall'Università di Roma Tor Vergata, in **via del Politecnico**, verso **via Cambridge** utilizzando un satellite geostazionario, il tempo di propagazione dipende principalmente dalla distanza che il segnale deve percorrere.

Un satellite geostazionario si trova a circa **35.786 km** di altitudine sopra l'equatore.

Pertanto, il segnale deve:

1. percorrere circa **35.786 km** per salire dalla stazione terrestre al satellite;
2. percorrere altri **35.786 km** per scendere dal satellite alla destinazione terrestre.

Il segnale elettromagnetico si propaga a una velocità prossima a quella della luce, pari a circa:

$$3 \times 10^8 \text{ m/s}$$

Il tempo di propagazione risulta quindi:

$$t = \frac{71.572.000}{3 \times 10^8} \approx 0,24 \text{ s}$$

Pertanto, il messaggio impiega circa **240 millisecondi** per andare da via del Politecnico a via Cambridge passando per un satellite geostazionario.

Si tratta esclusivamente del tempo di propagazione; eventuali ritardi di trasmissione, elaborazione o accodamento si sommerebbero a tale valore.

Un segnale elettromagnetico è un'onda costituita da campi elettrici e magnetici oscillanti che trasporta energia e informazione propagandosi nello spazio alla velocità della luce.

Quando invii un messaggio:

1. l'informazione viene convertita in un segnale elettrico;
2. il segnale modula un'onda elettromagnetica;
3. l'onda si propaga nello spazio;
4. il ricevitore la intercetta e la riconverte in informazione.

Se la distanza aumenta, il tempo di propagazione aumenta proporzionalmente

Ora, tornando al nostro discorso c'è un'analogia. Se il tempo di andata e ritorno (RTT) aumenta il throughput diminuisce. Ovviamente se l'RTT diminuisce il throughput aumenta.

CASI GRAVI DA CONSIDERARE

La figura rappresenta una situazione anomala tipica dei protocolli ARQ di tipo Stop-and-Wait, legata alla scadenza del Retransmission Timeout (RTO). Il mittente (Sender) trasmette il messaggio con numero di sequenza. Il messaggio viene correttamente ricevuto dal destinatario (Receiver).

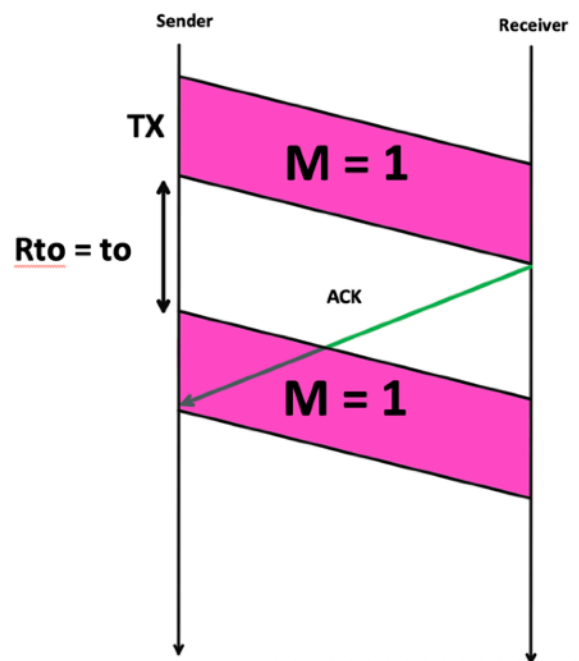
Il destinatario genera un **ACK** e lo invia verso il mittente. Tuttavia, l'ACK impiega un tempo di propagazione significativo.

Nel frattempo, sul lato del mittente è attivo un timer RTO. Se l'ACK non arriva prima della scadenza del timer, il mittente interpreta l'assenza dell'ACK come **perdita del pacchetto**.

Di conseguenza: il timer scade, il mittente ritrasmette il messaggio $M=1$.

In realtà il primo messaggio **non era stato perso**. L'unico problema era un ritardo dell'ACK. Quando l'ACK arriva (linea verde), il mittente ha già avviato la ritrasmissione.

Il ricevitore riceve quindi due copie dello stesso messaggio $M=1$.



La situazione non è un errore del protocollo, ma una **ritrasmissione spuriosa dovuta a RTO troppo breve**.

Soluzione: Per evitare ambiguità dovute a ritrasmissioni spurie, ogni messaggio viene associato a un **numero di sequenza** e ogni **ACK** contiene l'indicazione del numero di sequenza del messaggio correttamente ricevuto.

Quindi, il problema della ritrasmissione ambigua si risolve introducendo numeri di sequenza e ACK numerati, che consentono al protocollo di distinguere correttamente tra messaggi nuovi e duplicati

Se l'ACK = 1 arriva dopo la seconda trasmissione di $M = 1$, il sender lo accetta comunque come conferma valida e procede con il messaggio successivo.

Attenzione, comprendiamo un concetto che è propedeutico per la comprensione dei concetti futuri.

Immagina questo.

Tu (mittente) devi inviare un documento numerato **1**.

Lo mandi.

Il destinatario lo riceve correttamente e ti risponde:
“Ho ricevuto il documento 1”.

Ma la sua risposta (l'ACK) viaggia lentamente.

Tu stai aspettando.
Passa troppo tempo.
Pensi: “Forse non è arrivato”.

Allora lo rimandi: **di nuovo il documento 1**.

Nel frattempo, il destinatario: riceve il primo documento 1, lo registra, ti manda “Ricevuto 1”.

Poi riceve il secondo documento 1.

A quel punto dice:

“Un attimo... questo è lo stesso numero 1 che ho già ricevuto.”

Non lo considera un nuovo documento.
Non lo consegna di nuovo.
Semplicemente lo ignora e, per sicurezza, ti rimanda ancora:

“Ricevuto 1”.

Ora finalmente la conferma arriva a te.

Tu leggi:
“Ricevuto 1”.

E capisci che il documento 1 è stato correttamente ricevuto.

Idealmente il tempo di ritrasmissione T_o viene dettato come nella seguente formula:

$$RTO = T_{RTT} + T_{ack} + 2\varepsilon$$

E questa formula è un grande aiuto e ci permette di collegare dei punti rispetto al problema che stiamo affrontando.

Questa espressione indica che il timeout di ritrasmissione deve essere superiore al tempo necessario affinché il messaggio venga ricevuto e l'ACK torni al mittente, includendo un margine di sicurezza per evitare ritrasmissioni premature.

Se ricevo l'ack mentre sono in RTO faccio partire immediatamente il pacchetto successivo.

Per concludere, l'espressione seguente rappresenta il throughput medio nello Stop-and-Wait in presenza di errori, poiché solo una frazione $(1-P)$ dei messaggi viene consegnata con successo per ogni ciclo di trasmissione.

$$Throughput = \frac{MSG \cdot (1 - P)}{T_{RTT} + T_{msg}}$$

PROTOCOLLI A FINESTRA SCORREVOLE.

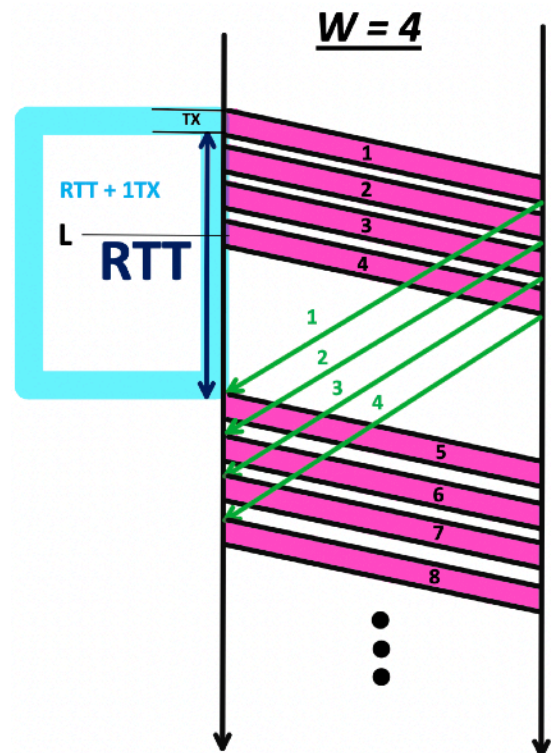
SLIDING WINDOW

Idea di fondo: Inviare un certo numero di pacchetti anche se non ottengo un riscontro (ACK).

Nel protocollo a finestra scorrevole si definisce una **finestra**, indicata con W , che rappresenta il numero massimo di trame che il mittente può trasmettere e mantenere simultaneamente “in volo”, ossia non ancora confermate.

Quando il numero di trame trasmesse ma non ancora riscontrate raggiunge il valore limite W , il mittente deve sospendere temporaneamente l'invio di ulteriori trame. La trasmissione potrà riprendere soltanto dopo la ricezione di almeno una conferma (ACK), che consente lo scorrimento della finestra e libera spazio per l'invio di nuovi dati.

All'istante di tempo L ho 4 pacchetti in volo.
Trasmetto il pacchetto numero 6 solo dopo l'ack del 2.

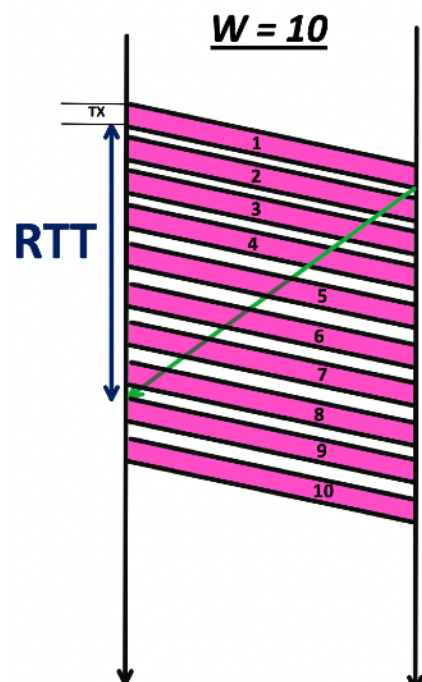


E se $W = 10$?

L'ACK relativo al pacchetto 1 è pervenuto prima che il mittente avesse completato la trasmissione dei dieci pacchetti consentiti dalla dimensione della finestra.

In linea teorica, a partire da quell'istante, il mittente potrebbe già trasmettere l'undicesimo pacchetto, poiché la finestra si è parzialmente liberata. Tuttavia, la trasmissione non avviene immediatamente, in quanto il mittente è ancora impegnato nell'invio del nono pacchetto.

In tale situazione, il canale viene utilizzato in modo continuo e senza interruzioni, ossia al 100% della sua capacità trasmissiva. Ci si trova pertanto in un regime di **trasmissione continua**, nel quale il flusso di dati non subisce pause dovute all'attesa delle conferme.



CONDIZIONE CHE CI IDENTIFICA LA
TRASMISSIONE CONTINUA NELLE
TELECOMUNICAZIONI Wi-Fi.

$$W \cdot T_{tx} > T_{RTT} + T_{tx}$$

Questa condizione esprime il requisito affinché il mittente possa mantenere la linea occupata in modo continuo, senza periodi di inattività dovuti all'attesa delle conferme.

W è la dimensione della finestra, Ttx è il tempo di trasmissione di una singola trama, TRTT è il Round Trip Time.

A questo punto possiamo definire il throughput:

$$Throughput = \min \left(\underbrace{C}_{\text{capacità della linea}}, \frac{\underbrace{W}_{\text{dimensione finestra}} \cdot \underbrace{MSG}_{\text{bit utili per trama}}}{\underbrace{T_{RTT}}_{\text{round trip time}} + \underbrace{T_{tx}}_{\text{tempo trasmissione trama}}} \right)$$

Il minimo con C compare perché il throughput non può mai superare la **capacità fisica del canale**.

Se la finestra W è piccola, il mittente non riesce a mantenere il canale sempre occupato.

In questo caso:

$$\frac{W \cdot MSG}{T_{RTT} + T_{tx}} < C$$

Se la finestra è grande abbastanza da coprire il RTT:

$$\frac{W \cdot MSG}{T_{RTT} + T_{tx}} > C$$

Qui la formula “di finestra” darebbe un valore superiore alla capacità fisica.

Ma fisicamente è impossibile trasmettere più di C bit al secondo.

Quindi:

$$Throughput = C$$

Esempio.

Quesito 1: Quanto tempo impiego per trasmettere 6 pacchetti assumendo che non ci siano errori?

Dati: MSG=500 bytes, $W = 4$, $C = 2$ Mbps, Propagation = 16ms = RTT.

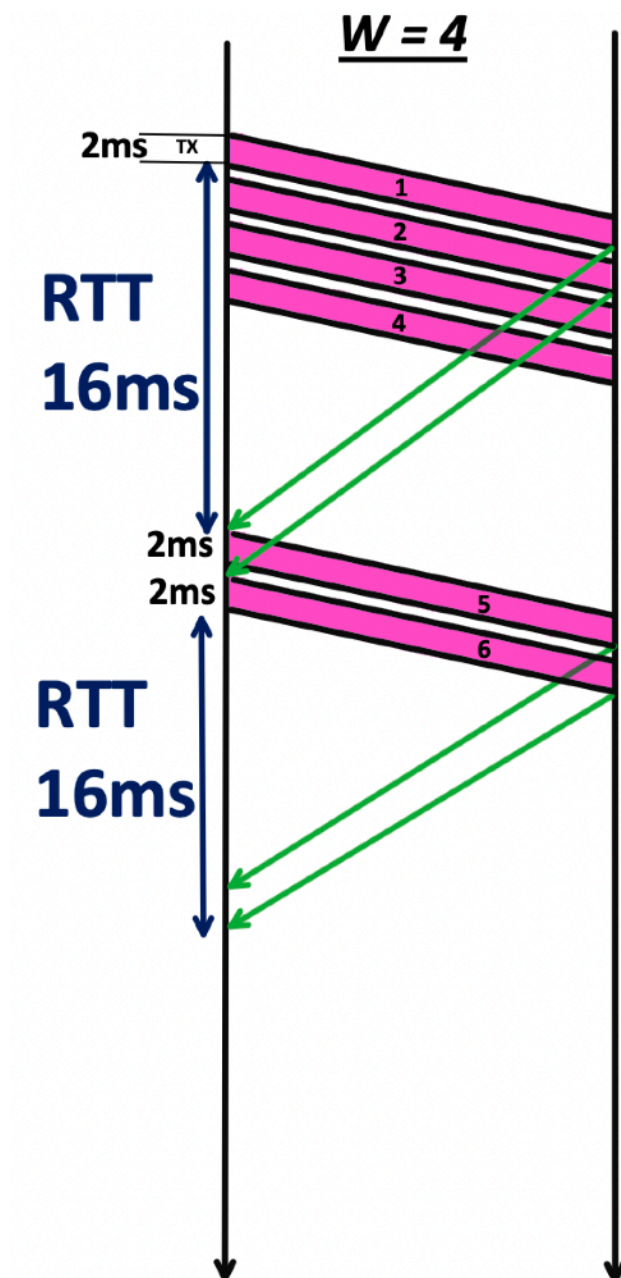
[Risultato 38ms]

Svolgimento.

Intanto troviamo subito il tempo di trasmissione di una trama.

$$T_{tx} = \frac{500 \cdot 8}{2 \text{ Mbps}} = 2 \text{ ms}$$

Il tempo di trasmissione della trama è quindi **2 millisecondi**. Gli header sono stati trascurati.



Sommiamo i tempi: 2ms+16ms+2ms+2ms+16ms = **38ms**.

Gli ack sono stati trascurati.

Quesito 2: Siamo in condizioni di trasmissione continua?

$4 \cdot 2 > 16 + 2$? No, 8 è minore di 18.

Quesito 3: Qual è la dimensione minima del messaggio *in bytes* per arrivare ad avere una trasmissione continua?

$$W \cdot T_{tx} > T_{RTT} + T_{tx}$$

$$W \cdot T_{tx} - T_{tx} > T_{RTT}$$

$$(W - 1) T_{tx} > T_{RTT}$$

$$\text{Sostituisco } T_{tx} = \frac{MSG}{C}$$

$$(W - 1) \frac{MSG}{C} > T_{RTT}$$

$$MSG > \frac{C T_{RTT}}{W - 1}$$

Sostituendo i valori del problema:

$$MSG \geq \frac{16 \text{ ms} \cdot 2 \text{ Mbps}}{3}$$

$$2 \text{ Mbps} = 2\,000\,000 \text{ bit/s}$$

$$\frac{2\,000\,000}{8} = 250\,000 \text{ Byte/s}$$

$$16 \text{ ms} = 0,016 \text{ s}$$

E allora è ovvio che:

$$MSG \geq \frac{0,016 \cdot 250\,000}{3}$$

E che quindi:

$$MSG \geq \frac{4\,000}{3}$$

$$MSG \geq 1\,333,33 \text{ Byte}$$

Quel valore (≈ 1333 byte) è la dimensione minima del messaggio (per trama) che, con $W=4$, ti consente di avere **trasmissione continua**, cioè di mantenere la linea sempre occupata senza “buchi” dovuti all’attesa degli ACK.

Quesito 4: Adesso l’RTT aumenta di 1ms ad ogni pacchetto.

Il pacchetto 1 ha RTT pari a 10ms,

Il pacchetto 2 ha RTT pari a 11ms,

Il pacchetto 3 ha RTT pari a 12ms,

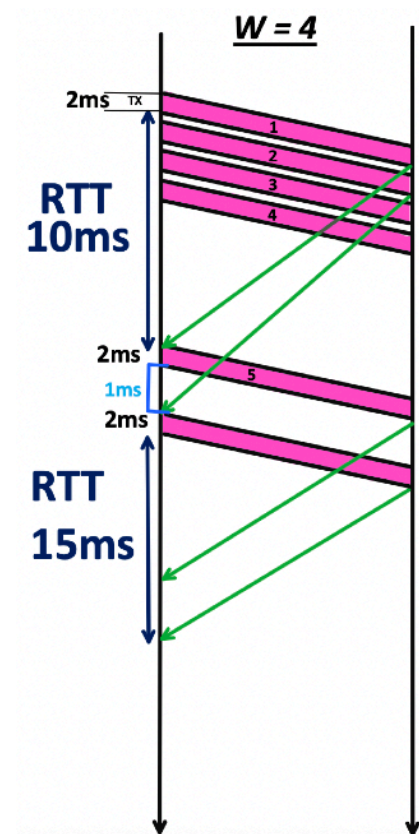
Il pacchetto 4 ha RTT pari a 13ms,

Il pacchetto 5 ha RTT pari a 14ms,

Il pacchetto 6 ha RTT pari a 15ms.

Quanto tempo impiego ora?

$$2 + 10 + 2 + 1 + 2 + 15 = 32 \text{ ms.}$$



Adesso prestiamo molta attenzione a quanto segue.

Bandwidth–Delay Product (BDP).

$$W_{\text{bit}} \approx C \cdot RTT$$

La finestra deve essere almeno pari al prodotto banda–ritardo per garantire una trasmissione continua.

Esempi di alcuni prodotti banda/ritardo.

Network	RTT(ms)	Rate(Kbps)	BxD (bytes)
Ethernet	3	10000	3750
T1, TransUS	60	1544	11580
T1, Satellite	480	1544	92640
T3, TransUS	60	45000	337500
Gigabit TransUS	60	1000000	7500000

10.000 kbps significa:

$$10\,000 \times 1000 = 10\,000\,000 \text{ bit/s}$$

Ora convertiamo in **Byte/s** (perché spesso il prof ragiona in Byte):

$$\frac{10\,000\,000}{8} = 1\,250\,000 \text{ Byte/s}$$

$$3 \text{ ms} = 0,003 \text{ s}$$

$$BDP = C \cdot RTT$$

$$BDP = 1\,250\,000 \cdot 0,003$$

$$BDP = 3\,750 \text{ Byte}$$

Questo significa che per poter usare questo link di 10Mbps al 100% dovrei mettere in volo 3750 bytes non riscontrati.

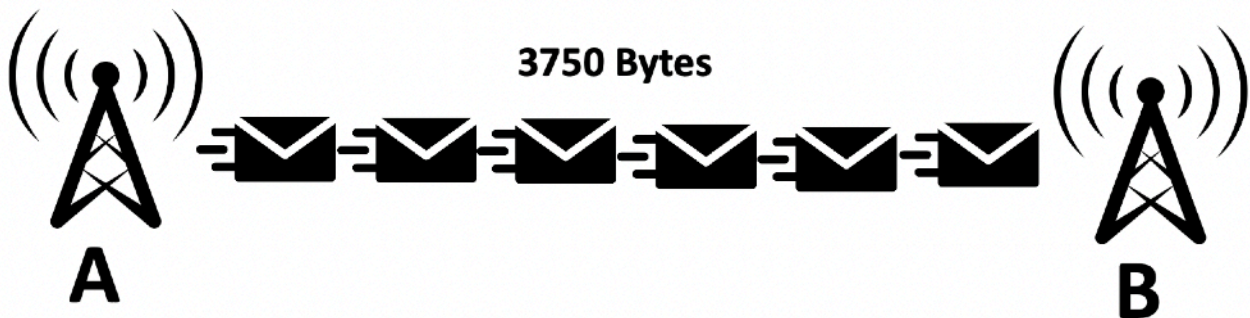
Per sfruttare un link da **10 Mbps** con **RTT = 3 ms** al **100%**, occorre mantenere in volo (cioè non ancora riscontrati) almeno:

$$BDP = C \cdot RTT = 10 \text{ Mbps} \cdot 3 \text{ ms} = 3750 \text{ Byte}$$

Se il “serpentone” di pacchetti in volo è **più corto di 3750 Byte**, il mittente prima o poi si fermerà in attesa degli ACK e il link non è pienamente utilizzato.

Se il serpentone è **pari o superiore a 3750 Byte**, il tubo è pieno, ho trasmissione continua e throughput pari alla capacità C.

Il “serpentone” è esattamente la visualizzazione del Bandwidth-Delay Product: la lunghezza minima della coda di pacchetti in volo necessaria per mantenere il link sempre occupato.



Nota: più è alto il prodotto banda-ritardo e più per andare veloci dobbiamo possedere **buffer** grandi.

Buffer lato mittente.

Immagina di usare sliding window. Tu mandi pacchetti.

Ma finché non arriva l'ACK, non puoi cancellarli dalla memoria. Perché? Se un pacchetto si perde, devi ritrasmetterlo.

Quindi il mittente: copia il pacchetto nel buffer, lo invia, lo tiene in memoria, lo elimina solo quando arriva l'ACK.

Se ho un collegamento intercontinentale, 10 Gbps e RTT = 100 ms, allora:

$$BDP = 10^{10} \cdot 0,1 = 10^9 \text{ bit} \approx 125 \text{ MB}$$

$$125 \text{ MB} = 125\,000\,000 \text{ byte}$$

Per saturare la linea servono buffer dell'ordine di **centinaia di megabyte**.

E quindi quanti pacchetti fisicamente in volo?

Stiamo parlando di bytes, ma quanti pacchetti devo mantenere in volo?

ESERCIZIO BIBBIA DA CAPIRE

Link Rate = 1 Mbps, MSG = 1500bytes, RTT = 500ms, W=16.

1. Trova il prodotto in bytes banda-ritardo per sapere come usare al 100% la capacità della linea.
2. Determinare la finestra W che consente di usare il rate al massimo.
3. Determinare il throughput.
4. Dimostrare che, se ottengo una determinata W al punto 2, mi avvicino il più possibile alla velocità della linea.
5. Se hai compreso questi quattro punti sorridi.

Svolgimento.

1.

$$BDP = \frac{1\,000\,000}{8} \text{ (Byte/s)} \cdot 0,5 \text{ s}$$

$$BDP = 62\,500 \text{ Byte}$$

2.

$$W = \frac{62\,500}{1\,500} \approx 41,67$$

Quindi, W circa 40.

3.

$$Thr = \frac{16 \cdot 1500 \text{ Byte}}{500 \text{ ms} + \frac{1500 \text{ Byte}}{1 \text{ Mbps}}}$$

$$Thr \approx 375 \text{ Kbps}$$

4.

$$Thr = \frac{40 \cdot 1500 \text{ Byte}}{500 \text{ ms} + \frac{1500 \text{ Byte}}{1 \text{ Mbps}}}$$

$$Thr \approx 938 \text{ Kbps}$$

Vedi la differenza enorme? Aumentare la finestra mi ha portato da **375 Kbps** a quasi **1 Mbps**.
Sto quasi alla velocità della linea, non so se rendo.

5.

:)

(Nota, con W=42 il throughput era a 0.98 Mbps, quasi al 100%).

PROTOCOLLO GO-BACK-N

Il **Go-Back-N (GBN)** è un protocollo ARQ a finestra scorrevole.

Permette al mittente di inviare **più pacchetti consecutivamente senza attendere l'ACK di ciascuno**, fino a un massimo definito dalla dimensione della finestra **W**.

Il ricevitore invia **ACK cumulativi**. Se riceve correttamente fino al pacchetto 3 allora manda **Ack = 3**, e significa che ha ricevuto il pacchetto 1,2 e 3.

Il ricevitore accetta le trame solo se le riceve in sequenza. Se vede che un pacchetto è fuori sequenza manda un NACK (oppure far scattare un timeout).

Se ricevo 0,1,2,3,4 ma il 3 si perde, io ricevitore manderò un NACK per il 4.

Con k bit posso rappresentare:

$$N = 2^k$$

E la finestra è:

$$W \leq 2^k - 1$$

ESEMPIO GBN

$B = 3$ bit, $N = 8 = \{0,1,2,3,4,5,6,7\}$, W al massimo 7.

Ecco la finestra:

4 5 6 7 0 1 2 3 4 5 6 7

Vediamo il disegno:

Nel caso considerato, il sistema opera in condizioni di trasmissione continua.

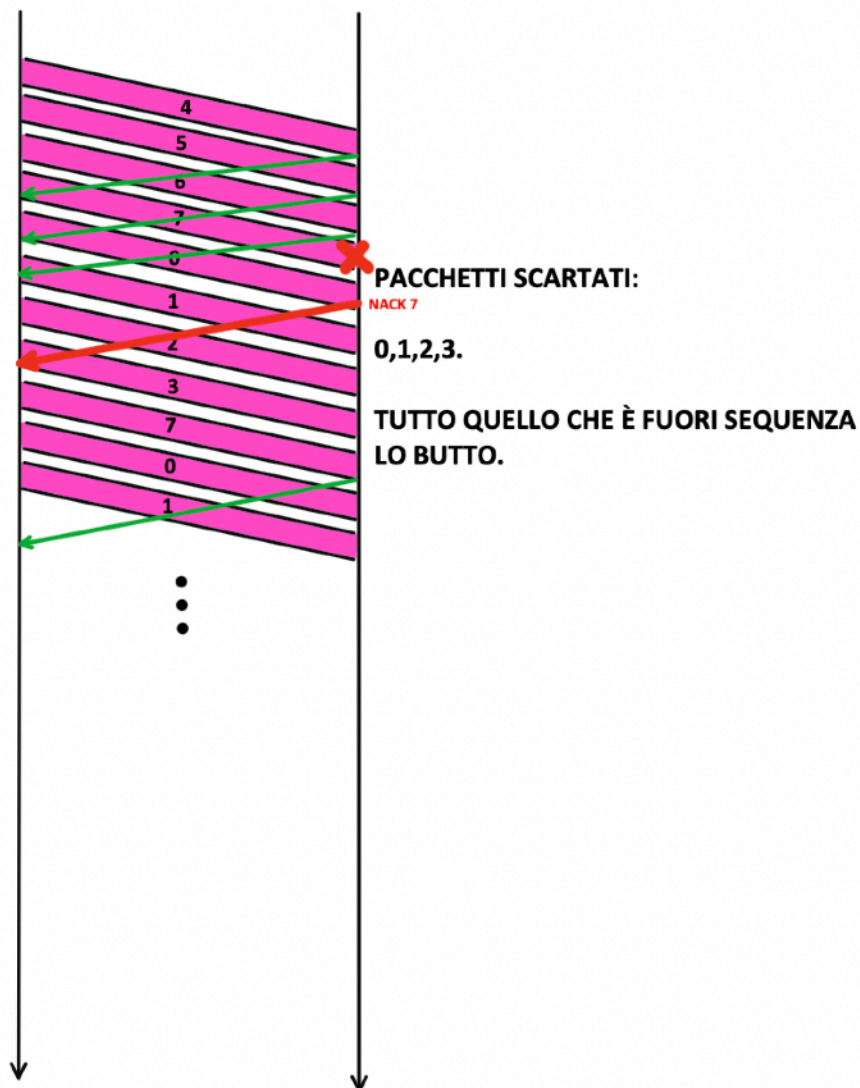
Il mittente invia i pacchetti 4, 5, 6 e successivamente il 7. Nel frattempo riceve l'ACK relativo al pacchetto 4. La ricezione di tale riscontro determina lo scorrimento della finestra di una posizione in avanti; da qui la denominazione di protocolli a finestra scorrevole (**sliding window**).

La trasmissione prosegue: il pacchetto 7 viene inviato ma si perde lungo il canale. Nel frattempo giungono gli ACK relativi ai pacchetti 5 e 6; il mittente, potendo far avanzare ulteriormente la finestra, continua l'invio dei pacchetti successivi (0, poi 1, quindi 2, e così via).

Successivamente viene trasmesso anche il pacchetto 3. A questo punto, tuttavia, perviene un NACK (o si verifica il timeout) relativo al pacchetto 7 precedentemente smarrito: si attiva così il meccanismo del protocollo **Go-Back-N**.

Il punto fondamentale è il seguente: il mittente avrebbe potuto continuare a trasmettere ulteriori pacchetti fintanto che la finestra lo consentiva; tuttavia, una volta rilevata la perdita di un pacchetto precedente, occorre interrompere l'invio di nuovi dati. Infatti, nel protocollo Go-Back-N, il ricevitore scarta tutti i pacchetti successivi a quello mancante. Proseguire la trasmissione non avrebbe utilità, poiché tali pacchetti verrebbero comunque ignorati. Pertanto, il mittente si arresta non perché abbia raggiunto il limite della finestra, ma perché l'ulteriore invio sarebbe inefficace.

Il ricevitore, mediante l'invio di un NACK, comunica che il pacchetto 7 non è stato ricevuto correttamente. Ciò implica che tutti i pacchetti fino al numero 6 sono stati acquisiti con successo, mentre il pacchetto 7 non è giunto a destinazione.



Quando il mittente rileva la perdita di un pacchetto — attraverso un NACK o la scadenza del timeout — deve riprendere la trasmissione a partire dal primo pacchetto non correttamente ricevuto. Nel caso in esame, avendo trasmesso il pacchetto 7 senza che questo sia stato consegnato con successo, il mittente riprende l'invio proprio dal pacchetto 7, ritrasmettendo da quel punto in avanti

secondo il meccanismo previsto dal protocollo Go-Back-N.

Nota: siccome gli **ack sono di tipo cumulativo**, il ricevitore poteva mandare semplicemente *l'ack del 6*. Non sarebbe cambiato nulla.

Piggybacking

L'ACK “viaggia in groppa” ai dati.

Il **piggybacking** è una tecnica utilizzata nei protocolli di trasporto bidirezionali per inserire l'ACK all'interno di un pacchetto dati che sta già viaggiando nella direzione opposta, invece di inviare un ACK separato.

(Concetto solo teorico da sapere).



PROTOCOLLO SELECTIVE REPEAT

Il **Selective Repeat ARQ** è un protocollo a finestra scorrevole in cui: il mittente può trasmettere più pacchetti consecutivamente (come in Go-Back-N), ma in caso di errore ritrasmette solo i pacchetti effettivamente persi.

Nel protocollo **Selective Repeat**, il ricevitore non scarta più i pacchetti che arrivano successivamente a un pacchetto perduto, come avviene invece nel protocollo Go-Back-N.

Al contrario, esso accetta e memorizza i pacchetti ricevuti fuori ordine.

L'obiettivo è ottenere una **ritrasmissione selettiva**: qualora un pacchetto venga smarrito, si richiede esclusivamente la ritrasmissione di quel singolo pacchetto, senza coinvolgere quelli già ricevuti correttamente.

A tal fine, il protocollo prevede l'adozione di una **finestra scorrevole anche lato ricevitore**.

Ciò consente di gestire e memorizzare pacchetti fuori sequenza.

Quindi abbiamo **Ws** (finestra mittente) e **Wr** (finestra ricevente).

Ogni volta che ricevo un pacchetto sposto la finestra di ricezione di una casella.

La finestra del ricevitore scorre solo quando viene ricevuto il pacchetto con numero di sequenza uguale al bordo inferiore della finestra. I pacchetti fuori ordine: vengono accettati, vengono bufferizzati, ma non fanno avanzare la finestra.

In questo protocollo **Ws + Wr ≤ N**.

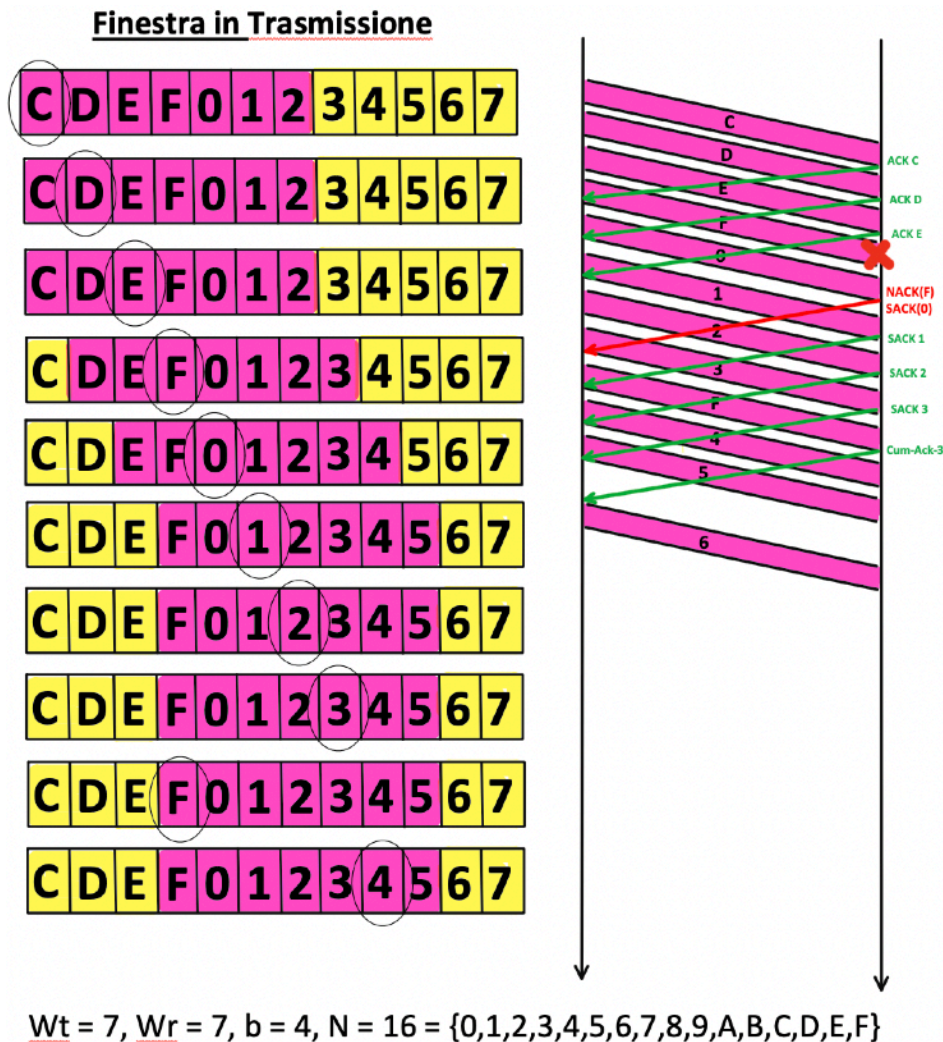
Il mittente trasmette i pacchetti C, D ed E. Nel frattempo riceve il riscontro relativo al pacchetto C e, conseguentemente, la finestra di trasmissione si sposta di una posizione in avanti.

Successivamente il mittente invia il pacchetto F, che tuttavia si perde lungo il canale. Prosegue quindi con l'invio dei pacchetti successivi (ad esempio 0 e 1). Nel frattempo giunge il riscontro del pacchetto E.

Dal lato del ricevitore, i pacchetti successivi a F vengono correttamente ricevuti ma risultano fuori ordine. Pertanto, essi vengono memorizzati in un buffer e il ricevitore genera un **Selective Acknowledgment (SACK)** per indicare i pacchetti ricevuti correttamente (ad esempio il pacchetto 0), insieme a una segnalazione di mancata ricezione (NACK) per il pacchetto F.

Quando il mittente riceve il NACK relativo a F, procede alla ritrasmissione selettiva esclusivamente di tale pacchetto. Nel periodo in cui attende la ritrasmissione di F, il ricevitore continua a inviare unicamente SACK per i pacchetti ricevuti fuori ordine, non potendo emettere un ACK cumulativo, poiché manca ancora un elemento della sequenza.

Una volta ricevuto correttamente il pacchetto F, il ricevitore può finalmente ricostruire la sequenza completa e inviare un ACK cumulativo relativo all'ultimo pacchetto consecutivo ricevuto correttamente (nel caso in esame, fino al pacchetto 3).

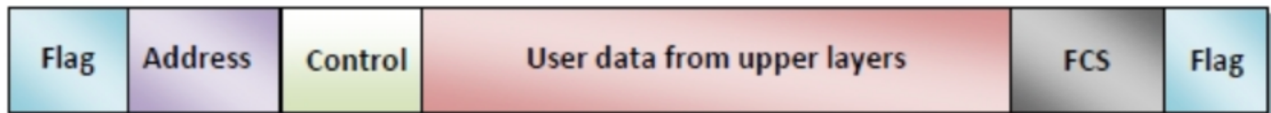


HDLC – High-Level Data Link Control

HDLC (High-Level Data Link Control) è un protocollo di livello 2 (Data Link) del modello OSI.

Wi-Fi usa il **proprio formato di frame MAC**, completamente diverso da HDLC.

HDLC non è presente nella trama Wi-Fi. Wi-Fi utilizza il formato di frame definito dallo standard IEEE 802.11, che è un protocollo di livello 2 diverso da HDLC.



Il campo **flag** è **1 byte**: è un delimitatore di trama, tipicamente **01111110**, ossia 0x7e.

Il campo **address** indica il destinatario della trama. Tipicamente 1 byte (ma può essere esteso).

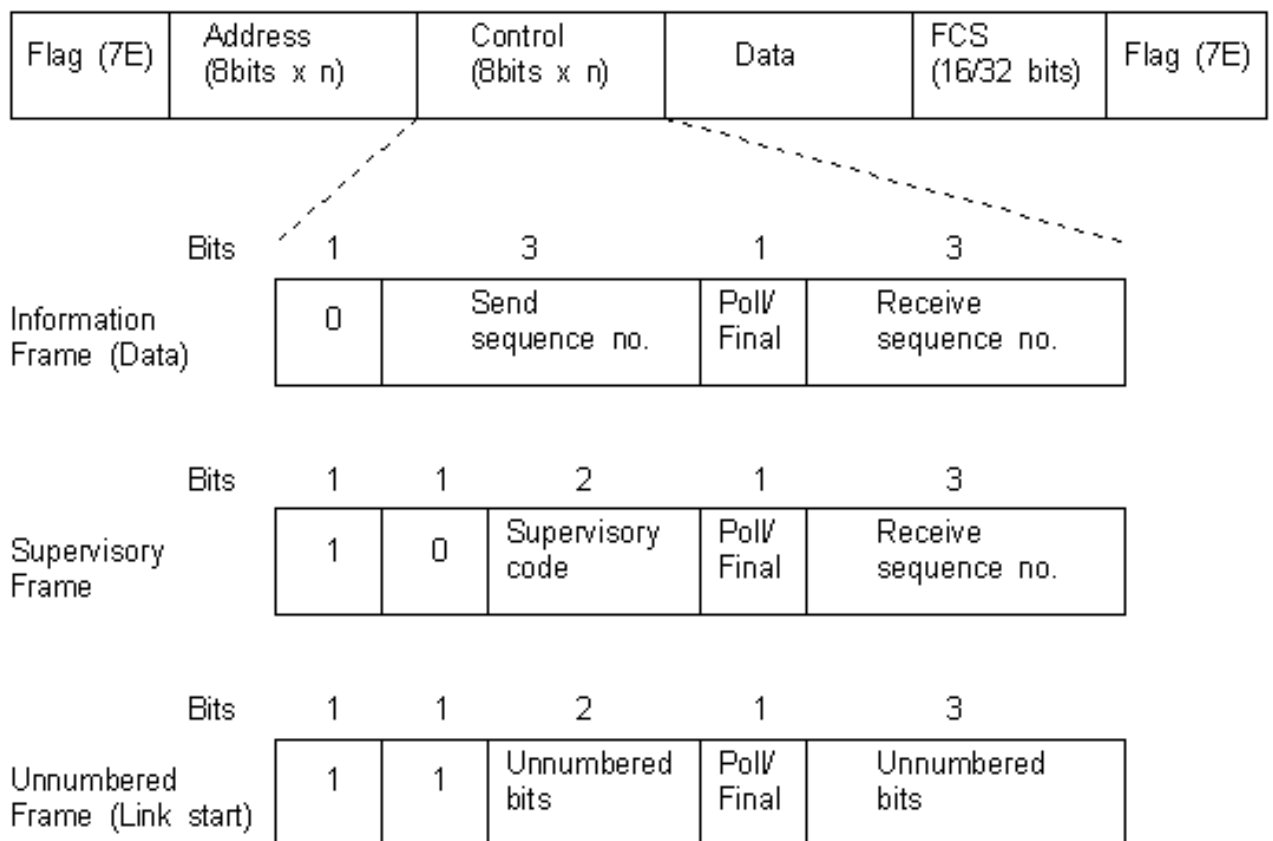
Il campo **control** è quello più importante (1-2 bytes).

Innanzitutto diciamo che esistono 3 tipi di trame: **I, S ed U**.

I = Trame d'informazione, ossia quelle che cominciano col bit **0**.

S = Trame di controllo, come l'ACK. Cominciano con **10**.

U = Trame di management, servono per segnalare qualcosa, cominciano con **11**.



L'immagine è presa dal manuale ufficiale che spiega il protocollo.

Le Supervisory (S-frame) hanno il codice, questo codice può essere:

- 00 = Ack cumulativo.
- 01 = Receive not Ready (Buffer pieno, non posso ricevere).
- 10 = Reject, ho avuto un nack.
- 11 = Ho un selective reject.

Bit Stuffing (usato in HDLC) - Su questo concetto ci sono esercizi d'esame

HDLC è un protocollo **orientato al bit**.

La trama inizia e finisce con il flag: 01111110 = 0x7e.

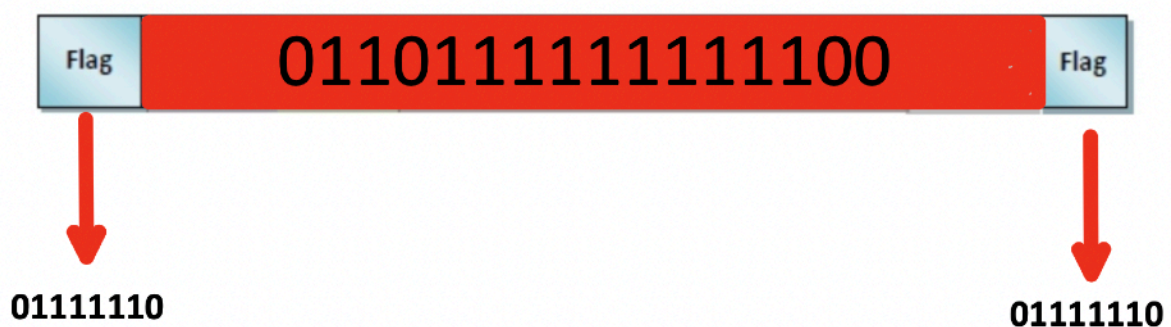
Problema enorme: E se questa sequenza appare nei dati?
Il ricevitore penserebbe che la trama sia finita.

Soluzione: Bit Stuffing.

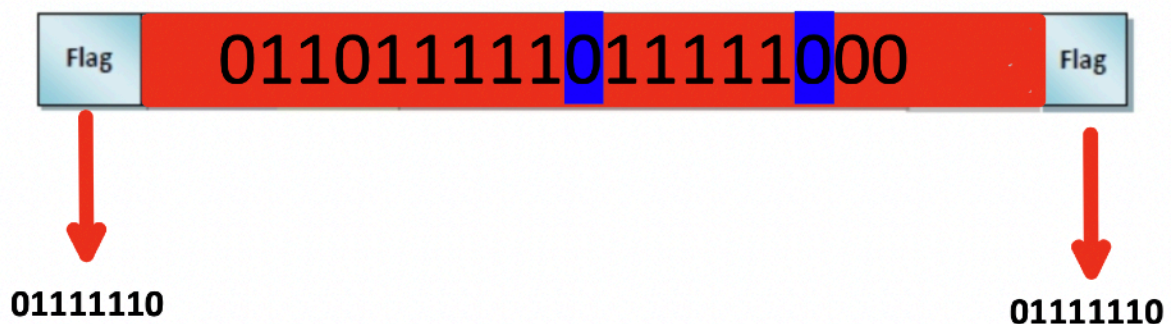
Dopo 5 bit consecutivi a 1, il trasmettitore inserisce automaticamente uno 0.

Supponiamo di voler spedire questi dati: **0110111111111100**.

Applichiamo la procedura di bit stuffing.



Applichiamolo:



Il ricevitore: ogni volta che vede 5 “1” consecutivi elimina il bit successivo se è 0. Se invece dopo 5 “1” arriva un 1, allora è davvero il flag.

Byte Stuffing (usato nei protocolli orientati al byte).

Qui le trame sono delimitate da un **byte speciale: 0x7E**.

E se quel byte compare nei dati?

Si usa un byte di **escape**, ad esempio **0x7D**.

Dopo il byte di escape (0x7D), il byte originale viene XORato con 0x20. È una regola fissa del protocollo, questo da come risultato 0x5E..

Regola: se nei dati compare 0x7E lo trasformo in **0x7D 0x5E**.

Supponiamo di voler trasmettere: 11 3F 7D 7E 02.

Tx: 7E 11 3F **7D 5D 7D 5E** 02 7E.

0x11 non è né 7E né 7D → si trasmette così com'è.

0x3F non è speciale → si trasmette così.

0x7D Questo è il byte di *escape* stesso.

Va trasformato. Calcoliamo: 0x7D XOR 0x20,

01111101

00100000

01011101

0x5D

Quindi si trasmette **7D 5D**.

0x7E questo è il flag. Va escapato.

Calcoliamo: 0x7E XOR 0x20

01111110

00100000

— — — —

01011110

Risultato:

0x5E

Quindi si trasmette:

7D 5E

0x02 non è speciale → si trasmette così.

Fine wifi.



Wireless Cellular Network

Nel corso di **FoTEL**, per studiare la propagazione si parte da un modello ideale.

Si assume un'antenna **isotropica**, ossia un'antenna **ideale** che irradia potenza in modo uniforme in tutte le direzioni dello spazio.

Questo significa che la potenza totale trasmessa P si distribuisce uniformemente su una superficie sferica centrata nel punto di trasmissione.

Non è un'antenna reale.

È un modello matematico di riferimento.



Nel modello base di propagazione wireless si assume un'antenna isotropica che irradia potenza uniformemente in tutte le direzioni. La potenza si distribuisce su una superficie sferica di area $4\pi d^2$, per cui la densità di potenza decresce con il quadrato della distanza.

Se l'antenna trasmette potenza P_t , a distanza d l'energia si è distribuita su una superficie sferica di area:

$$A = 4\pi d^2$$

Quindi la **densità di potenza** (power density) a distanza d è:

$$S(d) = \frac{P_t}{4\pi d^2} \quad S(d) = \frac{W}{m^2}$$

La potenza decresce con il quadrato della distanza.

Un'antenna ricevente non cattura tutta la potenza della sfera, intercetta solo la potenza che attraversa la sua **area efficace** A_e .

La potenza ricevuta sarà quindi:

$$P_r = S(d) \cdot A_e$$

$$P_r = \frac{P_t}{4\pi d^2} \cdot A_e$$

Ae si misura in:

$$\mathbf{m^2}$$

Quini è ovvio che:

$$P_r = (W/m^2) \cdot (m^2) = \mathbf{W}$$

L'area efficace è legata al **guadagno dell'antenna G** e alla **lunghezza d'onda (lambda in metri)**:

$$A_e = \frac{\lambda^2}{4\pi} G$$

Il guadagno dell'antenna è un valore adimensionale.

Se l'antenna è isotropica $G = 1$.

Quindi, in questo corso:

$$A_e = \frac{\lambda^2}{4\pi}$$

Da qui viene fuori la Formula di Friis (spazio libero).

$$P_r = P_t G_t G_r \left(\frac{\lambda}{4\pi d} \right)^2$$

Unità:

- $P_t, P_r \rightarrow$ Watt
- $G_t, G_r \rightarrow$ adimensionali
- $\lambda, d \rightarrow$ metri

$P_r(W)$ rappresenta la potenza ricevuta dall'antenna ricevente, cioè la potenza elettromagnetica che l'antenna riesce effettivamente a catturare dal campo incidente.
Il prof la definisce $P_r(d)$ = potenza ricevuta ad una certa distanza d .

La formula vale solo in spazio libero e nessun ostacolo.

Spesso si introduce un fattore di perdite complessive $L \geq 1$ (cavi, connettori, mismatch, filtri, ecc.).

L è **adimensionale** e rappresenta **perdite** (quindi sta al denominatore).

Se non ci sono perdite ideali $L=1$.

Il Professore scrive anche questa seconda formula, sostituendo $\lambda=c/f$, dove c è la velocità della luce.

$$P_r = \frac{P_t G_t G_r}{L} \left(\frac{c}{4\pi d f} \right)^2$$

La formula vale solo in spazio libero e nessun ostacolo.

Stiamo considerando una situazione ideale in cui due antenne si trovano a una certa distanza d in **spazio libero**, ossia in assenza di ostacoli, superfici riflettenti, pavimenti o pareti. In tali condizioni, la propagazione avviene secondo il modello di spazio libero (free-space), e la potenza ricevuta decresce proporzionalmente a

$$d^{-2}.$$

Quando però si introduce una superficie riflettente, come ad esempio un pavimento, il comportamento del canale cambia. Il segnale non si propaga più soltanto lungo un percorso diretto, ma subisce riflessioni che generano cammini multipli. Di conseguenza, l'attenuazione non segue più una legge inversamente proporzionale al quadrato della distanza.

In ambienti reali, la potenza ricevuta decresce secondo una legge più generale del tipo:

$$P_r(d) \propto d^{-n}$$

dove n è l'**esponente di path loss**, che dipende dall'ambiente di propagazione.

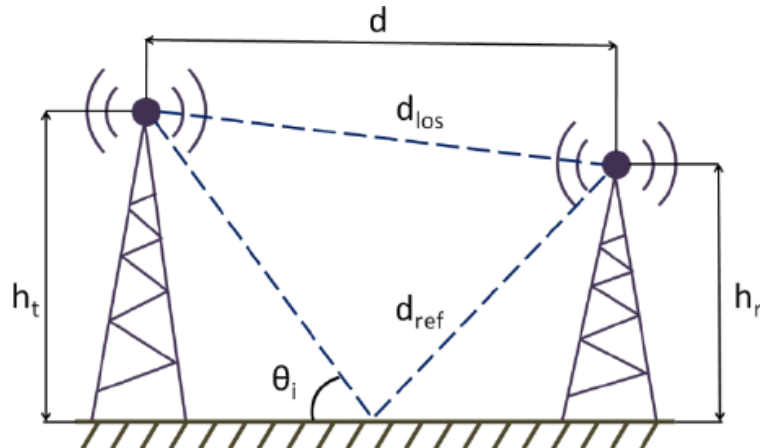
In spazio libero $n=2$; in presenza di riflessioni, ostacoli o fenomeni di multipath, il valore di n aumenta (tipicamente tra 2 e 6), indicando una più rapida attenuazione del segnale con la distanza.

Modello a due raggi (Two-Ray Ground Reflection) – LOS

Il modello a **due raggi** descrive una situazione in cui tra trasmettitore e ricevitore esistono: un **raggio diretto** (Line Of Sight, LOS) e un raggio riflesso dal suolo.

Per distanze elevate, l'attenuazione media della potenza ricevuta segue una legge proporzionale a d^{-4} , in contrasto con il comportamento d^{-2} dello spazio libero.

È un modello più realistico rispetto al free-space, perché tiene conto della riflessione sul terreno.



Antenna trasmittente a altezza h_t (m), antenna ricevente a altezza h_r (m), distanza d (m) orizzontale tra le antenne e presenza di un piano riflettente (il suolo).

$$P_r \approx \frac{P_t G_t G_r h_t^2 h_r^2}{d^4}$$

D alla meno 4 è un'attenuazione mostruosa.

CONVERSIONI DECIBEL E DBM

Chiariamolo una volta per tutte: questa è una di quelle cose che se la capisci bene una volta, poi non ti confonde più nulla in Wireless.

Cos'è il decibel (dB)? Il **decibel (dB)** NON è una potenza. È un'unità **logaritmica** che esprime un **rapporto** tra due quantità.

$$L_{dB} = 10 \log_{10} \left(\frac{P_2}{P_1} \right)$$

È adimensionale.

Cos'è il dBm? Il **dBm** è una potenza assoluta. Significa decibel riferiti a 1 milliwatt. Quindi il riferimento è:

$$P_{dBm} = 10 \log_{10} \left(\frac{P}{1 \text{ mW}} \right)$$

$$1 \text{ mW} = 0 \text{ dBm}$$

Quindi, per il dBm, subito un conto rapido. 1 W = 1000 mW:

$$10 \log_{10}(1000) = 30 \text{ dBm}$$

Pertanto:

$$1 \text{ W} = 30 \text{ dBm}$$

Come si passa da Watt a dBm?

$$P_{dBm} = 10 \log_{10}(P[W] \cdot 1000)$$

Come si torna da dBm a Watt?

$$P[W] = \frac{10^{(P_{dBm}/10)}}{1000}$$

Esempio sui decibel.

I dati sono:

$$P_a = 1 \text{ W}$$

$$P_b = 50 \text{ mW}$$

Troviamo i decibel.

$$50 \text{ mW} = 0.05 \text{ W}$$
$$L_{dB} = 10 \log_{10} \left(\frac{1}{0.05} \right)$$

Quindi è ovvio che:

$$13.01$$

Questo significa che:

$$P_a \text{ è circa } 13 \text{ dB maggiore di } P_b$$

La potenza di riferimento.

La potenza di riferimento è la potenza rispetto alla quale stai confrontando un'altra potenza.

La potenza P_{ref} è quella presente al denominatore nel calcolo dei decibel.

$$dB = 10 \log_{10} \left(\frac{P}{P_{ref}} \right)$$

E da questa formula posso tornare alla potenza P :

$$P = P_{ref} \cdot 10^{(dB/10)}$$

Cioè se conosco il rapporto in dB e conosco la potenza di riferimento, posso ricostruire la potenza reale. Ed è un risultato importantissimo.

Nel caso del **dBm** il riferimento è:

$$P_{ref} = 1 \text{ mW}$$

Rapporto segnale/rumore (SNR)

Il rapporto segnale/rumore (SNR) è proprio un rapporto di potenze.

$$\text{SNR} = \frac{P_s}{P_n}$$

È un numero puro (**adimensionale**).

Dove: P_s = potenza del segnale e P_n = potenza del rumore.

Dato che è un rapporto, lo esprimiamo in dB:

$$\text{SNR}_{dB} = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

Non serve aver preso 30 ad analisi per capire che uso la formula dei logaritmi:

$$\log \left(\frac{a}{b} \right) = \log(a) - \log(b)$$

E quindi:

$$\text{SNR}_{dB} = 10 \log_{10}(P_r) - 10 \log_{10}(N)$$

Ma cos'è $10 \log_{10}(P)$? È proprio la definizione di potenza in dB rispetto a un riferimento.

Nel caso del dBm:

$$P_r(dBm) = 10 \log_{10} \left(\frac{P_r}{1 \text{ mW}} \right)$$

$$N(dBm) = 10 \log_{10} \left(\frac{N}{1 \text{ mW}} \right)$$

E quindi arriviamo ad una **formula importante**:

$$P_r(dBm) - N(dBm) = \text{SNR}(dB)$$

Esempio numerico:

$$P_r = -70\text{dBm}$$

$$N = -100\text{dBm}$$

Allora

$$\text{SNR} = -70 - (-100) = 30\text{ dB}$$

Il segnale è 1000 volte il rumore.

Dimostrazione.

$$\text{SNR}_{dB} = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$30 = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$3 = \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$10^3 = \frac{P_s}{P_n}$$

$$\frac{P_s}{P_n} = 1000$$

Se il rapporto segnale rumore è nullo?

$$\text{SNR} = 0\text{ dB}$$

Il segnale ha la stessa potenza del rumore.

Attenzione: non è zero segnale. È **segnale = rumore**.

Se il rapporto segnale rumore è -10 dB?

$$\text{SNR} = -10 \text{ dB}$$

$$\text{SNR}_{dB} = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$-10 = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$-1 = \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$10^{-1} = \frac{P_s}{P_n}$$

Il segnale è 10 volte più piccolo del rumore.

Domanda: Quando è che il segnale è esattamente la **metà** del rumore?

Risposta: Quando il rapporto è il seguente.

$$\text{SNR} = -3 \text{ dB}$$

Dimostrazione.

$$-3 = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$-0.3 = \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$10^{-0.3} = \frac{P_s}{P_n}$$

$$\frac{P_s}{P_n} \approx 0.5$$

Il rumore è circa il doppio del segnale.

Ora, il Professore dalla formula di Friis, ci ricava una formula in scala **lineare**.
Scriviamola subito senza mostrare come ci è arrivato (non ci interessa).

$$P_r(d) = P_r(d_0) \left(\frac{d_0}{d} \right)^2$$

Se conosci la potenza ricevuta a una certa distanza di riferimento d_0 , puoi calcolare la potenza ricevuta a qualsiasi altra distanza d usando il rapporto tra le distanze al quadrato.

Ora esprimiamo la formula nel modo seguente:

$$P_r(d)_{dB} = 10 \log_{10} \left[P_r(d_0) \left(\frac{d_0}{d} \right)^n \right]$$

dove n è l'esponente di path loss: 2 nello spazio libero.

Usiamo le proprietà dei logaritmi:

$$P_r(d)_{dB} = 10 \log_{10} (P_r(d_0)) + 10 \log_{10} \left(\left(\frac{d_0}{d} \right)^n \right)$$

Portiamo fuori l'esponente:

$$= 10 \log_{10}(P_r(d_0)) + 10n \log_{10} \left(\frac{d_0}{d} \right)$$

Ma al lato sinistro anche applichiamo il logaritmo:

$$10 \log_{10} \left(\frac{P_r(d)}{1mW} \right)$$

Il lato sinistro diventa $P_r(d)$ in dBm perché stiamo applicando la definizione di dBm, che è proprio $10 \log_{10}$ della potenza rispetto a 1 mW.

Quindi, la formula in scala **logaritmica** è:

$$P_r(d)_{dBm} = 10 \log_{10} (P_r(d_0)) + 10n \log_{10} \left(\frac{d_0}{d} \right)$$

Quella formula è corretta **solo se $P_r(d_0)$** è espresso in **milliwatt (mW)**.

Ora, sulla base di questa affermazione, è ovvio dire che:

$$10 \log_{10} (P_r(d_0)) = P_r(d_0)_{dBm}$$

Il rapporto segnale/rumore è quasi una metafora esistenziale. Il segnale è informazione. Il rumore è disordine. SNR alto, il messaggio passa. SNR basso, il caos domina.

Non serve eliminare il rumore.

Serve solo che il segnale sia abbastanza forte rispetto ad esso.

Questa è una lezione che può essere applicata nella vita, prima come esseri umani e poi come figure professionali

Le emozioni negative non si eliminano.

Sono il rumore di fondo dell'esistenza.

La serenità non è assenza di rumore.

È quando il tuo segnale interno è più forte.

Ricorda che non serve perfezione.

Serve superare la soglia minima per funzionare bene.

Diminuzione della potenza ricevuta dovuta solo all'aumento della distanza.

$$L_p[d_0 \rightarrow d] = 10n \log_{10} \left(\frac{d}{d_0} \right)$$

è la perdita di propagazione (attenuazione) tra la distanza di riferimento d_0 e la distanza d .

È quanti dB di potenza perdi andando da d_0 a d . **Si misura in dB.**

La potenza di soglia.

La potenza di soglia è il punto oltre il quale il segnale diventa troppo debole per essere utile.

Spesso viene chiamata anche **sensibilità del ricevitore.**

Quando sono sotto soglia non sento più, quando passo sopra soglia sento.

ESERCIZIO PER CAPIRE

Possiedo la potenza ricevuta ad una certa distanza $d_0 = 10\text{m}$ di $0,1\text{W}$.

La potenza di soglia $P_{th} = -50\text{dBm}$.

La potenza scende con esponente $n = 3,7$.

Qual è il limite massimo di distanza dove posso sentire e, superato questo, non sento più?

Svolgimento.

Utilizziamo dapprima questa formula:

$$P_r(d) = P_r(d_0) \left(\frac{d_0}{d} \right)^2$$

Sostituiamo i dati

$$P_r(d) = 0.1 \left(\frac{10}{d} \right)^{3.7}$$

Ora converto -50dBm in Watt:

$$-50 \text{ dBm} = 10^{-8} \text{ W}$$

Quindi:

$$P_{th} = 10^{-8} \text{ W}$$

E allora impostiamo l'equazione in scala lineare e ricaviamo d:

$$0.1 \left(\frac{10}{d} \right)^{3.7} = 10^{-8}$$

$$d \approx 775 \text{ metri}$$

Okay, ora facciamolo in scala logaritmica:

$$P_r(d)_{dBm} = 10 \log_{10} (P_r(d_0)) + 10n \log_{10} \left(\frac{d_0}{d} \right)$$

Convertiamo $P_r(d_0)$ in dBm.

$$P_r(d_0)_{dBm} = 10 \log_{10}(100) = 20 \text{ dBm}$$

Dove 100 sono i milliwatt. (Sto dando per assodato le unità di misura, rileggi la teoria sopra).

E allora impostiamo l'equazione in scala logaritmica e ricaviamo d:

$$-50 = 20 + 10(3.7) \log_{10} \left(\frac{10}{d} \right)$$

$$d \approx 775 \text{ m}$$

Stesso risultato di prima, ma tutto in dB.

Pertanto, la distanza massima alla quale ci si può allontanare dalla stazione radio base mantenendo la potenza ricevuta almeno pari alla soglia $P_{th} = -50\text{dBm}$ è circa 780m.

Quindi a 780m sento -50dBm.

Teorema importante: **Alla distanza di circa 780 metri la potenza ricevuta coincide con la potenza di soglia. Tale distanza rappresenta il raggio massimo teorico di copertura nel modello considerato.**

$$\boxed{-50 \text{ dBm} = 10^{-8} \text{ W}}$$

Quindi a 780 m stavo ricevendo circa **10 nanowatt**.

Tipico esercizio d'esame (questo appena fatto).

Funzioni erf ed erfc – Cosa sono e perché compaiono nel Wireless.

Quando nel corso si parla di **probabilità di outage**, shadowing o rumore gaussiano, prima o poi compaiono due oggetti matematici:

$$\text{erf}(x) \quad \text{e} \quad \text{erfc}(x)$$

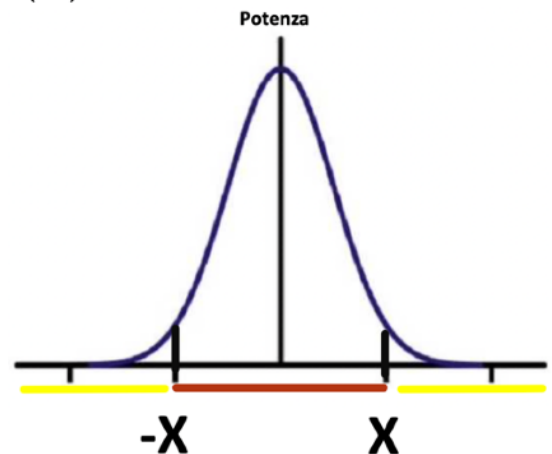
Erf significa *error function* (funzione errore): è una funzione matematica che nasce nello studio delle distribuzioni gaussiane.

$$\text{erfc}(x) = 1 - \text{erf}(x)$$

Erfc è la funzione errore complementare.

La gaussiana ti dice che il mondo reale fluttua. La erf ti dice quanto spesso quelle fluttuazioni ti fanno fallire.

Questa funzione (Erf) mi dice qual è la probabilità che un valore x sia **compreso** fra un determinato intervallo $[-x, x]$ intorno al valor medio. La funzione complementare mi dice qual è l'area che sta **fuori** da questo intervallo.



La potenza non è fissa. Oscilla attorno a un valore medio.

La zona rossa è la zona “normale” dove la potenza è attorno alla media. Qui la comunicazione funziona bene.

Che cos'è il margine di fading?

Il margine di fading M (fade margin) è la **quantità di potenza in più rispetto alla soglia** che aggiungiamo per compensare le fluttuazioni del canale.

$$L_p(\text{dB}) = P_T - P_{TH} - M$$

L_p (dB) → perdita massima di propagazione ammessa

P_T → potenza trasmessa (in dBm)

PTH → potenza di soglia del ricevitore (in dBm)

M → margine di fading (in dB).

Il margine è una “zona di sicurezza”.

l'introduzione del margine di fading riduce la massima perdita di propagazione tollerabile e quindi riduce il raggio della cella, aumentando però l'affidabilità del collegamento.

Se M aumenta LP diminuisce e se posso tollerare meno perdita posso arrivare a una distanza più piccola: in sostanza **la cella si restringe**.

Definizione del margine di fading

$$M = \overline{P_r(d)} - P_{TH}$$

$P_r(d)$ = potenza media ricevuta (in dBm),

PTH = potenza di soglia (in dBm),

M = margine di fading (in dB).

Il margine è quanti dB sopra la soglia sta la potenza media ricevuta.

Se negativo sono sotto soglia.

Regola mentale potente

- Outage < 50% → margine positivo
- Outage = 50% → margine zero
- Outage > 50% → margine negativo

Teorema: **se la potenza di soglia è uguale alla potenza ricevuta l'outage è del 50%.**

Adesso basta.

Okay adesso abbiamo tutto per svolgere le prove d'esame.

Mi scuso se alcune volte sono stato pressoché sbrigativo.

Prova d'esame del 30-07-2019

Se siamo a **400m** di distanza dalla stazione radio base c'è un outage del **2%**.
Se siamo a **1000m** di distanza dalla stazione radio base c'è un outage dell'**80%**.

La potenza di soglia del ricevitore P_{th} è maggiore rispetto alla potenza ricevuta P_r .

In questo scenario, calcolare il coefficiente di attenuazione, sapendo che $\sigma = 5$ dB.

Svolgimento.

Adesso calcoliamo il margine di fading a 400m, ossia M_{400} .

$$\frac{1}{2} \operatorname{erfc} \left(\frac{M_{400}}{\sqrt{2} \sigma} \right) = 0.02$$

Questa equazione esprime la probabilità di outage del 2% tramite la funzione errore complementare.

$$\operatorname{erfc} \left(\frac{M_{400}}{\sqrt{2} \sigma} \right) = 0.04$$

Di seguito la tabella Erfc alla meno 1:

	0,00	0,01	0,02	0,03	0,04	0,05	0,06	0,07	0,08	0,09
0,0		1,8214	1,6450	1,5345	1,4522	1,3859	1,3299	1,2812	1,2379	1,1988
0,1	1,1631	1,1301	1,0994	1,0706	1,0435	1,0179	0,9935	0,9703	0,9481	0,9267
0,2	0,9062	0,8864	0,8673	0,8488	0,8308	0,8134	0,7965	0,7800	0,7639	0,7482
0,3	0,7329	0,7179	0,7032	0,6888	0,6747	0,6609	0,6473	0,6339	0,6208	0,6078
0,4	0,5951	0,5826	0,5702	0,5580	0,5460	0,5342	0,5224	0,5109	0,4994	0,4881
0,5	0,4769	0,4659	0,4549	0,4441	0,4333	0,4227	0,4121	0,4017	0,3913	0,3810
0,6	0,3708	0,3607	0,3506	0,3406	0,3307	0,3209	0,3111	0,3013	0,2917	0,2820
0,7	0,2725	0,2629	0,2535	0,2440	0,2347	0,2253	0,2160	0,2067	0,1975	0,1883
0,8	0,1791	0,1700	0,1609	0,1518	0,1428	0,1337	0,1247	0,1157	0,1068	0,0978
0,9	0,0889	0,0799	0,0710	0,0621	0,0532	0,0443	0,0355	0,0266	0,0177	0,0089

Di seguito erfc:

ERFC	0,00	0,01	0,02	0,03	0,04	0,05	0,06	0,07	0,08	0,09
0,0	1,000000	0,988717	0,977435	0,966159	0,954889	0,943628	0,932378	0,921142	0,909922	0,898719
0,1	0,887537	0,876377	0,865242	0,854133	0,843053	0,832004	0,820988	0,810008	0,799064	0,788160
0,2	0,777297	0,766478	0,755704	0,744977	0,734300	0,723674	0,713100	0,702582	0,692120	0,681717
0,3	0,671373	0,661092	0,650874	0,640721	0,630635	0,620618	0,610670	0,600794	0,590991	0,581261
0,4	0,571608	0,562031	0,552532	0,543113	0,533775	0,524518	0,515345	0,506255	0,497250	0,488332
0,5	0,479500	0,470756	0,462101	0,453536	0,445061	0,436677	0,428384	0,420184	0,412077	0,404064
0,6	0,396144	0,388319	0,380589	0,372954	0,365414	0,357971	0,350623	0,343372	0,336218	0,329160
0,7	0,322199	0,315334	0,308567	0,301896	0,295322	0,288844	0,282463	0,276178	0,269990	0,263897
0,8	0,257899	0,251997	0,246189	0,240476	0,234857	0,229332	0,223900	0,218560	0,213313	0,208157
0,9	0,203092	0,198117	0,193232	0,188436	0,183729	0,179109	0,174576	0,170130	0,165768	0,161492
1,0	0,157299	0,153190	0,149162	0,145216	0,141350	0,137564	0,133856	0,130227	0,126674	0,123197
1,1	0,119795	0,116467	0,113212	0,110029	0,106918	0,103876	0,100904	0,098000	0,095163	0,092392
1,2	0,089686	0,087044	0,084466	0,081950	0,079495	0,077100	0,074764	0,072486	0,070266	0,068101
1,3	0,065992	0,063937	0,061935	0,059985	0,058086	0,056238	0,054439	0,052688	0,050984	0,049327
1,4	0,047715	0,046148	0,044624	0,043143	0,041703	0,040305	0,038946	0,037627	0,036346	0,035102
1,5	0,033895	0,032723	0,031587	0,030484	0,029414	0,028377	0,027372	0,026397	0,025453	0,024538
1,6	0,023652	0,022793	0,021962	0,021157	0,020378	0,019624	0,018895	0,018190	0,017507	0,016847
1,7	0,016210	0,015593	0,014997	0,014422	0,013865	0,013328	0,012810	0,012309	0,011826	0,011359
1,8	0,010909	0,010475	0,010057	0,009653	0,009264	0,008889	0,008528	0,008179	0,007844	0,007521
1,9	0,007210	0,006910	0,006622	0,006344	0,006077	0,005821	0,005574	0,005336	0,005108	0,004889
2,0	0,004678	0,004475	0,004281	0,004094	0,003914	0,003742	0,003577	0,003418	0,003266	0,003120

$$M_{400} = \operatorname{erfc}^{-1}(0.04) \sqrt{2} \sigma$$

$$\operatorname{erfc}^{-1}(0.04) = 1.4522$$

$$M_{400} = 1.4522 \cdot \sqrt{2} \cdot 5$$

$$M_{400} \approx 10.27 \text{ dB}$$

Questo è il margine di fading a 400 m con outage del 2% e $\sigma = 5$ dB.

La potenza media ricevuta è **10.27 dB sopra la soglia**.

Qui non conosco la potenza di soglia.

Ma siamo sopra soglia.

Adesso calcoliamo il margine di fading a 1000m, ossia M_{1000} .

M_{1000} è sotto soglia - ma per forza ho outage dell'80% - quindi sarà negativa.

$$\frac{1}{2} \operatorname{erfc} \left(\frac{M_{1000}}{\sqrt{2}\sigma} \right) = 0.20$$

$$M_{1000} \approx 4.21 \text{ dB}$$

$$M_{1000} \approx -4.21 \text{ dB}$$

$$\overline{P_r(400m)} - \overline{P_r(1000m)} = 14.48 \text{ dB}$$

$$L_p[400 \rightarrow 1000] = 10n \log_{10} \left(\frac{1000}{400} \right)$$

$$L_p[400 \rightarrow 1000] = 14.48 \text{ dB}$$

$$n \approx 3.64$$

Se ti allontani $\rightarrow$ perdita positiva

Se ti avvicini $\rightarrow$ perdita negativa (cioè recuperi potenza).

Gli esercizi sulla propagazione radiomobile si svolgono tutti così.